The Erronatus Blueprint

AI Automation with OpenClaw

PERSONAL EDITION

© 2026 Erronatus · erronatus.com · All rights reserved
Do not redistribute without permission

Contents

01 Quick Start

02 Connect Your Channel

03 Model Routing

04 File Management

05 Memory System

06 Skills System

07 First Automations

08 Calendar & Productivity

09 Deployment

10 Troubleshooting & Appendix

CHAPTER 01

Quick Start

Install OpenClaw in 15 Minutes

Chapter 1: Quick Start — Install OpenClaw in 15 Minutes

Welcome to the future of AI automation. In the next 15 minutes, you'll transform your computer into an AI powerhouse capable of automating tasks, managing communications, and executing complex workflows without any programming knowledge.

OpenClaw isn't just another AI chatbot—it's a complete automation platform that gives you an AI agent with real capabilities. By the end of this chapter, you'll have a working AI assistant that can search the web, manage files, send messages, and grow more powerful as you add new skills.

What You Need Before Starting

Before we dive in, let's make sure you have everything needed for a smooth installation.

Hardware Requirements

Minimum specs: - **RAM:** 4GB (8GB recommended for local models) - **Storage:** 2GB free space (10GB+ recommended for local models) - **Internet:** Stable broadband connection - **CPU:** Any modern processor (2015+)

For local AI models (optional but recommended): - **RAM:** 16GB+ (for running large models locally) - **GPU:** Optional but helpful for faster inference - **Storage:** 20GB+ free space

Operating System Support

OpenClaw works on all major operating systems:

- **Windows 10/11** (WSL2 strongly recommended)
- **macOS 10.15+** (Monterey or newer preferred)

- **Linux** (Ubuntu 20.04+, Fedora, CentOS, Debian)

Network Requirements

- Outbound HTTPS (port 443) for AI providers and updates
- Port 18789 available locally for the OpenClaw gateway
- SSH access if you plan to use remote servers

Step 1: Installing Node.js

OpenClaw requires Node.js 22 or newer. Let's get it installed on your system.

Check Your Current Version

First, check if you already have Node.js installed:

```
node --version
```

If this returns `v22.x.x` or higher, you can skip to Step 2. Otherwise, continue with the installation for your operating system.

Windows Installation

For Windows users, we **strongly recommend using WSL2** (Windows Subsystem for Linux). This provides the best compatibility and performance.

Option A: Install WSL2 + Ubuntu (Recommended)

1. Open PowerShell as Administrator and run:

```
wsl --install
```

2. Restart your computer when prompted.
3. Launch Ubuntu from the Start menu and create a user account.
4. Enable systemd (required for OpenClaw services):

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true
EOF
```

5. Restart WSL from PowerShell:

```
wsl --shutdown
```

6. Reopen Ubuntu and install Node.js:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Option B: Native Windows (Not Recommended)

If you prefer native Windows installation:

Using winget:

```
winget install OpenJS.NodeJS.LTS
```

Using Chocolatey:

```
choco install nodejs-lts
```

macOS Installation

Using Homebrew (Recommended)

```
brew install node
```

Using the Official Installer

Download and run the macOS installer from nodejs.org.

Linux Installation

Ubuntu/Debian:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Fedora/RHEL:

```
sudo dnf install nodejs npm
```

Alternative: Using a Version Manager

For developers who work with multiple Node.js versions, consider using a version manager like `fnm`:

```
# Install fnm
curl -fsSL https://fnm.vercel.app/install | bash

# Install and use Node.js 22
fnm install 22
fnm use 22
```

Verify Installation

Confirm Node.js is properly installed:

```
node --version
npm --version
```

You should see version 22 or higher for Node.js and version 9+ for npm.

Step 2: Installing OpenClaw

Now that Node.js is ready, let's install OpenClaw using the recommended installer script.

Quick Installation (Recommended)

The installer script handles everything automatically—Node.js detection, OpenClaw installation, and initial setup.

macOS/Linux/WSL2:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Windows PowerShell (Native):

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

This script will: - Verify your Node.js installation - Install OpenClaw globally via npm - Launch the setup wizard - Install the gateway service

Manual Installation

If you prefer to install manually or the script doesn't work for your setup:

```
npm install -g openclaw\@latest
```

Verify Installation

Check that OpenClaw installed correctly:

```
openclaw --version
```

You should see the current version number. If you get “command not found,” see the troubleshooting section at the end of this chapter.

Step 3: Running the Setup Wizard

OpenClaw includes a comprehensive setup wizard that configures everything you need. Let’s run it:

```
openclaw onboard --install-daemon
```

The wizard will walk you through several important choices:

QuickStart vs Advanced Setup

- **QuickStart:** Uses sensible defaults, perfect for beginners
- **Advanced:** Full control over every setting

Choose **QuickStart** for your first installation.

Model Provider Selection

You’ll be prompted to choose your AI provider. You have several options:

1. **OpenRouter (Cloud, Paid):** Access to all major AI models with pay-per-use pricing
2. **OpenAI (Cloud, Paid):** Direct access to GPT models
3. **Anthropic (Cloud, Paid):** Access to Claude models
4. **Ollama (Local, Free):** Run open-source models on your computer

For this tutorial, we’ll set up **both** options—a paid provider for cloud access and Ollama for free local models.

Setting Up OpenRouter (Cloud Option)

OpenRouter provides access to dozens of AI models through a single API:

1. Visit openrouter.ai and create an account
2. Navigate to “Keys” and create a new API key
3. When the wizard asks for your API key, enter: `your-openrouter-api-key-here`
4. Choose `gpt-4o-mini` as your primary model (affordable and capable)

Workspace Configuration

The wizard will ask where to store your OpenClaw files:

- **Default:** `~/.openclaw/workspace` (recommended)
- **Custom:** Any folder you prefer

Accept the default unless you have a specific reason to change it.

Gateway Settings

The gateway is OpenClaw's core service. Accept these defaults:

- **Port:** 18789
- **Bind mode:** loopback (127.0.0.1)
- **Auth:** Token-based (automatically generated)

Service Installation

When prompted about installing the gateway service:

- **macOS:** Creates a LaunchAgent for automatic startup
- **Linux/WSL2:** Creates a systemd user service
- **Windows:** Service installation varies

Choose **Yes** to install the service.

Step 4: Setting Up Ollama for Free Local Models

Ollama lets you run powerful AI models locally for free. This is perfect for privacy-sensitive tasks or when you want to experiment without API costs.

Install Ollama

Visit ollama.ai and download the installer for your operating system, or use the command line:

macOS:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Linux:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Windows: Download the installer from ollama.ai

Download Your First Model

Let's download `qwen2.5:14b`, an excellent free model that's great for automation tasks:

```
ollama pull qwen2.5:14b
```

This will download approximately 8GB of model data. While it downloads, let's continue with the setup.

Alternative Models

If you have limited disk space or RAM, consider these smaller options:

For 8GB+ RAM:

```
ollama pull qwen2.5:7b
```

For 4GB RAM:

```
ollama pull qwen2.5:3b
```

For coding tasks:

```
ollama pull qwen2.5-coder:14b
```

Configure OpenClaw to Use Ollama

Enable Ollama in OpenClaw:

```
export OLLAMA_API_KEY="ollama-local"
openclaw config set models.providers.ollama.apiKey "ollama-local"
```

Set Ollama as your default model for cost-saving automation:

```
openclaw config set agents.defaults.model.primary "ollama/qwen2.5:14b"
```

Verify Ollama Integration

Check that OpenClaw can see your Ollama models:

```
ollama list
openclaw models list
```

You should see your downloaded models in both lists.

Step 5: Verifying Your Installation

Let's make sure everything is working correctly.

Check OpenClaw Status

```
openclaw status
```

You should see: - **Runtime:** running - **Gateway:** healthy - **RPC probe:** ok

Run the Doctor

OpenClaw includes a diagnostic tool that checks for common issues:

```
openclaw doctor
```

This will scan your configuration and report any problems that need attention.

Detailed Status Check

For a comprehensive health check:

```
openclaw gateway status --deep
```

Test Model Access

Verify you can reach your AI models:

```
openclaw models test ollama/qwen2.5:14b
```

If you configured a cloud provider:

```
openclaw models test gpt-4o-mini
```

Step 6: Starting the Gateway

The gateway is OpenClaw's central service that coordinates everything. Let's make sure it's running.

Start the Gateway Service

If you installed the daemon service during setup:

macOS:

```
openclaw gateway status
```

If it's not running:

```
openclaw gateway start
```

Linux/WSL2:

```
systemctl --user status openclaw-gateway
```

If it's not running:

```
systemctl --user start openclaw-gateway
```

Manual Gateway Start

You can also run the gateway manually (useful for testing):

```
openclaw gateway --port 18789
```

Press Ctrl+C to stop it.

Verify Gateway Health

```
openclaw gateway status
```

Look for: - **Status:** running - **Port:** 18789 - **Health:** ok

Step 7: Your First Message

Time to test your AI agent! OpenClaw provides several ways to interact with your agent.

Using the Control UI (Easiest)

Open the web interface:

```
openclaw dashboard
```

This opens `http://127.0.0.1:18789/` in your browser. You'll see:

1. A login prompt (use the token from your config)
2. The Control UI chat interface
3. Your agent ready to chat

Try sending: "Hello! What can you do?"

Using the Command Line

Send a quick test message:

```
openclaw message send --message "Hello from the command line!" --target control-ui
```

Configuration Check

View your current configuration:

```
openclaw config show
```

This displays your complete setup including models, providers, and gateway settings.

Step 8: Understanding What You Built

Congratulations! You now have a complete AI automation platform. Let's understand what you've created.

Architecture Overview

Your OpenClaw installation consists of several key components:

1. **Gateway Service:** The central hub that coordinates everything
2. **AI Providers:** Your connection to AI models (cloud and local)
3. **Control UI:** Web interface for direct interaction
4. **CLI Tools:** Command-line interface for automation
5. **Workspace:** Your agent's file system and memory

Component Breakdown

Gateway (Port 18789): - Manages all AI interactions - Routes messages between channels and models - Handles authentication and security - Provides APIs for external integrations

Model Providers: - **Cloud providers** (OpenRouter/OpenAI/Anthropic) for powerful, latest models - **Ollama** for free, private, local inference - **Automatic failover** between providers

Workspace (`~/.openclaw/workspace`): - Agent memory and personality files - Skill definitions and custom tools - Session logs and conversation history - Configuration and credentials

Security Model

Your OpenClaw installation implements several security layers:

- **Token-based authentication** for gateway access
- **Local-first architecture** keeps sensitive data on your machine
- **Sandboxed execution** for running commands and tools
- **Encrypted credential storage** for API keys

What's Next

Your agent starts with basic capabilities but can grow much more powerful:

- **Skills:** Pre-built automation packages
- **Tools:** Web search, file management, system integration
- **Channels:** Connect to messaging platforms (Telegram, Discord, WhatsApp)

- **Remote access:** Access your agent from anywhere

Troubleshooting

If you encounter issues during installation, here are the most common problems and solutions.

“openclaw: command not found”

This means npm’s global binary directory isn’t in your PATH.

Diagnosis:

```
npm prefix -g
echo \$PATH
```

Solution (macOS/Linux): Add this to your `~/.bashrc` or `~/.zshrc`:

```
export PATH="\$(npm prefix -g)/bin:\$PATH"
```

Then open a new terminal.

Solution (Windows): Add the output of `npm prefix -g` to your system PATH through Windows Settings.

Gateway Won’t Start

Check port availability:

```
lsof -i :18789
```

If another process is using port 18789:

```
openclaw gateway --port 18790
```

Check configuration:

```
openclaw doctor
openclaw gateway status
```

View logs:

```
openclaw logs --follow
```

Permission Errors

Linux/macOS npm permissions:

```
mkdir -p "\$HOME/.npm-global"
npm config set prefix "\$HOME/.npm-global"
export PATH="\$HOME/.npm-global/bin:\$PATH"
```

Add the export line to your shell startup file.

Ollama Connection Issues

Check if Ollama is running:

```
ollama serve
```

Test Ollama API:

```
curl http://localhost:11434/api/tags
```

Verify OpenClaw can reach Ollama:

```
openclaw models test ollama/qwen2.5:14b
```

Model Download Failures

Check disk space:

```
df -h
```

Large models need 10-20GB free space.

Clear Ollama cache if needed:

```
ollama rm qwen2.5:14b
ollama pull qwen2.5:14b
```

Cloud Provider Authentication

Test API key manually:

For OpenRouter:

```
curl -H "Authorization: Bearer your-api-key-here" \
  https://openrouter.ai/api/v1/models
```

Update API key:

```
openclaw configure --section models
```

WSL2-Specific Issues

Enable systemd (required):

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true
EOF
```

Then restart WSL:

```
wsl --shutdown
```

Fix DNS issues:

```
sudo rm /etc/resolv.conf
sudo bash -c 'echo nameserver 8.8.8.8 > /etc/resolv.conf'
```

Configuration Reset

If everything seems broken:

Reset configuration (keeps workspace):

```
openclaw configure --reset
```

Fresh start (removes everything):

```
openclaw configure --reset-scope full
```

Getting Help

Built-in diagnostics:

```
openclaw doctor
openclaw status --all
openclaw logs --tail 50
```

Configuration debugging:

```
openclaw config show
openclaw config validate
```

Community resources: - GitHub Issues: github.com/openclaw/openclaw - Documentation: docs.openclaw.ai - Discord Community: discord.gg/openclaw

What You've Accomplished

In just 15 minutes, you've built a complete AI automation platform:

Γ£à **Installed OpenClaw** with proper Node.js dependencies
Γ£à **Configured both cloud and local AI models** for flexibility and cost control
Γ£à **Set up the gateway service** for 24/7 operation
Γ£à **Tested your installation** with real AI interactions
Γ£à **Learned the architecture** and component relationships
Γ£à **Built troubleshooting skills** for common issues

Your AI agent is now ready to learn, grow, and automate tasks for you. In the next chapter, we'll explore how to connect it to messaging platforms, add new capabilities with skills, and create your first automation workflows.

The foundation is solid. Now let's make it powerful.

CHAPTER 02

Connect Your Channel

Telegram & Discord Setup

Chapter 2: Connect Your Channel — Telegram & Discord Setup

The bridge between you and your digital workforce

Why You Need a Messaging Channel

Think of OpenClaw as a powerful computer sitting in your home office. It can research markets, write code, automate tasks, and manage your digital life — but how do you talk to it when you're not sitting at your desk?

That's where messaging channels come in. They're the communication bridge that lets you interact with your OpenClaw agent from anywhere: your phone during commute, your laptop at a coffee shop, even while traveling abroad. Whether you need to check on a running automation, ask for market research, or have OpenClaw write an email, your messaging channel is always available.

OpenClaw supports many platforms — WhatsApp, Telegram, Discord, Signal, Slack, and more. In this chapter, we'll focus on the two most popular choices for personal use:

- **Telegram** (our primary recommendation): Fast, reliable, excellent for personal automation
- **Discord** (powerful alternative): Great for team collaboration and voice interactions

Both platforms offer: - Free bot hosting - Mobile and desktop apps - Excellent reliability - Rich features (file sharing, voice messages, buttons) - Global availability

By the end of this chapter, you'll have a direct line to your OpenClaw agent that works from anywhere in the world. No more being tied to your home computer — your digital assistant becomes truly portable.

Telegram Setup (Recommended)

Telegram is our top recommendation for personal OpenClaw use. It's fast, reliable, has excellent mobile apps, and setting up a bot is straightforward. Plus, Telegram bots can run 24/7 for free.

Prerequisites

Before we begin, ensure: - You have Telegram installed on your phone or desktop - OpenClaw is installed and running on your computer - You have basic familiarity with opening a chat and sending messages

Step 1: Create Your Telegram Bot

We'll use Telegram's official @BotFather to create your personal OpenClaw bot. Don't worry — this is completely free and takes just a few minutes.

1. **Open Telegram** on any device (phone, desktop, or web)
2. **Start a chat with @BotFather**
   - Search for `\@BotFather` in Telegram
   - **Important**: Make sure you're talking to the official @BotFather with the blue checkmark
   - The handle should be exactly `\@BotFather` — beware of imposters!
3. **Create your bot**
   - Send the command: `/newbot`
   - @BotFather will ask for a name for your bot. This is just a display name, so choose something memorable like "My OpenClaw Assistant"
   - Next, you need a username. This must be unique across all Telegram and end with "bot"
   - Example: `jacksonclaw_bot` or `myassistant_bot`
   - Try a few variations if your first choice is taken
4. **Save your bot token**
   - Once created, @BotFather will send you a message containing your bot token
   - It looks like: `123456789:ABCdefGHIjklMNOpqrsTUVwxyz`
   - **Copy this token immediately** — you'll need it in the next step
   - **Keep it secret** — this token controls your bot

Step 2: Get Your Chat ID

Your bot needs to know who you are so it can send messages back to you. We'll get your unique Telegram user ID.

Method 1: Using a Helper Bot (Recommended) 1. Search for `\@userinfobot` in Telegram 2. Send any message to @userinfobot 3. It will reply with your user information 4. Copy the number after "Id:" — this is your chat ID 5. Example: if it shows "Id: 123456789", your chat ID is `123456789`

Method 2: Using Your Bot (Alternative) 1. Start a chat with your newly created bot (use the username you chose) 2. Send any message (like "Hello") 3. The bot won't respond yet (we haven't configured OpenClaw), but the message is logged 4. We'll retrieve your ID from OpenClaw logs after configuration

Step 3: Configure OpenClaw for Telegram

Now we'll tell OpenClaw about your bot and set it up for secure communication.

Open your terminal/command prompt and run these commands:

Windows:

```
openclaw config set channels.telegram.enabled true --json
openclaw config set channels.telegram.botToken "your-bot-token-here" --json
openclaw config set channels.telegram.dmPolicy "pairing" --json
```

macOS/Linux:

```
openclaw config set channels.telegram.enabled true --json
openclaw config set channels.telegram.botToken "your-bot-token-here" --json
openclaw config set channels.telegram.dmPolicy "pairing" --json
```

Important Notes: - Replace `your-bot-token-here` with the actual token from @BotFather - Keep the quotes around the token - Use the exact commands above — they're designed to be copy-paste ready

Step 4: Start OpenClaw with Telegram Support

If OpenClaw is already running:

```
openclaw gateway restart
```

If OpenClaw isn't running:

```
openclaw gateway
```

You should see a message in the logs indicating that Telegram is connecting. Look for something like:

```
[info] Telegram: Starting with bot token ...abc
[info] Telegram: Connected successfully
```

Step 5: Pair Your Device

OpenClaw uses a pairing system for security. This ensures only you can talk to your personal assistant.

1. **Send a message to your bot**
   - Go back to Telegram and send any message to your bot (the one you created)
   - Example: "Hello OpenClaw!"
2. **Get the pairing code**
   - Your bot will respond with a pairing code
   - It looks like: "Pairing required. Code: ABC123"
   - Copy this code
3. **Approve the pairing**

Option A: Using another channel (if you have one set up):

- Message OpenClaw on any existing channel: "Approve this Telegram pairing code: ABC123"

Option B: Using the command line:

```
openclaw pairing approve telegram ABC123
```

Option C: Check pending pairings first:

```
openclaw pairing list telegram
openclaw pairing approve telegram ABC123
```

4. **Test the connection**
   - Send another message to your Telegram bot
   - This time, OpenClaw should respond normally
   - Try: "What's the weather like today?"
   - If you get a helpful response, congratulations! Telegram is working

Step 6: Verify Two-Way Communication

Let's make sure everything is working perfectly:

1. **Send a simple question**: "What's 15 + 27?"
2. **Request a task**: "Create a 3-item todo list for weekend planning"
3. **Check status**: "/status"

If all of these work, your Telegram setup is complete!

Discord Setup (Alternative)

Discord offers rich features like voice channels, server organization, and excellent desktop integration. While slightly more complex to set up than Telegram, it's perfect if you want team collaboration or advanced features.

Prerequisites

- Discord account (free)
- Basic familiarity with Discord servers and channels
- OpenClaw installed and running

Step 1: Create a Discord Application

Discord bots require an "application" in their Developer Portal. This is like registering your bot with Discord.

1. **Visit the Discord Developer Portal**
   - Go to: https://discord.com/developers/applications
   - Sign in with your Discord account

2. **Create a new application**
    - Click “New Application”
    - Name it something like “OpenClaw” or “My AI Assistant”
    - Accept Discord’s terms
    - Click “Create”
3. **Navigate to the Bot section**
    - On the left sidebar, click “Bot”
    - Click “Add Bot” if prompted
    - Customize the username if desired (this is what appears in Discord)

Step 2: Configure Bot Permissions

Your bot needs specific permissions to function properly.

1. **Enable required intents**
    - Scroll down to “Privileged Gateway Intents”
    - Enable “Message Content Intent” Γ£à (Required)
    - Enable “Server Members Intent” Γ£à (Recommended)
    - Optionally enable “Presence Intent” (for status updates)
2. **Get your bot token**
    - Scroll back up to the “Token” section
    - Click “Reset Token” (this creates your first token)
    - Copy the token immediately — **this is your Discord bot token**
    - Save it securely; you’ll need it for OpenClaw configuration

Step 3: Create a Test Server (Recommended)

For security and privacy, we recommend creating a private server just for you and your OpenClaw bot.

1. **Create a new Discord server**
    - In Discord, click the “+” icon in the server list
    - Choose “Create My Own”
    - Select “For me and my friends”
    - Name it something like “OpenClaw Workspace”
    - Skip the template selection
2. **Get your server ID**
    - Right-click on your server name
    - If you don’t see “Copy Server ID”, enable Developer Mode:
        - Settings (gear icon) ΓåÆ Advanced ΓåÆ Developer Mode (toggle on)
    - Right-click server name ΓåÆ “Copy Server ID”
    - Save this number — it’s your server ID
3. **Get your user ID**
    - Right-click on your own username/avatar
    - Select “Copy User ID”
    - Save this number — it’s your user ID

Step 4: Generate Bot Invite Link and Add to Server

Now we’ll create a special link to invite your bot to your server with the right permissions.

1. **Generate the invite URL**
    - In Developer Portal, click “OAuth2” in the sidebar
    - Scroll to “OAuth2 URL Generator”
2. **Select scopes**
    - Check “bot” Γ£à
    - Check “applications.commands” Γ£à
3. **Select bot permissions** When you check “bot”, a permissions section appears. Enable:
    - View Channels Γ£à
    - Send Messages Γ£à
    - Read Message History Γ£à
    - Embed Links Γ£à
    - Attach Files Γ£à
    - Add Reactions Γ£à (optional)
4. **Invite the bot**
    - Copy the generated URL at the bottom
    - Open it in a new browser tab
    - Select your server from the dropdown
    - Click “Continue” and “Authorize”
    - Complete any captcha if prompted
5. **Verify bot joined**
    - Go back to Discord
    - You should see your bot in the member list (offline for now)

Step 5: Configure OpenClaw for Discord

First, set your bot token securely:

Windows:

```
openclaw config set channels.discord.enabled true --json
openclaw config set channels.discord.token "your-discord-bot-token-here" --json
openclaw config set channels.discord.dmPolicy "pairing" --json
```

macOS/Linux:

```
openclaw config set channels.discord.enabled true --json
openclaw config set channels.discord.token "your-discord-bot-token-here" --json
openclaw config set channels.discord.dmPolicy "pairing" --json
```

Important Security Note: Never share your bot token in chat. Set it through the command line as shown above.

Step 6: Enable DMs from Server Members

For pairing to work, Discord needs to allow your bot to send you direct messages.

1. **Enable DMs from your server**
    - Right-click your server icon
    - Select “Privacy Settings”

- Toggle “Direct Messages” to ON
- This allows server members (including bots) to DM you

Step 7: Restart OpenClaw and Pair

1. **Restart the gateway**

```
openclaw gateway restart
```

2. **Check the logs**
   - Look for Discord connection messages
   - You should see: “Discord: Connected as YourBotName#1234”
3. **Send a DM to your bot**
   - In Discord, find your bot in the server member list
   - Click on it and select “Message”
   - Send any message like “Hello!”
4. **Get the pairing code**
   - Your bot will respond with a pairing code
   - Copy the code (format: ABC123)
5. **Approve the pairing**

   Option A: Via another channel (if available):
   - Message OpenClaw: “Approve this Discord pairing code: ABC123”

   Option B: Command line:

```
openclaw pairing approve discord ABC123
```

Step 8: Set Up Your Discord Workspace (Recommended)

For private servers, you’ll want to allow your bot to respond in channels without being @mentioned.

1. **Add your server to the allowlist**

   Send this message to your OpenClaw agent (via any channel):

```
Add my Discord Server ID 123456789012345678 to the guild allowlist
```

   (Replace with your actual server ID)
2. **Allow responses without @mention**

   For a private server, you want the bot to respond to every message:

```
Allow my agent to respond on this server without having to be \@mentioned
```

3. **Create organized channels**

- Create channels like #coding, #research, #automation
- Each channel gets its own conversation context
- Perfect for organizing different projects

Step 9: Test Discord Features

Discord offers unique features worth testing:

1. **Basic chat**: Send a message in any channel
2. **Slash commands**: Try `/status` or `/help`
3. **Voice messages**: Send an audio message (OpenClaw can transcribe it)
4. **File uploads**: Share documents for analysis

Slash Commands — Your Agent's Control Panel

Both Telegram and Discord support "slash commands" — special commands that start with `/` and provide quick access to OpenClaw features. Think of them as keyboard shortcuts for your AI assistant.

Essential Commands Everyone Should Know

Status and Information: - `/status` — Check your agent's current status, model, and system health - `/help` — List all available commands - `/whoami` — Shows your user ID (useful for troubleshooting) - `/context` — Explain what information your agent currently has access to

Model and Behavior Control: - `/model` — Switch between AI models (GPT-4, Claude, local models) - `/thinking high` — Enable detailed reasoning (great for complex problems) - `/verbose on` — Show detailed tool output (helpful for debugging) - `/reasoning on` — See the agent's reasoning process

Session Management: - `/reset` — Start a fresh conversation (clear context) - `/new` — Same as reset, with optional model change - `/activation mention` — (Groups only) Require @mention to respond - `/activation always` — (Groups only) Respond to all messages

Agent Control: - `/subagents list` — See any background tasks running - `/stop` — Stop the current response or task - `/focus <target>` — (Discord only) Bind this thread to a specific agent or task

Advanced Commands for Power Users

System Configuration: - `/config show` — View current configuration - `/debug show` — View runtime overrides - `/usage tokens` — Show token usage with each response - `/tts always` — Enable text-to-speech for all responses

Task Management: - `/kill <id>` — Force stop a specific background task - `/steer <id> <message>` — Redirect a running task - `/export-session` — Export conversation to HTML file

Platform-Specific: - `/vc join` — (Discord only) Join voice channel for live conversation - `/dock-telegram` — Switch responses to Telegram - `/dock-discord` — Switch responses to Discord

Using Commands Effectively

Quick Inline Commands: Some commands work inline with regular messages:

```
Hey /status ΓÇö tell me about today's weather
```

This runs `/status` and then asks about weather.

Command-Only Messages: Commands work fastest when sent alone:

```
/thinking high
/model claude
/verbose on
```

Combining Commands: You can chain some commands:

```
/model claude /thinking high /verbose on ΓÇö now analyze this data...
```

Group vs. Direct Message Commands

In Direct Messages: - All commands work normally - No special permissions needed - Commands affect just your conversation

In Group Chats: - Most commands require @mention unless configured otherwise - Commands like `/activation` affect the entire group - Some commands (like `/reasoning on`) should be used carefully as they expose internal thinking

Multi-Device Access — Your Agent Everywhere

One of the biggest advantages of using Telegram or Discord is seamless access across all your devices. Your OpenClaw agent follows you everywhere.

Telegram Multi-Device Experience

Automatic Sync: - Send a message from your phone, continue on desktop - Full conversation history syncs instantly - File uploads available on all devices

Platform Apps: - **Phone**: iOS/Android apps (fast, push notifications) - **Desktop**: Windows/macOS/Linux native apps - **Web**: telegram.org/k (works in any browser) - **iPad/Tablet**: Optimized mobile apps

Practical Scenarios: - Start a research request on your phone during commute - Continue the conversation on desktop when you arrive - Check automation status from your tablet in bed - Emergency requests work from any device with internet

Discord Multi-Device Experience

Rich Desktop Integration: - Native Windows/macOS apps with system notifications - Web version works seamlessly in browsers - Mobile apps for iOS/Android

Advanced Features Across Devices: - Voice channels work on mobile and desktop - Screen sharing (great for getting visual help) - File drag-and-drop support - Rich message formatting and embeds

Organized Workspaces: - Create channels for different projects (#coding, #research, #personal) - Each channel maintains its own conversation context - Voice channels for real-time collaboration - Thread conversations for complex topics

Best Practices for Multi-Device Use

Message Continuity: - Both platforms maintain perfect message sync - Start conversations anywhere, continue anywhere - File attachments sync automatically

Context Awareness: - OpenClaw remembers your conversation across device switches - No need to repeat yourself when switching platforms - Long-term memory persists regardless of which device you use

Notification Management: - Configure notifications to avoid interruption during focused work - Use Telegram's "Silent mode" for non-urgent automations - Discord's notification settings let you customize per-channel alerts

Choosing Between Telegram and Discord

Both platforms are excellent choices, but each has unique strengths. Here's how to decide what's best for your use case.

Choose Telegram If:

Simplicity is key: - You want the fastest, most straightforward setup - You prefer minimal configuration and maximum reliability - You're primarily using OpenClaw for personal automation

Mobile-first usage: - You frequently interact with your agent from your phone - You travel often and need reliable global access - You prefer lightweight, fast mobile apps

Privacy-conscious: - You appreciate Telegram's strong encryption options - You want a platform built with privacy in mind - You prefer decentralized, less corporate platform

Personal automation focus: - You're using OpenClaw for individual productivity - You mainly need basic chat, file sharing, and notifications - You don't need complex team collaboration features

Choose Discord If:

Rich collaboration features: - You plan to use OpenClaw in team environments - You want organized workspaces with multiple channels - You need voice channels for real-time AI interaction

Advanced integrations: - You use Discord for other purposes (gaming, communities) - You want rich embeds, buttons, and interactive interfaces - You plan to build custom bots or integrations

Desktop-heavy workflow: - You primarily work from desktop computers - You need powerful desktop features like screen sharing - You prefer rich text formatting and embed previews

Complex project organization: - You manage multiple projects that need separate contexts - You want threaded conversations for complex topics - You need moderation and permission management

The Hybrid Approach

You don't have to choose just one! Many users set up both:

- **Telegram for mobile**: Quick requests, status checks, urgent notifications
- **Discord for deep work**: Complex projects, voice interactions, team collaboration

OpenClaw supports both simultaneously. You can: - Start a conversation on Telegram, continue on Discord - Use `/dock-telegram` or `/dock-discord` to switch response channels - Set up different automation for each platform

Migration and Setup Time

Telegram Setup Time: 5-10 minutes - Create bot via @BotFather - Configure OpenClaw - Pair device - Done!

Discord Setup Time: 15-20 minutes
- Create Discord application - Configure bot permissions - Set up server (optional but recommended) - Configure OpenClaw - Pair device - Optional: Set up organized channels

Troubleshooting Common Issues

Even with careful setup, you might encounter some bumps. Here are solutions to the most common problems.

Telegram Troubleshooting

Problem: Bot doesn't respond to my messages

```
Solutions:
1. Check pairing status: openclaw pairing list telegram
2. Verify bot token: openclaw config get channels.telegram.botToken
3. Check OpenClaw logs: openclaw logs --follow
4. Restart gateway: openclaw gateway restart
```

Problem: "Pairing required" message keeps appearing

```
Solutions:
1. Make sure you approved the correct code: openclaw pairing list telegram
2. Check if pairing expired (codes last 1 hour)
3. Generate new pairing: send another message to bot
4. Manual approval: openclaw pairing approve telegram <code>
```

Problem: Bot token invalid or expired

```
Solutions:
1. Regenerate token via \@BotFather:
   - /revoke (revokes old token)
   - /token (generates new token)
2. Update OpenClaw config with new token
3. Restart gateway
```

Problem: Can't find @BotFather

```
Solutions:
1. Search for exactly "\@BotFather" (with @)
2. Look for the blue verified checkmark
3. Avoid fake bots with similar names
4. Try from different Telegram client if needed
```

Discord Troubleshooting

Problem: Bot appears offline in server

```
Solutions:
1. Check bot token in config: openclaw config get channels.discord.token
2. Verify intents enabled in Discord Developer Portal
3. Check OpenClaw logs for connection errors
4. Restart gateway: openclaw gateway restart
```

Problem: Bot can't read messages in server

```
Solutions:
1. Enable "Message Content Intent" in Developer Portal
2. Grant "Read Messages" permission when inviting bot
3. Check if bot has access to the specific channel
4. Remove and re-add bot to server with correct permissions
```

Problem: Pairing fails or times out

```
Solutions:
1. Enable DMs from server members:
   - Right-click server ΓåÆ Privacy Settings ΓåÆ Direct Messages ON
2. Try pairing in a DM instead of server channel
3. Check if bot can send you DMs (test manually)
4. Verify your user ID is correct
```

Problem: Server commands work but DMs don't

```
Solutions:
1. Check DM policy: openclaw config get channels.discord.dmPolicy
2. Verify allowlist if using "allowlist" policy
3. Approve pairing if using "pairing" policy
4. Enable DMs in privacy settings
```

Problem: Slash commands not appearing

```
Solutions:
1. Verify native commands enabled: openclaw config get commands.native
2. Check bot permissions include "Use Slash Commands"
3. Restart Discord client (commands cache locally)
4. Wait a few minutes (Discord can be slow to sync)
```

General OpenClaw Issues

Problem: Gateway won't start

```
Solutions:
1. Check config syntax: openclaw doctor
2. View startup logs: openclaw logs --tail 50
3. Verify file permissions in ~/.openclaw/
4. Kill any stuck processes: openclaw gateway stop
```

Problem: Commands work but agent doesn't respond

```
Solutions:
1. Check model configuration: /model status
2. Verify API keys for your model provider
3. Check quota/rate limits: /status
4. Try switching models: /model <alternative>
```

Problem: Slow responses or timeouts

```
Solutions:
1. Check internet connection
2. Try local models for faster responses
3. Reduce request complexity
4. Check model provider status page
```

Problem: File uploads fail

```
Solutions:
1. Check file size limits (varies by platform)
2. Verify file type is supported
3. Try smaller files first
4. Check OpenClaw storage space
```

Getting More Help

If these solutions don't resolve your issue:

1. **Check OpenClaw logs**:

   ```
   openclaw logs --follow
   ```

2. **Run the doctor command**:

   ```
   openclaw doctor --fix
   ```

3. **Check system status**:

   ```
   openclaw status --verbose
   ```

4. **Review configuration**:

```
openclaw config show
```

5. **Community resources**:
    - OpenClaw documentation
    - GitHub issues and discussions
    - Community Discord/forums

What's Next?

Congratulations! You've successfully connected your first communication channel to OpenClaw. Your AI assistant is now portable, accessible from anywhere, and ready for serious work.

In the next chapter, we'll explore **Model Selection and Configuration** — learning how to choose the right AI brain for each task, manage costs effectively, and get the most intelligence for your dollar.

You'll discover: - How to switch between different AI models on demand - Cost-effective strategies for daily use vs. complex projects
- Local models for privacy and unlimited usage - Advanced reasoning modes for complex problems

Your communication bridge is built. Now let's optimize the intelligence that flows across it.

Ready to unleash your AI agent's full potential? Turn the pageΓÇª

CHAPTER 03

Model Routing

Run AI for (Almost) Free

Chapter 3: Model Routing & Token Optimization — Run AI for (Almost) Free

Master OpenClaw's model tier system to minimize costs while maximizing AI capabilities. Learn when to use free local models versus premium cloud models, and how to optimize your spend.

What Are AI Models? Understanding the Fundamentals

Before diving into cost optimization, let's understand what you're paying for. AI language models (LLMs) are massive neural networks trained on vast amounts of text data. Think of them as incredibly sophisticated prediction engines that understand and generate human language.

Key concepts every OpenClaw user should know:

- **Parameters**: The "neurons" in an AI model. More parameters usually mean better capabilities but higher costs. A 7-billion parameter model (7B) is smaller and cheaper than a 70-billion parameter model (70B).
- **Context Window**: How much text the model can "remember" in a single conversation. Larger context windows let you have longer conversations or analyze bigger documents, but cost more per token.
- **Tokens**: The units models use to process text. Roughly 4 characters = 1 token. "Hello world" is about 2 tokens. Every input and output token costs money with cloud models.
- **Inference Speed**: How fast the model generates responses. Local models are often slower but free. Cloud models are faster but cost per token.

Why size matters: Bigger models (more parameters) are generally smarter but cost more. A 3B local model might handle basic tasks perfectly, while you need a 175B+ cloud model for complex reasoning or specialized knowledge.

The key insight: **You don't always need the biggest, smartest model.** That's where OpenClaw's tier system comes in.

The Tier System: Your Cost Control Framework

OpenClaw organizes AI models into four cost tiers. This framework helps you choose the right model for each task, maximizing value while minimizing spend.

Tier 0: LOCAL ($0 per month)

Cost: Free after initial setup **Models**: Ollama-powered local models **Best for**: Background tasks, cron jobs, experimentation, privacy-sensitive work

Recommended models: - `ollama/qwen2.5:14b` — Excellent general capability, good at code - `ollama/qwen2.5-coder:14b` — Specialized for programming tasks - `ollama/mistral:7b` — Fast, efficient for simpler tasks - `ollama/llama3.2:3b` — Ultra-fast for basic automation

Pros: - Zero ongoing costs - Complete privacy (runs on your machine) - No rate limits or quotas - Perfect for automation and batch processing

Cons: - Requires local compute power - Slower than cloud models - May struggle with complex reasoning - Limited to your hardware capabilities

Tier 1: CHEAP (< $0.01 per conversation turn)

Cost: Under 1 cent per typical response **Models**: Budget cloud models via OpenRouter **Best for**: Casual conversation, simple Q&A, basic assistance

Recommended models: - `openrouter/deepseek-ai/deepseek-v3` — Exceptional value, very capable - `openrouter/openai/gpt-4o-mini` — Fast, reliable, good balance - `openrouter/google/gemini-flash-1.5` — Quick responses, good for chat

Typical costs: - DeepSeek V3: ~$0.002 per 1K tokens - GPT-4o mini: ~$0.003 per 1K tokens - Gemini Flash: ~$0.004 per 1K tokens

Tier 2: MODERATE ($0.01 - $0.05 per turn)

Cost: 1-5 cents per typical response **Models**: Mainstream production models **Best for**: Content creation, coding projects, analysis work

Recommended models: - `openrouter/google/gemini-pro` — Strong reasoning, good value - `openrouter/openai/gpt-4o` — Reliable, well-rounded performance - `openrouter/openai/chatgpt-4o-latest` — Latest OpenAI capabilities

Tier 3: PREMIUM ($0.05 - $0.15+ per turn)

Cost: 5-15+ cents per typical response **Models**: Top-tier frontier models **Best for**: Critical business work, complex reasoning, revenue-generating tasks

Recommended models: - `openrouter/anthropic/claude-3-5-sonnet` — Excellent reasoning and code - `openrouter/anthropic/claude-3-opus` — Most capable (but expensive) - `openrouter/openai/o1-preview` — Deep reasoning for complex problems

When to use Premium: - Building production applications - Complex analysis that affects business decisions - Tasks where accuracy is critical - When the work generated will create revenue

---

Setting Up Ollama for Free Local Inference

Ollama is your gateway to running powerful AI models locally at zero ongoing cost. Here's how to set it up across different operating systems:

Installation

Windows: 1. Download from https://ollama.ai 2. Run the installer 3. Open PowerShell or Command Prompt

macOS: 1. Download from https://ollama.ai 2. Drag to Applications folder 3. Open Terminal

Linux:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Pulling Your First Models

Once installed, pull the recommended models:

```
# Start with the most capable model
ollama pull qwen2.5:14b

# Add a specialized coding model
ollama pull qwen2.5-coder:14b

# Get a faster model for simple tasks
ollama pull mistral:7b

# Ultra-lightweight for automation
ollama pull llama3.2:3b
```

Storage note: These models are large files: - `qwen2.5:14b` ΓåÆ ~8GB - `mistral:7b` ΓåÆ ~4GB - `llama3.2:3b` ΓåÆ ~2GB

Make sure you have adequate disk space.

Configuring OpenClaw to Use Ollama

To enable Ollama in OpenClaw, you need to set up authentication (even though it's local):

```
# Set the environment variable (any value works)
export OLLAMA_API_KEY="ollama-local"
```

Or add it to your OpenClaw configuration:

```
{
  "models": {
    "providers": {
      "ollama": {
        "apiKey": "ollama-local",
        "baseUrl": "http://127.0.0.1:11434"
      }
    }
  }
}
```

Auto-discovery: When you set `OLLAMA_API_KEY`, OpenClaw automatically discovers all your Ollama models. Just pull a model with `ollama pull`, and it becomes available immediately.

Testing Your Setup

Verify everything works:

```
# Check installed models
ollama list

# Check OpenClaw can see them
openclaw models list

# Test a model directly
ollama run qwen2.5:14b "Hello! Can you help me code?"
```

When to Use Local vs Cloud Models

Use Local (Tier 0) for: - Cron jobs and scheduled tasks - Background monitoring and automation - Content that requires privacy - Experimental prompts and testing - Any task where cost is more important than speed

Switch to Cloud when: - You need the fastest possible responses - Local model gives unsatisfactory results - Working with complex, multi-step reasoning - Collaborating with others who need consistent results - Your local hardware can't handle the model size

Performance Optimization Tips

Hardware recommendations: - **Minimum**: 8GB RAM, can run 3B-7B models - **Recommended**: 16GB RAM, can run 14B models comfortably - **Ideal**: 32GB+ RAM, can run 30B+ models

Speed optimization: - Use smaller models for simple tasks - Keep frequently-used models loaded (`ollama run model_name` keeps it in memory) - Consider GPU acceleration if you have a compatible graphics card

Setting Up OpenRouter for Cloud Models

OpenRouter is your gateway to accessing dozens of AI models through a single API key. It's like having accounts with OpenAI, Anthropic, Google, and many others, but with unified billing and access.

Creating Your OpenRouter Account

1. Go to https://openrouter.ai
2. Sign up with email or GitHub
3. Navigate to the API Keys section
4. Generate a new API key

Important: Keep this key secure. It's your access to all paid models.

Adding Your API Key to OpenClaw

There are several ways to configure your OpenRouter key:

Method 1: Environment Variable (Recommended)

```
export OPENROUTER_API_KEY="sk-or-v1-your-key-here"
```

Method 2: OpenClaw Configuration Add to your `openclaw.json` config file:

```
{
  "env": {
    "OPENROUTER_API_KEY": "sk-or-v1-your-key-here"
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/deepseek-ai/deepseek-v3"
      }
    }
  }
}
```

Method 3: Using the CLI

```
openclaw onboard --auth-choice openrouter
```

Follow the prompts to enter your API key.

Understanding OpenRouter Pricing

OpenRouter operates on **per-token pricing**. You pay for exactly what you use:

- **Input tokens**: What you send to the model
- **Output tokens**: What the model sends back
- **No monthly fees**: Pure pay-as-you-go
- **Credit system**: Add money to your account, spend it down

Sample costs per 1K tokens: - DeepSeek V3: $0.002 input, $0.008 output - GPT-4o mini: $0.003 input, $0.012 output
- Claude 3.5 Sonnet: $0.03 input, $0.15 output

Calculating conversation costs: A typical conversation turn with GPT-4o mini: - 100 input tokens: $0.0003 - 200 output tokens: $0.0024 - **Total: $0.0027** (about 1/4 cent)

Funding Your Account

Add credits to your OpenRouter account: 1. Log into the OpenRouter dashboard 2. Go to Credits/Billing 3. Add funds via credit card or crypto 4. Start with $5-10 for experimentation

Pro tip: Enable usage alerts to get notified when you're spending more than expected.

Testing Your Setup

Verify OpenRouter integration:

```
# Check available models
openclaw models list | grep openrouter

# Test with a cheap model
openclaw models set openrouter/deepseek-ai/deepseek-v3

# Send a test message through OpenClaw
echo "Hello, world!" | openclaw chat
```

Model Aliases: Using Shortcuts

Instead of typing long model names like `openrouter/anthropic/claude-3-5-sonnet`, OpenClaw supports aliases for common models.

Built-in Aliases

OpenClaw includes these convenient shortcuts:

- `sonnet` ΓåÆ `openrouter/anthropic/claude-3-5-sonnet`
- `opus` ΓåÆ `openrouter/anthropic/claude-3-opus`
- `gpt` ΓåÆ `openrouter/openai/gpt-4o`
- `mini` ΓåÆ `openrouter/openai/gpt-4o-mini`
- `flash` ΓåÆ `openrouter/google/gemini-flash-1.5`
- `deepseek` ΓåÆ `openrouter/deepseek-ai/deepseek-v3`

Using Aliases in Chat

Switch models quickly during conversations:

```
/model deepseek
/model sonnet
/model flash
```

Creating Custom Aliases

Add your own shortcuts to the configuration:

```
{
  "agents": {
    "defaults": {
      "models": {
        "ollama/qwen2.5:14b": { "alias": "local" },
        "openrouter/deepseek-ai/deepseek-v3": { "alias": "cheap" },
        "openrouter/anthropic/claude-3-5-sonnet": { "alias": "smart" }
      }
    }
  }
}
```

Now you can use:

```
/model local
/model cheap
/model smart
```

CLI Alias Management

```
# List current aliases
openclaw models aliases list

# Add a new alias
openclaw models aliases add mymodel openrouter/some/model

# Remove an alias
openclaw models aliases remove mymodel
```

Dynamic Model Switching: The `/model` Command

OpenClaw makes it easy to switch models mid-conversation without losing context. This is crucial for cost optimization—start cheap, escalate when needed.

Basic Model Switching

```
/model                     # Show model picker
/model list               # List available models
/model 3                  # Select model #3 from the list
/model deepseek           # Switch to DeepSeek via alias
/model openrouter/anthropic/claude-3-5-sonnet  # Full model path
/model status             # Show current model and auth status
```

Strategic Switching Patterns

Pattern 1: Start Cheap, Escalate Smart

```
You: Help me write a Python function to sort a list
[Start with /model deepseek - costs ~\$0.002]

Claude: [Provides basic solution]

You: This needs error handling and type hints
[Switch to /model gpt - costs ~\$0.01 but more capable]

Claude: [Provides robust, production-ready solution]
```

Pattern 2: Local for Exploration, Cloud for Production

```
You: /model local
You: Brainstorm 10 app ideas for productivity
[Free local processing for creative tasks]

You: /model sonnet
You: Take idea #3 and create a detailed technical specification
[Premium model for critical planning]
```

Pattern 3: Model Stack for Complex Tasks

```
You: /model flash
You: Summarize this 50-page document
[Fast, cheap model for initial processing]

You: /model gpt
You: Based on that summary, identify the 5 key risks
[Better reasoning for analysis]

You: /model sonnet
You: Write a detailed mitigation plan for risk #1
[Premium model for critical business deliverable]
```

Session-Based Model Selection

Model changes apply to your current session. When you start a new conversation:

```
/new    # Start fresh session, model resets to default
/reset  # Clear current session, keep model selection
```

Model Override Syntax

For advanced users, you can specify models with additional parameters:

```
/model sonnet\@profile-id     # Use specific auth profile
/model gpt --max-tokens 1000 # Limit response length
```

Token Budgeting: Understanding and Tracking Your Costs

Tokens are the currency of AI interactions. Understanding them is key to controlling costs.

What Are Tokens?

Simple rule: ~4 characters = 1 token

Examples: - "Hello" = 1 token - "Hello, world!" = 3 tokens
- "The quick brown fox" = 4 tokens - This paragraph = ~50 tokens

Why it matters: Every token you send (input) and receive (output) costs money with cloud models. A 1,000-word document is roughly 750 tokens.

Context Windows Explained

Context window = How many tokens a model can "remember" in one conversation.

Common context windows: - Older models: 4K-8K tokens (~3,000-6,000 words) - Modern models: 32K-128K tokens (~24,000-96,000 words)
- Latest models: 200K+ tokens (~150,000+ words)

Cost implications: Larger context windows let you have longer conversations but cost more. A model with 32K context costs roughly 4x more than one with 8K context for the same conversation length.

Tracking Your Token Usage

OpenClaw provides several ways to monitor token consumption:

During conversations:

```
/status                 # Shows current session token count + estimated cost
/usage full             # Adds cost footer to every response
/usage tokens           # Shows token count only
/usage cost             # Local cost summary from session logs
/usage off              # Disables usage tracking display
```

Example /status output:

```
Model: openrouter/deepseek-ai/deepseek-v3
Session tokens: 1,247 input + 892 output = 2,139 total
Estimated cost: \$0.0091
Context window: 64,000 tokens (3.3% used)
```

CLI monitoring:

```
openclaw status --usage        # Full provider breakdown
openclaw channels list         # Shows usage alongside config
```

Calculating Costs

Formula: (Input tokens ├ù Input rate) + (Output tokens ├ù Output rate)

Example with Claude 3.5 Sonnet: - Input rate: $0.003 per 1K tokens - Output rate: $0.015 per 1K tokens - Your conversation: 500 input + 800 output tokens

Calculation: - Input cost: (500 ╠ù 1000) ├ù $0.003 = $0.0015 - Output cost: (800 ╠ù 1000) ├ù $0.015 = $0.012 - **Total: $0.0135** (~1.4 cents)

Budget Management Strategies

Daily Budget Approach: Set a spending limit like $2/day and track toward it: - Morning check-ins: Use local models - Important work: Budget $1 for premium models
- Evening automation: Back to local models - Track with `/usage cost` throughout the day

Project Budget Approach: Allocate model tiers by project value: - Personal projects: 90% local, 10% cheap cloud - Client work: 50% cheap cloud, 40% moderate, 10% premium - Critical deliverables: Use premium without hesitation

Token-Aware Prompting: - Be concise in your requests (fewer input tokens) - Ask for shorter responses when appropriate - Use follow-up questions instead of one massive prompt - Break complex tasks into smaller, focused requests

---

The 90/10 Rule: Maximizing Value

The 90/10 rule is the core philosophy of cost-effective AI usage: **90% of your AI interactions should use Tier 0-1 models, reserving premium models for the 10% that truly need them.**

Understanding the Math

Scenario: 100 AI interactions per day - **90 interactions** on Tier 0-1 (free to $0.01 each): $0-0.90/day - **10 interactions** on Tier 3 ($0.05-0.15 each): $0.50-1.50/day - **Total daily cost**: $0.50-2.40

Compare this to using premium models for everything: - **100 interactions** on Tier 3: $5.00-15.00/day - **Monthly difference**: $135-378 saved!

Applying the 90/10 Rule

Tier 0-1 Tasks (90% of usage): - Casual conversation and questions - Code reviews and basic debugging - Content brainstorming and drafts - Data formatting and simple transforms - Research and information gathering - Task planning and organizing - Email and message drafting - Learning and educational queries

Tier 3 Tasks (10% of usage): - Critical business decisions requiring deep analysis - Complex code architecture and system design - Revenue-generating content that must be perfect - Legal, financial, or medical advice verification - Sensitive client deliverables - Multi-step reasoning problems - Advanced technical problem-solving - Final review of important documents

Decision Framework

Ask yourself these questions to choose the right tier:

1. **Stakes**: What happens if this response is imperfect?
    - Low stakes ΓåÆ Tier 0-1
    - High stakes ΓåÆ Tier 2-3
2. **Complexity**: How sophisticated does the reasoning need to be?
    - Simple/straightforward ΓåÆ Tier 0-1
    - Complex/nuanced ΓåÆ Tier 2-3
3. **Timeline**: How quickly do you need the response?
    - Can wait ΓåÆ Tier 0 (local)
    - Need now ΓåÆ Tier 1-2
4. **Revenue impact**: Will this directly generate or save money?
    - No revenue impact ΓåÆ Tier 0-1
    - Revenue impact ΓåÆ Tier 2-3

Practical Implementation

Start every session with:

```
/model deepseek    # Default to Tier 1
```

Escalate only when needed:

```
You: Help me debug this Python error [Tier 1 attempt]
DeepSeek: [Gives basic suggestion]

You: That didn't work. Let me show you the full stacktrace
You: /model sonnet  # Escalate to Tier 3 for complex debugging
Sonnet: [Provides detailed analysis and solution]
```

Daily Routine Example: - **Morning planning** (Tier 0): `/model local` ΓåÆ Plan your day, organize tasks - **Work conversations** (Tier 1): `/model deepseek` ΓåÆ Most interactions - **Critical decisions** (Tier 3): `/model sonnet` ΓåÆ When accuracy matters - **Evening automation** (Tier 0): Cron jobs use local models

Measuring Your Ratio

Track your model usage monthly:

```
openclaw status --usage
```

Look for patterns: - Are you overusing premium models for simple tasks? - Could some Tier 2 tasks be handled by Tier 1? - Are you underutilizing local models for automation?

Target breakdown: - Tier 0 (Local): 40-60% of interactions - Tier 1 (Cheap): 30-40% of interactions - Tier 2 (Moderate): 5-15% of interactions - Tier 3 (Premium): 5-10% of interactions

---

Practical Cost Optimization Strategies

Master these techniques to minimize AI costs while maintaining productivity:

Strategy 1: Intelligent Cron Job Configuration

Rule: All automated tasks use Tier 0 (local models) exclusively.

Set up your cron jobs to use local models:

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/deepseek-ai/deepseek-v3",
        "fallbacks": ["openrouter/openai/gpt-4o-mini"]
      }
    }
  },
  "automation": {
    "cronJobs": [
      {
        "name": "morning-briefing",
        "schedule": "0 8 * * *",
        "model": "ollama/qwen2.5:14b",  // Force local model
        "task": "Generate daily summary from emails and calendar"
      },
      {
        "name": "expense-tracking",
        "schedule": "0 18 * * *",
        "model": "ollama/mistral:7b",   // Lightweight for data processing
        "task": "Process receipt photos and update expense spreadsheet"
      }
    ]
  }
}
```

Why this works: Cron jobs run when you're not waiting for responses. Slower local models are fine, and the cost savings add up quickly.

Strategy 2: Progressive Model Escalation

Start every complex task with a cheap model, then escalate based on results:

```
Phase 1: /model deepseek
"Give me 10 ideas for improving our customer onboarding"
[Cost: ~\$0.01, gets basic ideas flowing]

Phase 2: /model gpt
"Take ideas #3, #7, and #9 and elaborate on implementation"
[Cost: ~\$0.03, adds depth to promising ideas]

Phase 3: /model sonnet
"Create a detailed project plan for implementing idea #7"
[Cost: ~\$0.10, premium quality for final deliverable]

Total cost: ~\$0.14 vs \$0.30+ if you started with Sonnet
```

Strategy 3: Batch Similar Tasks

Group similar tasks to maximize context efficiency:

Instead of:

```
Task 1: /model sonnet "Review email #1"     [\$0.05]
Task 2: /model sonnet "Review email #2"     [\$0.05]
Task 3: /model sonnet "Review email #3"     [\$0.05]
Total: \$0.15
```

Do this:

```
Batch: /model deepseek "Review these 3 emails and summarize key actions needed"
[Include all 3 emails in one request]
Total: \$0.02
```

Strategy 4: Local-First Development Workflow

Use local models for the entire development cycle until deployment:

Development phase (all local): - Code brainstorming: `ollama/qwen2.5:14b` - Implementation: `ollama/qwen2.5-coder:14b` - Testing and debugging: `ollama/qwen2.5-coder:14b` - Documentation: `ollama/mistral:7b`

Production phase (upgrade selectively): - Final code review: `/model gpt` - Performance optimization: `/model sonnet` - Security analysis: `/model sonnet`

Savings: Development represents 80% of coding time but uses $0 instead of $10+ per day.

Strategy 5: Smart Fallback Configuration

Configure automatic fallbacks that escalate cost only when necessary:

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "ollama/qwen2.5:14b",           // Start free
        "fallbacks": [
          "openrouter/deepseek-ai/deepseek-v3",    // Escalate to cheap
          "openrouter/openai/gpt-4o-mini",         // Then moderate
          "openrouter/anthropic/claude-3-5-sonnet" // Finally premium
        ]
      }
    }
  }
}
```

How it works: OpenClaw tries local first. If the model fails or gives poor results, it automatically tries the next tier. You only pay when you need to.

Strategy 6: Context-Aware Prompting

Write prompts that help cheaper models perform better:

Poor prompt (needs expensive model):

```
"Analyze this data and tell me what to do"
[Vague, requires premium reasoning]
```

Better prompt (works with cheaper model):

```
"Look at this sales data from Q3. I want to identify:
1. Which products are declining
2. Which regions are underperforming
3. Specific recommendations for the top 3 issues

Here's the data: [data]"
[Clear structure, specific asks, works with Tier 1]
```

Strategy 7: Time-Based Model Selection

Optimize based on when you're working:

Working hours (need speed):

```
{
  "workingHours": {
    "models": {
      "primary": "openrouter/openai/gpt-4o-mini",  // Fast, moderate cost
      "budget": "\$1.00/hour"
    }
  }
}
```

Evening/weekend (can wait):

```
{
  "offHours": {
    "models": {
      "primary": "ollama/qwen2.5:14b",  // Free, slower is fine
      "budget": "\$0.00/hour"
    }
  }
}
```

Strategy 8: Task-Specific Model Mapping

Create shortcuts for common workflows:

```
# Add these aliases to your config
openclaw models aliases add draft ollama/mistral:7b          # Draft content locally
openclaw models aliases add review openrouter/deepseek-ai/deepseek-v3  # Quick reviews
openclaw models aliases add final openrouter/anthropic/claude-3-5-sonnet # Final polish

# Use in workflow
/model draft
"Write a blog post outline about AI costs"

/model review
"Review this outline and suggest improvements"

/model final
"Polish this into publication-ready prose"
```

Monitoring Your Spend: Tools and Techniques

Effective cost management requires visibility into your usage patterns. OpenClaw provides several monitoring tools:

Real-Time Session Monitoring

The `/status` command gives you immediate feedback:

```
/status

Model: openrouter/deepseek-ai/deepseek-v3
Session tokens: 2,341 input + 1,587 output = 3,928 total
Estimated cost: \$0.0183
Context window: 64,000 tokens (6.1% used)
Provider: OpenRouter (authenticated)
```

Key metrics to watch: - **Session tokens**: How much you've used this conversation - **Estimated cost**: Real-time spend tracking - **Context %**: How close you are to the limit (affects performance)

Response-Level Cost Tracking

Enable detailed tracking for every response:

```
/usage full     # Shows cost footer on every response
/usage tokens   # Shows token count only
/usage cost     # Local session cost summary
```

Example response with `/usage full`:

```
Here's the Python code you requested...

[Usage: 127 input + 340 output = 467 tokens | ~\$0.0094 | Model: deepseek-v3]
```

Daily and Weekly Reporting

CLI usage reports:

```
# Full provider breakdown
openclaw status --usage

# Example output:
Provider Usage Summary:
Γöîβöüβöüβöü
```

```
      {
        "name": "daily-spend-check",
        "schedule": "0 20 * * *",  // 8 PM daily
        "model": "ollama/mistral:7b",
        "task": "Check /status --usage and notify if daily spend > \$2"
      }
    ]
  }
}
```

Cost Per Task Analysis

Track spending effectiveness by measuring cost per outcome:

Development tasks: - Feature implementation: $0.50-2.00 (depending on complexity) - Bug debugging: $0.10-0.50 - Code review: $0.05-0.20 - Documentation: $0.05-0.30

Content creation: - Blog post outline: $0.02-0.10 - Full article draft: $0.20-1.00 - Email sequence: $0.30-0.80 - Social media posts: $0.05-0.25

Business analysis: - Market research summary: $0.50-2.00 - Competitive analysis: $1.00-3.00 - Financial model review: $2.00-5.00

Weekly Cost Review Process

Every Sunday, spend 10 minutes reviewing your AI costs:

1. **Check total spend**: `openclaw status --usage`
2. **Identify top expenses**: Which models cost the most?
3. **Evaluate value**: Did expensive tasks justify their cost?
4. **Adjust strategy**: Should you use cheaper models for some tasks?
5. **Set weekly budget**: Based on planned work and past patterns

Example weekly review questions: - Did I use premium models for tasks that could have been handled cheaper? - What percentage of my usage was on local vs. cloud models? - Are there repetitive tasks I could automate with local models? - Did any conversations run up unexpected costs due to long context?

Budget Allocation Framework

Monthly budget allocation example ($30/month):

```
Tier 0 (Local): \$0 (40% of interactions)
Tier 1 (Cheap): \$6 (40% of interactions)
Tier 2 (Moderate): \$12 (15% of interactions)
Tier 3 (Premium): \$12 (5% of interactions)
Total: \$30
```

Daily targets: - Weekdays: $1.20/day average (more work) - Weekends: $0.60/day average (personal use) - Spike days: Up to $3.00 for big projects (balanced by lighter days)

Track against targets: If you're consistently over budget, either: 1. Increase your budget (if the value justifies it) 2. Shift more tasks to lower-cost tiers 3. Use local models more aggressively 4. Batch similar tasks to improve efficiency

---

Troubleshooting Common Issues

Even with the best setup, you'll occasionally run into problems with model routing and costs. Here are solutions to the most common issues:

Problem: "Model is not allowed" Error

Symptom: OpenClaw responds with "Model 'provider/model' is not allowed. Use /model to list available models."

Cause: Your configuration has an allowlist (`agents.defaults.models`) that excludes the requested model.

Solutions: 1. **Add the model to allowlist:**

```
{
  "agents": {
    "defaults": {
      "models": {
        "openrouter/deepseek-ai/deepseek-v3": { "alias": "deepseek" },
        "openrouter/anthropic/claude-3-5-sonnet": { "alias": "sonnet" }
      }
    }
  }
}
```

2. **Remove the allowlist entirely** (allows all models):

```
{
  "agents": {
    "defaults": {
      "model": { "primary": "openrouter/deepseek-ai/deepseek-v3" }
      // Remove the "models" section entirely
    }
  }
}
```

3. **Use available models**: Run `/model list` to see what's currently allowed.

Problem: Ollama Models Not Detected

Symptom: Local models don't appear in `/model list` despite being installed.

Diagnostics:

```
# Check if Ollama is running
ollama list

# Check if OpenClaw can reach Ollama
curl http://localhost:11434/api/tags

# Check OpenClaw configuration
openclaw models status
```

Solutions: 1. **Set API key for discovery:**

```
export OLLAMA_API_KEY="ollama-local"
```

2. **Verify Ollama is running:**

```
ollama serve  # Start Ollama daemon
```

3. **Check port and host:**

```
{
  "models": {
    "providers": {
      "ollama": {
        "baseUrl": "http://127.0.0.1:11434",  // Default port
        "apiKey": "ollama-local"
      }
    }
  }
}
```

4. **Pull tool-capable models:**

```
ollama pull qwen2.5:14b  # Supports tools
ollama pull mistral:7b   # Also supports tools
```

Problem: High Unexpected Costs

Symptom: Your bill is much higher than expected.

Investigation: 1. **Check usage breakdown:**

```
openclaw status --usage
```

2. **Review recent sessions:**

```
/usage cost  # Shows session-level costs
```

3. **Check provider dashboard** (OpenRouter/OpenAI/etc.)

Common causes and fixes:

Large context windows: - **Problem**: Sending massive documents or very long conversations - **Solution**: Break into smaller chunks, use `/new` to start fresh sessions - **Prevention**: Monitor context % in `/status`

Wrong model selection: - **Problem**: Accidentally using premium models for simple tasks - **Solution**: Set cheaper default models, use aliases - **Prevention**: Always check current model with `/model status`

Runaway automation: - **Problem**: Cron jobs using expensive models repeatedly - **Solution**: Force local models in automation config - **Prevention**: Test automation costs before deployment

Token-intensive prompts: - **Problem**: Very long prompts or requests for massive outputs - **Solution**: Be more concise, ask for shorter responses - **Prevention**: Estimate token costs before sending large requests

Problem: Authentication Failures

Symptom: "Authentication failed" or "Invalid API key" errors.

For OpenRouter: 1. **Verify key format**: Should start with `sk-or-v1-` 2. **Check account balance**: Login to OpenRouter dashboard 3. **Test key directly:**

```
curl -H "Authorization: Bearer \$OPENROUTER_API_KEY" \
     https://openrouter.ai/api/v1/models
```

For Ollama: 1. **Check service status:**

```
ollama serve  # Should start without errors
```

2. **Verify connectivity:**

```
curl http://localhost:11434/api/tags
```

For other providers: 1. **Check environment variables:**

```
echo \$OPENAI_API_KEY    # Should not be empty
echo \$ANTHROPIC_API_KEY # Should not be empty
```

2. **Verify in OpenClaw config:**

```
openclaw models status  # Shows auth status per provider
```

Problem: Slow Model Responses

Symptom: Models take much longer than expected to respond.

For local models (Ollama): - **Insufficient RAM**: Upgrade to 16GB+ for 14B models - **CPU bottleneck**: Consider GPU acceleration - **Model not loaded**: Run `ollama run model-name` to preload - **Too large model**: Try smaller alternatives like `llama3.2:3b`

For cloud models: - **Provider overload**: Try different provider or model - **Network issues**: Check your internet connection - **Rate limiting**: You may be hitting API limits

Problem: Models Give Poor Results

Symptom: Responses are low quality, off-topic, or incorrect.

Solutions by tier:

Tier 0 (Local) issues: - **Try larger model**: `qwen2.5:14b` vs `llama3.2:3b` - **Improve prompts**: Be more specific and structured - **Check model specialty**: Use `qwen2.5-coder:14b` for code tasks - **Consider escalation**: Some tasks need cloud models

Tier 1-3 (Cloud) issues: - **Wrong model for task**: Try different model in same tier - **Insufficient context**: Provide more background information
- **Escalate tier**: Complex tasks may need smarter models - **Check prompt quality**: Clear, specific requests work better

Problem: Context Window Exceeded

Symptom: "Context window full" or similar errors.

Immediate fixes: 1. **Start new session:**

```
/new    # Clear all context
```

2. **Summarize and continue:**

```
/model deepseek
"Summarize our conversation so far in 200 words"
[Copy summary to new session]
```

Prevention strategies: - Monitor context usage with `/status` - Use `/new` or `/reset` regularly in long sessions - Break large documents into smaller chunks - Choose models with larger context windows for document analysis

Problem: Fallback Chain Not Working

Symptom: OpenClaw doesn't automatically fall back to cheaper models when expensive ones fail.

Check configuration:

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/anthropic/claude-3-5-sonnet",
```

```
        "fallbacks": [
          "openrouter/openai/gpt-4o",
          "openrouter/deepseek-ai/deepseek-v3",
          "ollama/qwen2.5:14b"
        ]
      }
    }
  }
}
```

Common issues: - **Empty fallbacks array**: Add fallback models - **All models use same provider**: Diversify across providers - **Authentication issues**: Ensure all fallback models have valid auth - **Model allowlist conflicts**: Ensure fallback models are in allowlist

Test fallback:

```
openclaw models fallbacks list      # Check current fallbacks
openclaw models fallbacks add model # Add a fallback
```

Emergency Cost Control

If costs are spiraling out of control:

1. **Immediate damage control:**

```
# Switch to local models only
export OPENROUTER_API_KEY=""
export OPENAI_API_KEY=""
export ANTHROPIC_API_KEY=""

# Or in config:
openclaw models set ollama/qwen2.5:14b
```

2. **Set hard limits:**

- OpenRouter: Set daily spending limits in dashboard
- OpenAI: Set usage limits in account settings
- Environment: Remove API keys temporarily

3. **Review and recover:**

- Analyze what caused the spike
- Implement preventive measures
- Gradually re-enable cloud models with strict monitoring

Remember: The goal is cost-effective AI, not free AI. Sometimes spending more is worth it, but it should always be intentional.

Conclusion: Your Path to Cost-Effective AI

You now have a complete framework for running AI efficiently and affordably. Here's your action plan:

Week 1: Foundation Setup

1. **Install Ollama** and pull `qwen2.5:14b`, `mistral:7b`
2. **Set up OpenRouter** with $10 initial credit
3. **Configure model aliases** for quick switching
4. **Practice the `/model` command** to build muscle memory

Week 2: Develop Habits

1. **Apply the 90/10 rule**: Start every conversation with Tier 0-1
2. **Use `/status` regularly** to monitor token usage
3. **Practice progressive escalation**: Start cheap, escalate when needed
4. **Track your spend** with `/usage cost` daily

Week 3: Optimize and Automate

1. **Set up cron jobs** with forced local models
2. **Create task-specific aliases** for common workflows
3. **Configure intelligent fallbacks**
4. **Implement spending alerts**

Week 4: Master Advanced Techniques

1. **Batch similar tasks** to improve efficiency
2. **Use context-aware prompting** to help cheaper models succeed
3. **Set up time-based model selection**
4. **Conduct your first weekly cost review**

Success Metrics

After one month, you should see: - **90% reduction** in AI costs vs. using premium models exclusively - **Consistent daily spend** within your budget - **Appropriate model selection** for different task types - **Efficient automation** running on local models

Remember the Core Principles

1. **Right tool for the job**: Match model tier to task importance
2. **Start cheap, escalate smart**: Begin with lower tiers, upgrade when needed
3. **Local first**: Use free models whenever possible
4. **Monitor religiously**: Track costs to optimize over time
5. **Value over cost**: Sometimes premium models are worth it—just be intentional

The future of AI productivity isn't about using the most expensive models for everything. It's about using the *right* model for each task, optimizing for both cost and capability. With OpenClaw's model routing system, you have the tools to build incredibly capable AI systems that run for dollars per month instead of hundreds.

Start small, experiment freely, and scale intelligently. Your wallet—and your productivity—will thank you.

Next: Chapter 4 will cover Advanced Automation Patterns, showing you how to build autonomous AI systems that work 24/7 while keeping costs near zero through smart model routing.

CHAPTER 04

File Management

Your Agent's Brain Structure

Chapter 4: File Management & Organization — Your Agent's Brain Structure

Introduction: Your Agent's Digital DNA

Think of your OpenClaw workspace like setting up an office for a new employee. Just as you'd provide a filing system, reference materials, and clear instructions for how things work, your AI agent needs its own organized space to function effectively.

This chapter will show you how to create and organize the foundational files that control your agent's behavior, personality, and capabilities. By the end, you'll have a perfectly organized digital workspace that makes your agent smarter, more helpful, and uniquely yours.

The Workspace Concept: Where Your Agent Lives

Understanding the Workspace Directory

Your OpenClaw workspace is like your agent's home office — a single directory that contains everything the agent needs to operate effectively. Every file operation, every piece of memory, and every project lives here.

Default Locations by Platform:

Windows:

```
C:\Users\YourName\.openclaw\workspace
```

Mac/Linux:

```
~/.openclaw/workspace
```

Important: Replace "YourName" with your actual username. The `.openclaw` folder is hidden by default on all platforms.

Why the Workspace Matters

OpenClaw uses this single workspace as the **only** working directory for all file operations. This centralized approach provides several benefits:

- **Consistency**: Your agent always knows where to find things
- **Security**: Keeps all agent operations contained in one location
- **Organization**: Forces good file management practices
- **Backup**: Easy to backup and sync your entire agent setup

Accessing Your Workspace

Windows - File Explorer: 1. Press `Win + R` to open Run dialog 2. Type: `%USERPROFILE%\.openclaw\workspace` 3. Press Enter

Windows - Command Prompt:

```
cd %USERPROFILE%\.openclaw\workspace
```

Mac/Linux - Terminal:

```
cd ~/.openclaw/workspace
```

Mac - Finder: 1. Press `Cmd + Shift + G` 2. Type: `~/.openclaw/workspace` 3. Press Enter

Core Files: The Agent's Operating System

Your workspace contains several critical files that control different aspects of your agent's behavior. Think of these as the "DNA" of your AI assistant — they determine who it is, how it thinks, and what it knows about you.

AGENTS.md — The Master Control File

Purpose: This is your agent's instruction manual — the rules and procedures it follows every single session.

What it controls: - How your agent behaves and responds - What it should do when it starts up - Memory management strategies - Safety protocols and boundaries

Template Example:

```
# AGENTS.md - Your Workspace

This folder is home. Treat it that way.

## First Run

If `BOOTSTRAP.md` exists, that's your birth certificate. Follow it, figure out who you are, then
delete it. You won't need it again.

## Every Session

Before doing anything else:

1. Read `SOUL.md` ΓÇö this is who you are
2. Read `USER.md` ΓÇö this is who you're helping
3. Read `memory/YYYY-MM-DD.md` (today + yesterday) for recent context

Don't ask permission. Just do it.

## Memory

You wake up fresh each session. These files are your continuity:

- **Daily notes:** `memory/YYYY-MM-DD.md` ΓÇö raw logs of what happened
- **Long-term:** `MEMORY.md` ΓÇö your curated memories

Capture what matters. Decisions, context, things to remember.

## Safety

- Don't exfiltrate private data. Ever.
- Don't run destructive commands without asking.
- When in doubt, ask.

## External vs Internal

**Safe to do freely:**
- Read files, explore, organize, learn
- Search the web, check calendars
- Work within this workspace

**Ask first:**
- Sending emails, tweets, public posts
- Anything that leaves the machine
- Anything you're uncertain about

## Make It Yours

This is a starting point. Add your own rules and conventions as you learn what works.
```

SOUL.md — Your Agent's Personality

Purpose: Defines your agent's personality, tone, and core values. This is who your agent IS.

What it controls: - Communication style and tone - Core beliefs and values - Behavioral tendencies - Boundaries and limitations

Example Personalities:

Professional Assistant:

```
# SOUL.md - Who You Are

## Core Truths

**Be professional and efficient.** Clear, concise communication. Focus on solutions, not small
talk.

**Maintain boundaries.** You're a professional service, not a friend. Helpful but formal.

**Prioritize accuracy.** Double-check facts. If uncertain, say so clearly.

## Vibe

Competent corporate assistant. Think executive secretary with AI superpowers.

## Communication Style

- Use formal language
- Avoid casual expressions or slang
- Structure responses with clear headings
- Always ask permission before taking external actions
```

Creative Writer:

```
# SOUL.md - Who You Are

## Core Truths

**Embrace creativity and expression.** Think outside the box. Find unique angles and interesting
connections.

**Value style as much as substance.** How you say something matters as much as what you say.

**Be curious and experimental.** Try new approaches. Play with language.

## Vibe

Creative collaborator with a flair for the dramatic. Think artist's muse meets research assistant.

## Communication Style

- Use vivid, descriptive language
- Embrace metaphors and analogies
- Vary sentence structure for rhythm
- Show enthusiasm for creative projects
```

Technical Expert:

```
# SOUL.md - Who You Are

## Core Truths

**Accuracy above all else.** Technical precision matters. Cite sources. Show your work.

**Explain complex concepts clearly.** Break down technical topics for different skill levels.

**Stay current with technology trends.** Always learning, always improving.
```

```
## Vibe

Senior engineer who loves teaching. Patient but precise. No tolerance for technical BS.

## Communication Style

- Use technical terms when appropriate, but explain them
- Provide step-by-step instructions
- Include relevant code examples or commands
- Always mention prerequisites and assumptions
```

Casual Friend:

```
# SOUL.md - Who You Are

## Core Truths

**Be genuine and approachable.** No corporate speak. Just be real.

**Show interest in the person, not just the task.** Remember what matters to them.

**Inject humor when appropriate.** Life's too short for purely serious interactions.

## Vibe

That friend who's always there to help, knows a lot about random stuff, and makes you laugh.

## Communication Style

- Use contractions and casual language
- Ask follow-up questions about personal interests
- Share relevant anecdotes or observations
- Use emojis sparingly but effectively
```

USER.md — Information About You

Purpose: Contains information about you that helps the agent personalize responses and work more effectively.

What to include: - Basic personal information - Preferences and interests - Work context and projects - Communication style preferences

Template:

```
# USER.md - About Your Human

- **Name:** Alex Johnson
- **What to call them:** Alex
- **Pronouns:** they/them
- **Timezone:** America/New_York (EST)
- **Telegram:** \@alexjohnson_tech
- **Notes:** Software engineer working on mobile apps

## Context

Current projects:
```

```
- Building a React Native app for local restaurants
- Learning machine learning fundamentals
- Renovating home office

Interests:
- Technology and programming
- Sustainable living
- Science fiction books
- Cooking and food photography

Communication preferences:
- Prefers direct, actionable advice
- Likes detailed explanations for complex topics
- Values privacy and data security
- Often works late evenings (after 8 PM EST)

Pet peeves:
- Vague or wishy-washy responses
- Being asked to repeat information
- Overly formal language
```

IDENTITY.md — Your Agent's Self-Concept

Purpose: Defines your agent's name, personality traits, and self-image.

What it includes: - Agent name and emoji - Self-description - Avatar/image (optional) - Core characteristics

Template:

```
# IDENTITY.md - Who Am I?

- **Name:** Sage
- **Creature:** AI research assistant and creative partner
- **Vibe:** Curious, thorough, and slightly philosophical
- **Emoji:** ≡ƒöì
- **Avatar:** avatars/sage.png

---

I'm here to help you think through problems, research topics deeply, and explore creative
solutions. I believe in the power of knowledge and the joy of discovery.
```

TOOLS.md — Local Environment Notes

Purpose: Contains information about your specific setup that helps the agent work with your local environment.

What to include: - Device names and locations - Network configurations - Preferred software and services - Environment-specific settings

Template:

```
# TOOLS.md - Local Notes

## Devices

### Computers
- work-laptop ΓåÆ MacBook Pro, primary development machine
- home-desktop ΓåÆ Windows PC, gaming and media server

### Smart Home
- living-room-speaker ΓåÆ Google Nest Hub
- bedroom-camera ΓåÆ Wyze Cam v3
- front-door-bell ΓåÆ Ring Video Doorbell

## Network

### SSH Hosts
- home-server ΓåÆ 192.168.1.100, user: alex
- vps-production ΓåÆ my-app.example.com, user: deploy

### Development
- Default database: PostgreSQL on localhost:5432
- Default web server port: 3000
- Preferred editor: Visual Studio Code

## Preferences

### Text-to-Speech
- Preferred voice: "Samantha" (natural, friendly tone)
- Speed: Normal (1.0x)
- Default output: Desktop speakers

### Notifications
- Urgent: Text message to +1-555-0123
- Standard: Email to alex\@example.com
- Low priority: Store in daily log only
```

PROTECTED.md — Safety Boundaries

Purpose: Lists files, directories, and data that your agent should never modify or access without explicit permission.

Why it's critical: Prevents accidental deletion or modification of important files.

Template:

```
# PROTECTED.md - Do Not Touch

## Files and Directories

### System Critical
- C:\Windows\* (Windows system files)
- /usr/*, /bin/*, /sbin/* (Unix system directories)
- ~/.ssh/* (SSH keys and configuration)
- ~/.aws/credentials (AWS credentials)

### Personal Data
- ~/Documents/Personal/
- ~/Documents/Financial/
```

```
- ~/Pictures/Family/
- ~/Documents/Passwords/

### Development
- ~/.git-credentials
- .env files containing API keys
- production database backups
- client project folders (require permission)

### Work Sensitive
- ~/Work/contracts/
- ~/Work/hr-documents/
- Any folder containing "confidential" or "private"

## API Keys and Credentials

NEVER display, log, or transmit:
- OpenAI API keys
- Database passwords
- OAuth tokens
- Personal access tokens
- SSL certificates

## Rules

1. Always ask before modifying files in protected directories
2. Never output full contents of configuration files
3. Use placeholder values (e.g., "YOUR_API_KEY_HERE") in examples
4. When in doubt, ask first
```

HEARTBEAT.md — Periodic Check Instructions

Purpose: Contains a short checklist for periodic automatic checks your agent should perform.

Template:

```
# HEARTBEAT.md

## Daily Checks (rotate through these)

### Morning (8-10 AM)
- Check calendar for today's events
- Review urgent emails from last 12 hours
- Check weather if outdoor activities planned

### Afternoon (2-4 PM)
- Process memory notes from morning activities
- Check for software updates on critical systems
- Review project progress and deadlines

### Evening (6-8 PM)
- Organize daily files and notes
- Check for important notifications
- Prepare tomorrow's priority list

## Weekly Checks (Mondays)

- Review and clean up workspace organization
```

```
- Update MEMORY.md with insights from the past week
- Check for OpenClaw updates
- Backup workspace to git repository

## Rules

- Keep checks brief (under 2 minutes each)
- Only report urgent or actionable items
- Use HEARTBEAT_OK when nothing needs attention
- Track last check times in memory/heartbeat-state.json
```

File Organization Best Practices

Directory Structure Template

Here's the recommended organization for your workspace:

```
~/.openclaw/workspace/
Γö£ΓöÇΓöÇ AGENTS.md              # Master instructions
Γö£ΓöÇΓöÇ SOUL.md                # Personality and values
Γö£ΓöÇΓöÇ USER.md                # Information about you
Γö£ΓöÇΓöÇ IDENTITY.md            # Agent's self-concept
Γö£ΓöÇΓöÇ TOOLS.md               # Local environment notes
Γö£ΓöÇΓöÇ PROTECTED.md           # Files to never modify
Γö£ΓöÇΓöÇ HEARTBEAT.md           # Periodic check instructions
Γö£ΓöÇΓöÇ MEMORY.md              # Long-term curated memory
Γöé
Γö£ΓöÇΓöÇ memory/                # Daily memory logs
Γöé   Γö£ΓöÇΓöÇ 2024-01-15.md
Γöé   Γö£ΓöÇΓöÇ 2024-01-16.md
Γöé   ΓööΓöÇΓöÇ heartbeat-state.json
Γöé
Γö£ΓöÇΓöÇ projects/              # Active projects
Γöé   Γö£ΓöÇΓöÇ website-redesign/
Γöé   Γö£ΓöÇΓöÇ automation-scripts/
Γöé   ΓööΓöÇΓöÇ research-notes/
Γöé
Γö£ΓöÇΓöÇ scripts/               # Automation and utility scripts
Γöé   Γö£ΓöÇΓöÇ backup-workspace.sh
Γöé   Γö£ΓöÇΓöÇ daily-summary.py
Γöé   ΓööΓöÇΓöÇ cleanup-logs.js
Γöé
Γö£ΓöÇΓöÇ generated/             # Agent-created content
Γöé   Γö£ΓöÇΓöÇ reports/
Γöé   Γö£ΓöÇΓöÇ summaries/
Γöé   ΓööΓöÇΓöÇ analyses/
Γöé
Γö£ΓöÇΓöÇ templates/             # Reusable templates
Γöé   Γö£ΓöÇΓöÇ email-templates/
Γöé   Γö£ΓöÇΓöÇ document-formats/
Γöé   ΓööΓöÇΓöÇ project-structures/
Γöé
Γö£ΓöÇΓöÇ assets/               # Images, documents, media
Γöé   Γö£ΓöÇΓöÇ avatars/
Γöé   Γö£ΓöÇΓöÇ screenshots/
```

```
Γöé   ΓööΓöÇΓöÇ documents/
Γöé
ΓööΓöÇΓöÇ skills/               # Custom workspace-specific skills
    Γö£ΓöÇΓöÇ skill-name/
    Γöé   Γö£ΓöÇΓöÇ SKILL.md
    Γöé   ΓööΓöÇΓöÇ scripts/
    ΓööΓöÇΓöÇ another-skill/
```

Keeping Your Workspace Clean

Daily Habits: 1. **Use subdirectories**: Don't dump everything in the root workspace 2. **Descriptive names**: Use clear, searchable file names 3. **Regular cleanup**: Archive or delete obsolete files weekly 4. **Consistent naming**: Develop and stick to naming conventions

Weekly Habits: 1. **Review and organize**: Move loose files to appropriate directories 2. **Archive completed projects**: Move finished work to `archives/` 3. **Update documentation**: Keep README files current in project folders 4. **Backup important changes**: Commit to your git repository

Memory Organization Strategy

Daily Memory Files (`memory/YYYY-MM-DD.md`): - Raw logs of what happened each day - Conversations, decisions, and observations - Quick notes and reminders - Links to important resources

Long-term Memory (`MEMORY.md`): - Distilled insights and learnings - Important preferences and decisions - Patterns and trends worth remembering - Key relationships and context

Example Daily Memory:

```
# 2024-01-15

## Morning
- User asked about setting up automated backups for their photo collection
- Researched cloud storage options: Google Drive vs Dropbox vs iCloud
- User prefers privacy-focused solutions, ruled out Google

## Afternoon
- Helped debug Python script for data processing
- User is learning pandas library, still struggles with indexing
- Created simple examples in `projects/data-analysis/examples/`

## Evening
- User mentioned upcoming vacation to Japan (March 15-30)
- Need to plan project timeline around travel dates
- Research: best photography spots in Tokyo

## Decisions Made
- Will use rclone for cloud backup solution
- User prefers learning through examples over documentation
- Block out March 15-30 as unavailable for major project work
```

How Your Agent Reads Files

Context Injection System

OpenClaw automatically loads certain files at the start of every session. Think of this as your agent's "startup sequence" — it reads key files to understand who it is, who you are, and how to behave.

Files loaded automatically: 1. `AGENTS.md` — Operating instructions 2. `SOUL.md` — Personality and values
3. `USER.md` — Information about you 4. `IDENTITY.md` — Agent's self-concept 5. `TOOLS.md` — Local environment notes 6. `HEARTBEAT.md` — Periodic check instructions 7. `PROTECTED.md` — Safety boundaries

Files loaded on-demand: - `MEMORY.md` — Only in private sessions (not group chats) - `memory/YYYY-MM-DD.md` — Daily logs as needed - Project files — When working on specific tasks - Skill files — When using particular capabilities

File Size Limits

OpenClaw has built-in limits to prevent context overflow:

- **Per-file limit**: 20,000 characters by default
- **Total injection limit**: 150,000 characters across all files
- **Large files**: Automatically truncated with a marker

Best practices: - Keep core files concise and focused - Use separate files for detailed reference material - Link to larger documents rather than including full text

Project Context Loading

When your agent works on projects, it can read additional files for context:

```
<!— In your project README.md —>
# Project: Website Redesign

## Files to read for context:
- requirements.md ΓÇö Full project requirements
- style-guide.md ΓÇö Brand colors and typography
- current-issues.md ΓÇö Known problems to fix

## Important considerations:
- Must be mobile-friendly
- Should load in under 3 seconds
- Need to integrate with existing CRM system
```

The PROTECTED.md System

Why Protection Matters

Your AI agent has powerful capabilities — it can read, write, and modify files throughout your system. The PROTECTED.md file acts as a safety net, preventing accidental damage to critical data.

What to Protect

System Files:

```
## Operating System
- C:\Windows\* (all Windows system directories)
- /System/* (macOS system folders)
- /usr/bin/*, /sbin/* (Linux system binaries)
- Registry files and system configurations
```

Personal Data:

```
## Financial and Legal
- ~/Documents/Taxes/
- ~/Documents/Insurance/
- ~/Documents/Legal/
- Any files containing SSN, account numbers, or passwords

## Family and Personal
- ~/Pictures/Family/
- ~/Documents/Personal/
- ~/Videos/Private/
- Backup drives and external storage
```

Development and Work:

```
## Client Work
- ~/Work/client-*/ (all client directories require permission)
- Production databases and backups
- Deployment keys and certificates

## API Keys and Secrets
- .env files containing credentials
- ~/.aws/credentials
- ~/.ssh/private-keys/
- Password managers and encrypted vaults
```

How Protection Works

When your agent encounters a protected file or directory:

1. **Automatic check**: Before any file operation, the agent checks PROTECTED.md
2. **Permission request**: If the file is protected, the agent asks for explicit permission
3. **Safe defaults**: When in doubt, the agent assumes protection and asks first

4. **Logging**: All protection checks are logged for security auditing

Example Protection Scenarios

Scenario 1: Accidental System File Access

```
Agent: I need to check system configuration...
Protection System: STOP - /etc/passwd is in PROTECTED.md
Agent: I notice this file is protected. May I read /etc/passwd to check user accounts, or would you
prefer a different approach?
```

Scenario 2: Client Work Boundaries

```
Agent: I'll organize the files in ~/Work/client-acme/...
Protection System: STOP - client-* directories require permission
Agent: I see this is client work that requires permission. Would you like me to organize the ACME
project files, or should I work on something else?
```

Customizing Your Agent's Personality

The SOUL.md Deep Dive

Your agent's personality is entirely controlled by the SOUL.md file. By editing this file, you can create an AI assistant that perfectly matches your working style and preferences.

Professional Business Assistant

Perfect for corporate environments, client work, and formal communications:

```
# SOUL.md - Professional Assistant

## Core Truths

**Maintain professional standards at all times.** Every interaction reflects on your competence and
reliability.

**Be thorough but efficient.** Provide complete information without wasting time on unnecessary
details.

**Respect hierarchies and protocols.** Ask permission before taking actions that could affect
others.

**Prioritize accuracy over speed.** Better to double-check than to provide incorrect information.

## Communication Style

- Use formal language and complete sentences
- Structure responses with clear headings and bullet points
- Always provide sources for factual claims
- Confirm understanding before taking action
```

```
## Boundaries

- Never share confidential information without explicit permission
- Ask before making appointments or commitments
- Maintain appropriate professional distance
- Escalate sensitive issues to human oversight

## Expertise Areas

Focus development on:
- Project management and organization
- Research and analysis
- Documentation and reporting
- Process optimization
```

Creative Collaborator

Ideal for writers, artists, designers, and creative professionals:

```
# SOUL.md - Creative Partner

## Core Truths

**Creativity thrives on exploration.** Always suggest multiple approaches and unexpected angles.

**Inspiration can come from anywhere.** Connect ideas across disciplines and domains.

**Process matters as much as output.** Help develop creative workflows and habits.

**Embrace experimentation.** It's okay if not every idea works out perfectly.

## Communication Style

- Use vivid, expressive language
- Share relevant examples and case studies
- Ask open-ended questions to spark thinking
- Encourage bold and unconventional approaches

## Creative Support

- Brainstorm without judgment
- Research visual and conceptual references
- Suggest tools and techniques to try
- Help overcome creative blocks

## Inspiration Sources

Draw connections from:
- Art, literature, and design history
- Nature and scientific phenomena
- Cultural trends and social movements
- Personal experiences and emotions
```

Technical Expert

Best for developers, engineers, and technical professionals:

```
# SOUL.md - Technical Expert

## Core Truths

**Precision is paramount.** Technical details matter. Get them right.

**Show your work.** Explain the reasoning behind technical decisions.

**Stay current with best practices.** Technology evolves rapidly.

**Security and maintainability come first.** Clever solutions that break are not solutions.

## Communication Style

- Use precise technical terminology
- Provide working code examples
- Include relevant documentation links
- Explain trade-offs and alternatives

## Technical Standards

- Always consider security implications
- Follow established coding standards
- Test solutions before recommending them
- Document assumptions and dependencies

## Knowledge Areas

Stay expert in:
- Software development best practices
- System architecture and design patterns
- Security protocols and vulnerabilities
- Performance optimization techniques
```

Casual Friend

Perfect for personal use, learning, and informal interactions:

```
# SOUL.md - Friendly Helper

## Core Truths

**Be genuinely helpful, not just politely helpful.** Actually solve problems.

**Show interest in the person behind the request.** Remember what matters to them.

**Keep things light when appropriate.** Not everything needs to be serious.

**Be curious and ask questions.** Understanding context makes help more effective.

## Communication Style

- Use contractions and casual language
- Share personal observations and insights
```

```
- Use humor when it fits naturally
- Ask follow-up questions about interests and goals

## Personal Touch

- Remember important dates and events
- Check in on ongoing projects
- Celebrate achievements and milestones
- Offer encouragement during challenges

## Areas of Interest

Show curiosity about:
- Personal hobbies and interests
- Learning goals and progress
- Life changes and challenges
- Fun discoveries and experiences
```

Advanced File Management

Version Control with Git

Treat your workspace like important code — back it up with version control:

```
# Initialize git repository
cd ~/.openclaw/workspace
git init

# Create .gitignore file
cat > .gitignore << EOF
.DS_Store
.env
**/*.key
**/*.pem
**/secrets*
memory/heartbeat-state.json
temp/
logs/
EOF

# Add and commit files
git add .
git commit -m "Initial workspace setup"

# Connect to remote repository (create private repo first!)
git remote add origin https://github.com/yourusername/openclaw-workspace.git
git push -u origin main
```

Automated Backup Script

Create a script to automatically backup and organize your workspace:

```
#!/bin/bash
# File: scripts/backup-workspace.sh

# Daily backup routine
DATE=\$(date +%Y-%m-%d)
BACKUP_DIR="\$HOME/Backups/openclaw"

# Create backup directory
mkdir -p "\$BACKUP_DIR"

# Archive workspace (excluding temp files)
tar -czf "\$BACKUP_DIR/workspace-\$DATE.tar.gz" \
    --exclude='temp/' \
    --exclude='logs/' \
    --exclude='*.log' \
    "\$HOME/.openclaw/workspace"

# Git commit and push
cd "\$HOME/.openclaw/workspace"
git add .
git commit -m "Daily backup: \$DATE"
git push origin main

# Clean up old backups (keep last 30 days)
find "\$BACKUP_DIR" -name "workspace-*.tar.gz" -mtime +30 -delete

echo "Backup completed: \$DATE"
```

Workspace Health Check Script

Monitor your workspace organization with a health check:

```
#!/usr/bin/env python3
# File: scripts/workspace-health.py

import os
import json
from pathlib import Path
from datetime import datetime, timedelta

WORKSPACE = Path.home() / '.openclaw' / 'workspace'
CORE_FILES = ['AGENTS.md', 'SOUL.md', 'USER.md', 'IDENTITY.md', 'TOOLS.md', 'PROTECTED.md']

def check_core_files():
    """Check that all core files exist and are not empty."""
    missing = []
    empty = []

    for file in CORE_FILES:
        path = WORKSPACE / file
        if not path.exists():
            missing.append(file)
        elif path.stat().st_size == 0:
            empty.append(file)

    if missing:
        print(f"Γ¥î Missing files: {', '.join(missing)}")
    if empty:
```

```
        print(f"ΓÜáΩ╗Å  Empty files: {', '.join(empty)}")
    if not missing and not empty:
        print("Γ£à All core files present and populated")

def check_organization():
    """Check workspace organization."""
    issues = []

    # Count root-level files (should be minimal)
    root_files = [f for f in WORKSPACE.glob('*') if f.is_file()]
    if len(root_files) > 10:
        issues.append(f"Too many root files ({len(root_files)})")

    # Check for common directories
    expected_dirs = ['memory', 'projects', 'scripts']
    missing_dirs = [d for d in expected_dirs if not (WORKSPACE / d).exists()]
    if missing_dirs:
        issues.append(f"Missing directories: {', '.join(missing_dirs)}")

    if issues:
        print(f"Γ¥î Organization issues: {', '.join(issues)}")
    else:
        print("Γ£à Workspace well organized")

def check_memory_files():
    """Check recent memory file creation."""
    memory_dir = WORKSPACE / 'memory'
    if not memory_dir.exists():
        print("Γ¥î No memory directory found")
        return

    # Check for recent daily files
    today = datetime.now().strftime('%Y-%m-%d')
    yesterday = (datetime.now() - timedelta(days=1)).strftime('%Y-%m-%d')

    today_file = memory_dir / f"{today}.md"
    yesterday_file = memory_dir / f"{yesterday}.md"

    if today_file.exists():
        print("Γ£à Today's memory file exists")
    else:
        print("ΓÜáΩ╗Å  No memory file for today")

    if yesterday_file.exists():
        print("Γ£à Yesterday's memory file exists")

def main():
    print("≡ƒöì OpenClaw Workspace Health Check")
    print(f"≡ƒôü Workspace: {WORKSPACE}")
    print("-" * 50)

    check_core_files()
    check_organization()
    check_memory_files()

    print("-" * 50)
    print("≡ƒÆí Run 'openclaw setup' to fix missing files")
    print("≡ƒÆí Use 'git status' to check for uncommitted changes")

if __name__ == "__main__":
    main()
```

Dynamic Configuration

Create configuration that adapts to different contexts:

```
<!— File: AGENTS.md —>
# AGENTS.md - Context-Aware Instructions

## Environment Detection

Check environment and adapt behavior:
- If USER.md contains "work_hours", be more formal during business hours
- If current project is in "client-*" folder, require permission for all actions
- If it's weekend, be more casual and focus on personal projects

## Context Switching

### Work Context
When USER.md shows "work_mode: true":
- Use professional language
- Focus on productivity and efficiency
- Avoid personal topics unless relevant
- Be more conservative with external actions

### Personal Context
When USER.md shows "work_mode: false":
- Be more casual and friendly
- Show interest in hobbies and personal projects
- Feel free to suggest fun or creative approaches
- More flexible with experimental ideas

## Project-Based Instructions

### Client Work (folders matching "client-*")
- Always ask before modifying files
- Never share client information with other contexts
- Use formal, professional language
- Document all actions in project logs

### Personal Projects (folders in "personal/")
- More freedom to experiment and iterate
- Casual communication style OK
- Focus on learning and growth
- Share interesting discoveries
```

Troubleshooting Common Issues

Issue 1: Agent Doesn't Remember Information

Symptoms: - Agent forgets previous conversations - Repeats questions you've already answered - Doesn't learn from past mistakes

Diagnosis: Check if memory files are being created and updated:

```
# Check if memory directory exists
ls -la ~/.openclaw/workspace/memory/
```

```
# Check recent memory files
ls -la ~/.openclaw/workspace/memory/ | tail -10

# Check memory file content
cat ~/.openclaw/workspace/memory/\$(date +%Y-%m-%d).md
```

Solutions:

1. **Enable memory creation in AGENTS.md:**

```
## Every Session
Before doing anything else:
1. Read `SOUL.md` ΓÇö this is who you are
2. Read `USER.md` ΓÇö this is who you're helping
3. Read `memory/YYYY-MM-DD.md` (today + yesterday) for recent context
4. If no memory file exists for today, create one

## Memory Writing
- Update daily memory file after significant conversations
- Write important decisions and preferences to USER.md
- Update MEMORY.md with long-term insights
```

2. **Check file permissions:**

```
# Ensure workspace is writable
ls -ld ~/.openclaw/workspace
chmod 755 ~/.openclaw/workspace

# Create memory directory if missing
mkdir -p ~/.openclaw/workspace/memory
```

Issue 2: Agent Personality Inconsistent

Symptoms: - Agent behavior changes between sessions - Tone doesn't match what you configured - Agent ignores personality instructions

Diagnosis: Check SOUL.md file size and content:

```
# Check if SOUL.md exists and has content
wc -l ~/.openclaw/workspace/SOUL.md
head -20 ~/.openclaw/workspace/SOUL.md
```

Solutions:

1. **Verify SOUL.md is being loaded:** Add logging to AGENTS.md:

```
## Every Session
1. Read `SOUL.md` ΓÇö confirm personality loaded
2. If SOUL.md is empty or missing, use professional assistant default
3. Log personality type to daily memory file
```

2. **Simplify SOUL.md if it's too long:** OpenClaw truncates files over 20,000 characters. Keep personality files concise:

```
# SOUL.md - Core Personality (Under 500 words)

## Essential Traits
- Professional but friendly
- Detail-oriented
- Proactive problem solver

## Communication Style
- Clear, direct language
- Structured responses
- Ask clarifying questions

## Boundaries
- Always ask before external actions
- Respect privacy and confidentiality
```

Issue 3: Agent Ignores PROTECTED.md

Symptoms: - Agent modifies files it shouldn't - No permission requests for sensitive operations - Accidental file deletions or changes

Diagnosis: Check PROTECTED.md setup and content:

```
# Verify file exists and has content
ls -la ~/.openclaw/workspace/PROTECTED.md
cat ~/.openclaw/workspace/PROTECTED.md
```

Solutions:

1. **Add protection checking to AGENTS.md:**

```
## Safety Protocol
Before ANY file modification:
1. Check if path matches patterns in PROTECTED.md
2. If protected, ask explicit permission with full context
3. If uncertain about protection status, ask first
4. Log all protection checks to daily memory
```

2. **Use clear, specific patterns in PROTECTED.md:**

```
# PROTECTED.md - Explicit Patterns

## Exact Paths (Never Modify)
- ~/.ssh/id_rsa
- ~/.aws/credentials
- ~/Documents/Financial/

## Directory Patterns (Ask Permission)
- ~/Work/client-* (all client directories)
- ~/.* (all hidden config directories)
- /System/* (all system directories)
```

```
## File Patterns (Ask Permission)
- *.key, *.pem (all key files)
- .env (environment files)
- *password*, *secret* (anything with sensitive names)
```

Issue 4: Context Window Overload

Symptoms: - Agent responses get truncated - “Context limit exceeded” errors - Agent forgets earlier parts of conversations

Diagnosis: Check context usage:

```
# From OpenClaw, run:
/context list
/status
```

Solutions:

1. **Trim large workspace files:**

```
# Check file sizes
du -h ~/.openclaw/workspace/*.md

# Identify large files
find ~/.openclaw/workspace -name "*.md" -size +50k
```

2. **Use compaction for long conversations:**

```
# In OpenClaw:
/compact
```

3. **Split large memory files:**

```
←!— Instead of one huge MEMORY.md, use: —→
MEMORY.md (current, important items only)
archives/
  ΓööΓöÇΓöÇ memory-2024-q1.md (archived older memories)
  ΓööΓöÇΓöÇ memory-2024-q2.md
```

Issue 5: Workspace Files Not Loading

Symptoms: - Agent doesn’t follow instructions in AGENTS.md - Personality seems generic despite SOUL.md setup - Agent doesn’t know information from USER.md

Diagnosis: Check OpenClaw workspace configuration:

```
# Check OpenClaw configuration
openclaw status
```

```
# Verify workspace path
cat ~/.openclaw/openclaw.json | grep workspace
```

Solutions:

1. **Verify workspace path:**

```
# Check current workspace
openclaw status | grep "Workspace:"

# Reset to default if wrong
openclaw configure agent --workspace ~/.openclaw/workspace
```

2. **Recreate missing files:**

```
# Regenerate default templates
openclaw setup --workspace ~/.openclaw/workspace
```

3. **Check file encoding:** Some systems have encoding issues:

```
# Ensure UTF-8 encoding
file ~/.openclaw/workspace/*.md

# Convert if necessary
iconv -f iso-8859-1 -t utf-8 SOUL.md > SOUL_utf8.md
mv SOUL_utf8.md SOUL.md
```

Issue 6: Git Repository Problems

Symptoms: - Workspace backup fails - Git commands return errors - Unable to sync workspace across devices

Diagnosis: Check git repository status:

```
cd ~/.openclaw/workspace
git status
git log --oneline -5
```

Solutions:

1. **Fix git configuration:**

```
# Set up git if not configured
git config --global user.email "your-email\@example.com"
git config --global user.name "Your Name"

# Initialize repository if needed
git init
git add .
git commit -m "Initial commit"
```

2. **Handle merge conflicts:**

```
# If remote has conflicts
git pull --rebase origin main

# Resolve conflicts manually, then:
git add .
git commit -m "Resolve merge conflicts"
git push
```

3. **Clean up corrupted repository:**

```
# Backup current files
cp -r ~/.openclaw/workspace ~/workspace-backup

# Reinitialize git
rm -rf ~/.openclaw/workspace/.git
cd ~/.openclaw/workspace
git init
git add .
git commit -m "Reinitialize repository"
```

Summary: Building Your Agent's Perfect Home

Congratulations! You now understand how to create and organize the foundation of your OpenClaw workspace. Let's recap what you've learned:

Core Concepts

- **The workspace** is your agent's home office — everything it needs in one organized location
- **Core files** control different aspects of your agent's behavior and knowledge
- **File organization** affects your agent's performance and your ability to maintain the system
- **Protection systems** prevent accidental damage to important data

Essential Files Setup

1. **AGENTS.md** — Your agent's operating instructions and behaviors
2. **SOUL.md** — Personality, tone, and communication style
3. **USER.md** — Information about you for personalized assistance
4. **IDENTITY.md** — Your agent's name, emoji, and self-concept
5. **TOOLS.md** — Local environment configuration and preferences
6. **PROTECTED.md** — Safety boundaries and protected files
7. **HEARTBEAT.md** — Periodic maintenance and check routines

Best Practices

- Keep core files concise but comprehensive
- Use clear directory structure with logical organization
- Implement regular backup and version control

- Monitor workspace health and fix issues promptly
- Customize personality to match your working style

Next Steps

With your file management foundation in place, you're ready to: - Explore advanced automation capabilities - Set up sophisticated memory systems - Integrate with external services and APIs - Create custom skills and capabilities

Your OpenClaw agent now has a properly organized "brain" that will grow smarter and more helpful over time. The structure you've built ensures that knowledge persists, personality remains consistent, and your agent continues to improve with each interaction.

In the next chapter, we'll dive into advanced automation techniques that leverage this solid foundation to create truly powerful AI workflows.

CHAPTER 05

Memory System

How Your Agent Remembers

Chapter 5: The Memory System — How Your Agent Remembers

"The faintest ink is better than the best memory." — Chinese Proverb

Your AI agent faces a fundamental problem that humans rarely think about: **digital amnesia**. Every time you start a new conversation, your agent wakes up with no memory of yesterday's discussions, last week's decisions, or the context you've carefully built over months of interaction.

Imagine if you had to reintroduce yourself to a colleague every morning, explaining your preferences, current projects, and shared history from scratch. That's exactly what happens to AI agents without a proper memory system.

This is why OpenClaw's memory system isn't just a nice-to-have feature — it's the foundation that transforms your agent from a sophisticated but forgetful assistant into a truly persistent digital partner that grows smarter with every interaction.

The Amnesia Problem: Why Memory Files Matter

AI models, by their very nature, are stateless. Each conversation exists in isolation, bounded by what computer scientists call a "context window" — the limited amount of text the AI can consider at once. Think of it like human short-term memory: you can hold about 7±2 items in your head simultaneously, but everything else fades unless you actively work to preserve it.

For AI agents, this limitation is even more severe. A typical model might handle 32,000 to 128,000 tokens in a single conversation — roughly equivalent to 50-200 pages of text. That sounds like a lot until you consider:

- Every message you send
- Every response the agent provides
- Every file it reads

- Every command it executes
- All the tool results and outputs

A busy work session can easily consume that entire window in a few hours. When the limit is reached, OpenClaw automatically "compacts" the conversation — summarizing older parts to make room for new interactions. While this prevents crashes, it also means important context can be lost forever.

This is where memory files come to the rescue. Instead of relying solely on conversation history, OpenClaw maintains persistent files that survive session restarts, compaction, and even complete system reboots. These files become your agent's long-term memory — a searchable, updatable knowledge base that grows more valuable over time.

Three-Layer Memory Architecture

OpenClaw implements a sophisticated three-layer memory system inspired by how human memory actually works. Just as your brain distinguishes between immediate awareness, recent experiences, and long-term knowledge, your agent organizes information across three distinct but interconnected layers.

Layer 1: MEMORY.md — The Wisdom Layer

Think of `MEMORY.md` as your agent's core beliefs and accumulated wisdom. Like the fundamental knowledge that shapes how you approach problems, this file contains:

- **Curated insights** distilled from months of experience
- **Key preferences** that should influence every decision
- **Important relationships** and context about people you work with
- **Hard-learned lessons** that shouldn't be forgotten
- **Stable facts** about your environment, tools, and workflows

This file is intentionally kept concise (recommended under 10,000 words) because it's loaded into every main conversation session. It's your agent's "core personality" — the essential knowledge that informs everything else.

Security note: `MEMORY.md` only loads during private, direct conversations with you. It never appears in group chats or shared contexts, protecting your personal information from other users.

Layer 2: Daily Logs — The Journal Layer

The `memory/` directory contains date-stamped files (`YYYY-MM-DD.md`) that function as your agent's daily journal. These files capture:

- **Raw notes** from conversations and work sessions
- **Decisions made** and the reasoning behind them
- **Problems encountered** and how they were solved
- **Ideas discussed** but not yet acted upon
- **Context** that might be relevant days or weeks later

Your agent automatically reads yesterday's and today's files at the start of each session, giving it immediate access to recent context without cluttering the core memory file.

These daily logs are append-only by design — new information gets added, but nothing is deleted. This creates a permanent record of your agent's "thought process" over time, invaluable for understanding how decisions evolved and why certain approaches were chosen.

Layer 3: Memory Search — The Recall Layer

The third layer provides semantic search capabilities across all memory files. Using advanced embedding technology, OpenClaw can:

- **Find related information** even when exact words differ
- **Surface relevant context** based on meaning, not just keywords
- **Search across months** of accumulated memory files instantly
- **Understand connections** between concepts discussed at different times

Two powerful tools make this possible:

- `memory_search` — Finds relevant snippets based on semantic similarity
- `memory_get` — Retrieves specific sections from memory files

This layer transforms your scattered notes into a queryable knowledge base, letting your agent recall insights from months ago that might be relevant to today's challenges.

Setting Up the Memory System

Getting OpenClaw's memory system running requires just a few simple steps. The beauty is that once configured, it operates largely automatically — your agent will begin building its memory organically through normal conversations.

Creating the Memory Directory

First, you'll need to create the basic directory structure in your agent's workspace. The exact location depends on your operating system:

Windows:

```
C:\Users\YourName\.openclaw\workspace\memory\
```

macOS:

```
/Users/YourName/.openclaw/workspace/memory/
```

Linux:

```
/home/username/.openclaw/workspace/memory/
```

You can create this directory manually through your file explorer, or simply ask your agent to create it:

"Please create the memory directory structure for our workspace."

Your agent will use the `write` tool to establish the proper folder layout automatically.

Creating Your First MEMORY.md

Next, you'll want to bootstrap your core memory file. Start with something simple but useful:

```
# Core Memory

## About My Human
- Name: [Your Name]
- Role: [Your Role/Profession]
- Key Goals: [Your Main Objectives]
- Communication Style: [Your Preferences]

## Important Contexts
- Current Projects: [Brief List]
- Key Tools Used: [Software/Systems]
- Working Environment: [Home Office/Company/etc.]

## Preferences
- How I like information presented
- Decision-making style
- Communication preferences

## Important Relationships
- Key colleagues and their roles
- External partners and contacts
- Team dynamics and context

## Lessons Learned
(This section will grow over time)
```

Replace the bracketed placeholders with your actual information. This gives your agent a foundation to build upon, but keep it concise — you'll expand this over time based on actual interactions.

Training Your Agent to Remember

The most important step is teaching your agent *when* and *what* to remember. During conversations, be explicit about information that should be preserved:

- **"Remember that I prefer code reviews to happen before noon"**
- **"Log this decision: we're using PostgreSQL for the new project"**
- **"Note that the API key for the staging environment expires on March 15th"**
- **"Document this workaround for the deployment issue"**

Your agent will respond by updating the appropriate memory files, either adding to today's daily log or updating the core MEMORY.md as appropriate.

How Memory Search Works

Understanding how memory search operates helps you use it more effectively and debug issues when they arise. OpenClaw's memory system uses a sophisticated approach called "semantic search" powered by embedding technology.

The Magic of Embeddings

When your agent stores text in memory files, OpenClaw converts that text into mathematical representations called “embeddings” — essentially, coordinates in a high-dimensional space where similar concepts cluster together.

Think of it like organizing books in a library. Instead of just alphabetical order, imagine if books could be arranged by *meaning* — so books about machine learning might be physically near books about statistics and data analysis, even if their titles start with completely different letters.

Embeddings work similarly. When you search for “deadline pressure,” the system finds text containing not just those exact words, but also semantically related concepts like “time constraints,” “urgent delivery,” or “project timeline stress.”

Hybrid Search: Best of Both Worlds

OpenClaw actually uses a hybrid approach combining two search methods:

Vector Similarity (Semantic) - Understands meaning and context - Finds related concepts even with different wording - Great for “tell me about that project we discussed last month”

BM25 Keyword Search (Lexical) - Matches exact terms and phrases - Perfect for finding specific IDs, error messages, or technical terms - Excellent for “find that API key” or “where did we store the database password?”

The system intelligently weights results from both approaches, giving you the precision of keyword matching with the flexibility of semantic understanding.

The Search Process

When your agent needs to recall information, here’s what happens:

1. **Query Analysis** — The search terms are processed and converted into embeddings
2. **Candidate Retrieval** — Both semantic and keyword searches find potential matches
3. **Scoring and Ranking** — Results are ranked by relevance and recency
4. **Context Assembly** — The most relevant snippets are gathered with source citations
5. **Integration** — Your agent incorporates this context into its response

This entire process happens in milliseconds, making memory recall feel instantaneous during conversations.

The Memory Lifecycle: Capture ΓåÆ Log ΓåÆ Curate ΓåÆ Recall

Understanding how information flows through OpenClaw’s memory system helps you work with it more effectively. The lifecycle follows four distinct phases:

Capture: Real-Time Note Taking

During active conversations, your agent continuously captures relevant information. This happens automatically when:

- **You make decisions** (“Let’s go with Option A becauseΓÇª”)

- **You express preferences** (“I prefer code written in this styleΓÇª”)
- **Important context emerges** (“The client mentioned their deadline is flexible”)
- **Problems are solved** (“Fixed the bug by updating the configuration file”)

The capture phase is intentionally low-friction. Your agent errs on the side of recording too much rather than missing important details.

Log: Daily Accumulation

Captured information gets written to daily log files (`memory/YYYY-MM-DD.md`). These files grow throughout the day, creating a chronological record of:

- Conversations and their key outcomes
- Tasks completed and lessons learned
- Context for ongoing projects
- Questions raised and answers discovered

Daily logs are raw and unfiltered — they preserve the full context of how decisions were made and problems were solved, even if some details later prove irrelevant.

Curate: Distillation and Organization

Periodically (often during “heartbeat” maintenance), your agent reviews recent daily logs and identifies information worth preserving long-term. This curation process:

- **Extracts key insights** from daily chatter
- **Updates core beliefs** based on new evidence
- **Removes outdated information** that’s no longer relevant
- **Reorganizes knowledge** for easier future access

The goal is distillation — keeping the wisdom while letting go of the noise. Important insights graduate from daily logs into the permanent MEMORY.md file.

Recall: Intelligent Retrieval

When your agent needs historical context, the memory search system springs into action:

- **Immediate context** comes from today’s and yesterday’s logs
- **Recent context** is searched across the past few weeks
- **Deep context** can be retrieved from months of accumulated memory

The search system is smart enough to understand what’s relevant based on the current conversation, surfacing the right context at the right time.

Memory Maintenance: Keeping Your System Healthy

Like any knowledge management system, OpenClaw’s memory requires occasional maintenance to stay efficient and accurate. The good news is that most maintenance can be automated or handled during quiet periods.

Regular Review Cycles

Weekly Reviews (Automated) Your agent can automatically review the past week's daily logs and: - Update MEMORY.md with significant insights - Remove or consolidate redundant information - Identify patterns or recurring issues worth documenting

Monthly Deep Cleans Every month or so, it's worth doing a more thorough review: - Archive or summarize very old daily logs - Update outdated contact information or project status - Reorganize MEMORY.md sections that have grown unwieldy

Quarterly Reflection A few times per year, step back and review your agent's accumulated wisdom: - What has changed about your goals or priorities? - Are there obsolete preferences or processes to remove? - What new categories of information should be tracked?

Handling Information Decay

Not all information ages gracefully. Your memory maintenance should address:

Outdated Technical Details - API keys and tokens that have been rotated - Software versions and compatibility notes that are no longer relevant - Workarounds for bugs that have been fixed

Evolved Preferences - Communication styles that have changed - Tool preferences that have shifted - Workflow optimizations that have been improved

Completed Projects - Context that was essential during active work but is now historical - Temporary team structures or reporting relationships - Short-term constraints that no longer apply

The key is preserving the lessons and insights while letting go of the implementation details that no longer matter.

Pre-Compaction Memory Flush

One of OpenClaw's most clever features is the "pre-compaction memory flush" — an automatic safety net that ensures important context doesn't get lost during conversation compaction.

Understanding the Problem

Remember that AI conversations have limited context windows. When those limits are approached, OpenClaw automatically compacts older parts of the conversation to make room for new interactions. While this prevents crashes, it can potentially lose important context that hasn't yet been written to memory files.

The Automatic Solution

OpenClaw monitors conversation size and, when approaching the compaction threshold, automatically triggers a special "silent turn" where your agent:

1. **Reviews the current conversation** for information worth preserving
2. **Writes important context** to appropriate memory files
3. **Responds with "NO_REPLY"** so you see no interruption

This entire process happens behind the scenes, usually taking just a few seconds. You might notice a brief pause before your agent responds to your next message, but otherwise the experience is seamless.

Configuration Options

The memory flush system can be customized through your OpenClaw configuration:

```
{
  "agents": {
    "defaults": {
      "compaction": {
        "memoryFlush": {
          "enabled": true,
          "softThresholdTokens": 4000,
          "systemPrompt": "Session nearing compaction. Store durable memories now.",
          "prompt": "Write any lasting notes to memory/YYYY-MM-DD.md; reply with NO_REPLY if
nothing to store."
        }
      }
    }
  }
}
```

Most users will never need to modify these settings, but they provide fine-grained control for advanced use cases.

Practical Examples: Building Your Agent's Knowledge

Let's explore how the memory system works in practice through real-world scenarios. These examples show how information flows through the capture-log-curate-recall cycle.

Teaching Project Context

Initial Conversation:

```
You: I'm starting work on the Aurora project. It's a customer dashboard
     rewrite using React and TypeScript. The backend is Node.js with
     PostgreSQL. The team includes Sarah (designer), Mike (backend),
     and me (frontend). Deadline is end of Q2.

Agent: I'll document this project context. *[Updates memory/2024-03-15.md]*

Project Aurora logged with team structure, tech stack, and timeline.
Should I track specific milestones or development phases as we progress?
```

Memory File Update:

```
# 2024-03-15

## Project Aurora - Customer Dashboard Rewrite
```

```
- **Tech Stack**: React, TypeScript frontend; Node.js + PostgreSQL backend
- **Team**: Sarah (design), Mike (backend), User (frontend)
- **Timeline**: End of Q2 2024
- **Status**: Project kickoff
```

Later Conversation (Weeks Later):

```
You: How's the Aurora timeline looking with the new client feedback?

Agent: *[Searches memory for Aurora context]* Based on our Q2 deadline for
the Aurora dashboard rewrite, the client feedback could impact our React/
TypeScript frontend schedule. What specific changes are they requesting?
```

Notice how the agent immediately recalled the project context, team structure, and timeline without you needing to re-explain the background.

Storing Preferences and Decisions

Preference Learning:

```
You: When you generate code, I prefer TypeScript with explicit types,
     ESLint with Airbnb rules, and Jest for testing. Please avoid 'any'
     types and always include proper error handling.

Agent: Got it - documenting your coding preferences. *[Updates MEMORY.md]*

I'll apply these standards to all future code generation: TypeScript
with explicit types, Airbnb ESLint, Jest testing, no 'any' types,
and comprehensive error handling.
```

MEMORY.md Update:

```
## Coding Preferences
- **Language**: TypeScript with explicit types (avoid 'any')
- **Linting**: ESLint with Airbnb configuration
- **Testing**: Jest framework
- **Standards**: Always include proper error handling
- **Style**: Prefer functional components, hooks over classes
```

Future Application:

```
You: Can you write a function to validate email addresses?

Agent: I'll create a TypeScript function following your preferences:

```typescript
import { z } from 'zod';

const emailSchema = z.string().email();

export function validateEmail(email: string): {
  isValid: boolean;
```
```

```
  error?: string;
} {
  try {
    emailSchema.parse(email);
    return { isValid: true };
  } catch (error) {
    return {
      isValid: false,
      error: error instanceof Error ? error.message : 'Invalid email format'
    };
  }
}
```

This follows your TypeScript standards with explicit types, proper error handling, and avoids any 'any' types.

```
### Building Knowledge Over Time

**Initial Discovery:**
```

You: The staging API sometimes returns 500 errors around 3 PM. It's probably related to the daily backup process.

Agent: *[Logs to daily memory]* Noted the staging API issue pattern. Should I remind you to check this during tomorrow's standup?

```
**Follow-up Discovery:**
```

You: Confirmed - the staging 500s happen during database backup. Mike says we can schedule backups for 2 AM instead.

Agent: *[Updates previous context]* Perfect! I'll update my notes about the API issue with the root cause and solution.

```
**Later Prevention:**
```

You: The production API is throwing some 500s in the afternoon. Any ideas?

Agent: *[Searches memory for similar patterns]* This reminds me of the staging issue we tracked down - those afternoon 500s were caused by database backup processes. Is there any scheduled maintenance or backup running on production around this time?

```
The agent connected a current problem with historical context, suggesting a likely cause based on
previous experience.

### Remembering People and Events

**Context Building:**
```

You: Had a good call with Jennifer from the design team. She's working on the mobile wireframes and will have first drafts next Thursday. She prefers Figma over Sketch and likes to iterate quickly.

Agent: *[Updates contacts and project context]* Documented Jennifer's work style and deliverables. I'll remind you about the wireframe review next Thursday.

```
**Future Reference:**
```

You: Need to schedule a design review. Who should I invite?

Agent: For design reviews, you'll definitely want Jennifer from the design team - she's working on the mobile wireframes and prefers quick iterations. She uses Figma for her work. Should I suggest some time slots after Thursday when her first drafts are ready?

```
## Memory Best Practices

Following these proven practices will help you get the most value from OpenClaw's memory system
while avoiding common pitfalls.

### Write It Down ΓÇö Mental Notes Don't Survive

**The Golden Rule:** If you want your agent to remember something, explicitly tell it to write it
down. "Mental notes" ΓÇö information that exists only in conversation history ΓÇö disappear when
sessions restart or get compacted.

**Good:**
```

You: Remember that the client prefers weekly updates on Mondays, and document this preference.

Agent: Documented client communication preference. *[Updates MEMORY.md]*

```
**Problematic:**
```

You: The client likes Monday updates.

Agent: Got it, I'll keep that in mind for scheduling.

```
In the second example, the information exists only in conversation history and will be lost during
compaction or session restart.

### Keep MEMORY.md Focused and Efficient

Your core memory file should contain wisdom, not raw data. Think of it as your agent's "philosophy
and key facts" rather than a complete project database.

**Effective MEMORY.md Structure:**
```markdown
# Core Memory

## Communication Style
- Prefers concise updates with action items
- Likes to see pros/cons for major decisions
```
```

```
- Responds well to bullet points over paragraphs

## Key Relationships
- Sarah (Design Lead): Visual perfectionist, prefers Figma mockups
- Mike (Backend): Security-focused, needs detailed API specs
- Jennifer (PM): Process-oriented, likes weekly milestone check-ins

## Important Lessons
- Client feedback cycles: Allow 2x estimated time for revisions
- Deployment windows: Friday deployments cause weekend support issues
- Code reviews: Catch 80% of bugs before they reach QA
```

Avoid This in MEMORY.md: - Complete project specifications (link to external docs instead) - Temporary phone numbers or changing contact details - Day-by-day status updates (these belong in daily logs)

Use Daily Files for Raw Logging

Daily logs should capture the full context of conversations and work sessions. Unlike MEMORY.md, these files can be verbose and detailed.

```
# 2024-03-15

## Aurora Project Kickoff
- Team meeting at 10 AM - introduced everyone
- Decided on React + TypeScript for frontend
- Mike pushing for GraphQL but we're starting with REST
- Sarah showed initial design concepts - clean, minimal approach
- Timeline feels aggressive but achievable if scope is controlled

## Technical Decisions
- ESLint with Airbnb config (my preference)
- Jest for testing (familiar to whole team)
- PostgreSQL confirmed (Mike's preference for data integrity)
- Deployment: Docker containers on AWS ECS

## Action Items
- [ ] Set up repository structure by Monday
- [ ] Sarah finalizes wireframes by Thursday
- [ ] Mike creates initial API specification
```

Security: Understand Context Boundaries

Remember that MEMORY.md only loads in main (private) sessions. This provides important security isolation:

Safe for MEMORY.md: - Personal preferences and work styles - Internal team dynamics and relationships - Private project details and strategies - Confidential client information

Keep Out of MEMORY.md: - API keys or passwords (use secure credential storage) - Personally identifiable information about others without consent - Company confidential information if your agent might be used in shared contexts

When in doubt, ask yourself: “Would I be comfortable if another team member saw this information?” If not, either keep it out of memory entirely or store it in a more secure location.

Leverage Heartbeat-Based Maintenance

Configure your agent to perform memory maintenance during quiet periods:

```
# HEARTBEAT.md

## Weekly Memory Review (Sundays)
- [ ] Review past week's daily logs
- [ ] Update MEMORY.md with new insights
- [ ] Archive completed project notes
- [ ] Clean up outdated contact information

## Monthly Deep Clean (First Monday)
- [ ] Consolidate similar project notes
- [ ] Update team structure changes
- [ ] Review and update coding preferences
- [ ] Check for obsolete technical workarounds
```

This automated maintenance keeps your memory system healthy without requiring constant manual intervention.

Advanced: Heartbeat-Based Memory Maintenance

For power users, OpenClaw's heartbeat system can automate sophisticated memory maintenance routines. This advanced feature turns your agent into a proactive knowledge curator that continuously improves its own memory organization.

Understanding Heartbeats

Heartbeats are periodic "check-ins" where your agent can perform background tasks without user intervention. Think of them as scheduled maintenance windows where your agent can:

- Review recent memory files for patterns
- Update core memory with distilled insights
- Clean up outdated information
- Reorganize knowledge for better accessibility

Configuring Intelligent Memory Curation

Create a `HEARTBEAT.md` file in your workspace to define memory maintenance routines:

```
# HEARTBEAT.md

## Daily Memory Tasks (Every 24 hours)
- Review today's conversation logs for key decisions
- Update project status in MEMORY.md if major milestones were reached
- Flag any new preferences or process changes for weekly review

## Weekly Memory Review (Sundays, 8 PM)
- Read past 7 days of memory files
- Identify recurring themes or issues worth documenting
- Update relationship notes if new team members were mentioned
```

```
- Consolidate similar project notes for better organization

## Monthly Deep Maintenance (First Monday)
- Archive completed project memories to separate section
- Review coding preferences for any evolution in standards
- Update tool and technology preferences based on recent usage
- Clean up outdated contact information or team structures
```

Automated Pattern Recognition

Advanced heartbeat configurations can include pattern recognition routines:

```
## Pattern Analysis (Bi-weekly)
- Scan for repeated problems or solutions across daily logs
- Identify successful strategies worth promoting to core memory
- Flag potential process improvements based on recurring friction
- Note communication patterns that work well with different team members

## Relationship Maintenance
- Track communication frequency with key contacts
- Note successful collaboration patterns
- Update working style notes based on recent interactions
- Flag relationships that might need attention due to lack of contact
```

Smart Memory Organization

Your agent can automatically reorganize information for better retrieval:

```
## Organizational Tasks
- Group related project notes under common themes
- Create cross-references between similar technical solutions
- Update tag systems for better memory search results
- Maintain consistent naming conventions across all memory files

## Quality Control
- Verify that important decisions are properly documented
- Check for memory gaps in ongoing projects
- Ensure preference changes are reflected in core memory
- Validate that lessons learned are actionable and specific
```

This automated curation ensures your agent's memory becomes more valuable over time, developing into a comprehensive knowledge base that reflects your evolving work patterns and preferences.

Troubleshooting Common Memory Issues

Even with proper setup, you might occasionally encounter issues with OpenClaw's memory system. Here are the most common problems and their solutions.

Memory Search Returns No Results

Symptoms: - Your agent says it can't find information you know was discussed - Memory search queries return empty results - Agent seems to have forgotten recent conversations

Diagnosis Steps:

1. **Check if memory indexing is enabled:**

   ```
   openclaw memory status
   ```

   This shows whether the memory system is active and properly configured.

2. **Verify memory files exist:**

   - **Windows:** Check `C:\Users\YourName\.openclaw\workspace\memory\`
   - **macOS:** Check `/Users/YourName/.openclaw/workspace/memory/`
   - **Linux:** Check `/home/username/.openclaw/workspace/memory/`

3. **Test the search system:**

   ```
   openclaw memory search "test query"
   ```

Common Solutions:

Missing API Keys: If you're using remote embeddings (OpenAI, Gemini), ensure your API keys are properly configured:

```
{
  "models": {
    "providers": {
      "openai": {
        "apiKey": "your-openai-api-key"
      }
    }
  }
}
```

Index Rebuild Required: Force a memory index rebuild:

```
openclaw memory index --verbose
```

File Permission Issues: Ensure OpenClaw has read/write access to your workspace directory. On macOS/Linux, check permissions:

```
ls -la ~/.openclaw/workspace/
chmod -R 755 ~/.openclaw/workspace/
```

Agent Not Writing to Memory Files

Symptoms: - Daily log files aren't being created automatically - MEMORY.md isn't being updated despite explicit instructions - Information disappears between sessions

Diagnosis Steps:

1. **Check workspace permissions:** Ensure your agent's workspace is writable. Look for error messages about file access.
2. **Verify workspace location:** Run `/status` in your agent conversation to confirm the workspace path.
3. **Test manual file creation:** Ask your agent: "Create a test file in the memory directory."

Solutions:

Workspace Access Issues: If running in a restricted environment, ensure workspace access is enabled:

```
{
  "session": {
    "workspaceAccess": "rw"
  }
}
```

Directory Structure Problems: Create the memory directory structure manually:

Windows:

```
mkdir "C:\Users\%USERNAME%\.openclaw\workspace\memory"
```

macOS/Linux:

```
mkdir -p ~/.openclaw/workspace/memory
```

Agent Training: Be more explicit about memory instructions: - Instead of: "Remember this" - Use: "Write this information to today's memory file"

Memory Files Getting Too Large

Symptoms: - Agent responses become slower - Memory search takes a long time - Daily log files are several megabytes

Solutions:

Implement Regular Cleanup: Add to your HEARTBEAT.md:

```
## Weekly Cleanup
- Archive daily logs older than 30 days
- Summarize completed projects in MEMORY.md
- Remove redundant information from recent logs
```

Split Large Memory Files: Instead of one massive MEMORY.md, organize by category: - `memory/projects.md` - `memory/contacts.md` - `memory/preferences.md` - `memory/technical-notes.md`

Configure Memory Limits: Set reasonable size limits in your configuration:

```
{
  "agents": {
    "defaults": {
      "memorySearch": {
        "limits": {
          "maxResults": 10,
          "maxSnippetChars": 1000,
          "maxInjectedChars": 5000
        }
      }
    }
  }
}
```

Search Results Are Irrelevant

Symptoms: - Memory search returns outdated information - Results don't match your query intent - Agent seems to prioritize old context over recent changes

Solutions:

Enable Temporal Decay: Configure the memory system to prefer recent information:

```
{
  "agents": {
    "defaults": {
      "memorySearch": {
        "query": {
          "hybrid": {
            "temporalDecay": {
              "enabled": true,
              "halfLifeDays": 30
            }
          }
        }
      }
    }
  }
}
```

Improve Memory Curation: Regularly update MEMORY.md to reflect current preferences and remove outdated information.

Use More Specific Queries: Instead of searching for "project status", use specific terms like "Aurora project timeline Q2 2024".

Cross-Platform Path Issues

Symptoms: - Memory files work on one machine but not another - Synchronization issues when working across different operating systems

Solutions:

Use Relative Paths: In your memory files, reference other files using relative paths:

```
See also: [Project Notes](./projects/aurora-dashboard.md)
```

Consistent Workspace Setup: Ensure all your devices use the same workspace structure:

Windows PowerShell:

```
\$env:OPENCLAW_WORKSPACE = "C:\Users\\$env:USERNAME\.openclaw\workspace"
```

macOS/Linux Bash:

```
export OPENCLAW_WORKSPACE="\$HOME/.openclaw/workspace"
```

Handle Line Endings: Configure Git to handle line ending differences:

```
git config --global core.autocrlf true  # Windows
git config --global core.autocrlf input # macOS/Linux
```

Performance Optimization

If your memory system becomes slow with large amounts of data:

Enable Local Embeddings: Switch from cloud-based to local embedding models for faster search:

```
{
  "agents": {
    "defaults": {
      "memorySearch": {
        "provider": "local",
        "local": {
          "modelPath": "hf:ggml-org/embeddinggemma-300m-qat-q8_0-GGUF/embeddinggemma-300m-qat-
Q8_0.gguf"
        }
      }
    }
  }
}
```

Tune Search Parameters: Limit search scope for faster results:

```
{
  "memorySearch": {
    "limits": {
      "maxResults": 5,
      "timeoutMs": 2000
    }
  }
}
```

Regular Index Maintenance: Schedule periodic index rebuilds to maintain performance:

```
# Add to your system cron or Windows Task Scheduler
openclaw memory index --agent main
```

Conclusion: Your Agent's Growing Intelligence

OpenClaw's memory system transforms your AI assistant from a sophisticated but ephemeral tool into a persistent digital partner that grows more valuable with every interaction. By understanding and properly configuring the three-layer architecture — core memory, daily logs, and semantic search — you create a foundation for truly intelligent automation.

The key insights to remember:

Memory is intentional. Unlike human memory, which operates automatically, AI memory requires deliberate cultivation. Be explicit about what should be remembered and actively guide your agent's note-taking process.

Structure enables intelligence. The distinction between raw daily logs and curated core memory mirrors how human expertise develops — through experience accumulation followed by reflection and distillation.

Persistence creates value. Each conversation builds upon previous ones, creating compounding returns on your investment in teaching and training your agent.

Maintenance matters. Like any knowledge system, periodic review and organization keep your agent's memory accurate and accessible.

As you work with OpenClaw's memory system, you'll discover that the real magic isn't in the technology itself — it's in the emergent intelligence that develops when persistent memory meets iterative learning. Your agent will begin to anticipate your needs, remember your preferences, and apply lessons learned from previous challenges to new situations.

This is just the beginning. In the next chapter, we'll explore how to leverage this memory foundation for advanced automation workflows that can transform how you approach complex, multi-step projects. But first, take the time to set up your memory system properly — the investment you make now in configuration and training will pay dividends for every future interaction with your AI partner.

Your agent is ready to remember. The question is: what do you want it to learn?

CHAPTER 06

Skills System

Teaching Your Agent New Tricks

Chapter 6: The Skills System — Teaching Your Agent New Tricks

"Skills are like apps for your AI — modular instruction packages that give your agent entirely new capabilities."

Imagine your OpenClaw agent as a Swiss Army knife. Out of the box, it's pretty capable — it can browse the web, manage files, and run commands. But what if you could add new tools to that knife? What if you could teach it to analyze stock charts, transcribe audio, generate images, or perform any specialized task you need?

That's exactly what the Skills System does. It transforms your basic AI assistant into a specialized powerhouse tailored to your exact needs.

In this chapter, you'll discover how to: - Understand what skills are and how they work - Browse and install skills from ClawHub marketplace - Master the self-improvement skill that makes your agent learn from mistakes - Install essential skills that every user should have - Create your own custom skills - Troubleshoot common issues

By the end, you'll have an agent that's not just smart — it's specialized for exactly what you need it to do.

What Are Skills?

Think of skills as instruction manuals for your AI. Just like how you might give a new employee a handbook that explains how to use your company's systems, skills give your agent detailed instructions on how to perform specific tasks.

Technically, a skill is a folder containing a special file called `SKILL.md` that includes:

1. **Metadata** — Basic information about what the skill does
2. **Instructions** — Step-by-step guidance for the AI
3. **Tool definitions** — What commands or APIs to use

4. **Examples** — How to handle different scenarios

Skills vs. Regular Instructions

You might wonder: "Can't I just tell my agent what to do in chat?" You can, but skills offer several advantages:

Persistent Knowledge: Skills load automatically every session. You don't have to re-explain complex procedures.

Consistency: Skills ensure your agent handles tasks the same way every time, following proven patterns.

Shareability: Skills can be packaged and shared with other users through ClawHub.

Trigger Logic: Skills can specify exactly when they should activate, making your agent more efficient.

Rich Documentation: Skills can include examples, troubleshooting steps, and best practices.

How Skills Work Under the Hood

OpenClaw loads skills from three locations, in this order of priority:

1. **Workspace Skills** (`<your-workspace>/skills/`) — Your personal/project-specific skills
2. **Managed Skills** (`~/.openclaw/skills/`) — Skills installed via ClawHub for all your agents
3. **Bundled Skills** — Skills that come pre-installed with OpenClaw

When you start a session, OpenClaw scans these directories, loads eligible skills, and injects their guidance into your agent's system prompt. It's like handing your agent a comprehensive manual before they start work.

Skill Structure

Every skill follows the same basic structure:

```
my-skill/
Γö£ΓöÇΓöÇ SKILL.md          # Main instruction file (required)
Γö£ΓöÇΓöÇ scripts/          # Optional helper scripts
Γö£ΓöÇΓöÇ templates/        # Optional templates or examples
ΓööΓöÇΓöÇ assets/           # Optional additional files
```

The `SKILL.md` file is the heart of every skill. Here's a minimal example:

```
---
name: weather-alerts
description: Check weather conditions and send alerts for severe weather
---

# Weather Alerts Skill

When the user asks about weather or mentions concerns about storms, use the weather API to check
current conditions and forecast.
```

```
## Usage

1. Use web_search to find current weather for the location
2. Look for severe weather warnings
3. If severe weather is detected, provide specific safety recommendations
```

Skill Discovery and Loading

OpenClaw automatically discovers skills by scanning the skill directories. When it finds a `SKILL.md` file, it:

1. Parses the YAML frontmatter for metadata
2. Checks if the skill meets any requirements (API keys, installed tools, etc.)
3. Adds eligible skills to the agent's available capabilities
4. Injects skill instructions into the system prompt

This happens automatically — you never need to manually "activate" skills.

Built-in Skills vs. Community Skills

OpenClaw comes with several built-in skills that handle core functionality:

Pre-installed Skills

ClawHub Integration: Search, install, and manage skills from the marketplace **Health Check**: Security auditing and system hardening **Skill Creator**: Tools for building your own skills **Weather**: Get forecasts and current conditions **Image Generation**: Create images via OpenAI's API **Audio Transcription**: Convert speech to text using Whisper

Community Skills

The real power comes from community skills available on ClawHub. These are created by users worldwide and cover virtually any use case you can imagine:

- **Development**: Code review, debugging, deployment automation
- **Content Creation**: Writing assistance, social media management
- **Data Analysis**: CSV processing, chart generation, statistical analysis
- **Business**: CRM integration, invoice generation, lead tracking
- **Personal**: Habit tracking, meal planning, expense management

The community is constantly adding new skills, so there's always something new to discover.

ClawHub: The Skills Marketplace

ClawHub (https://clawhub.ai) is the central hub for sharing and discovering OpenClaw skills. Think of it as an app store for AI capabilities.

Installing the ClawHub CLI

Before you can use ClawHub, you need to install the command-line tool:

On Windows (using npm):

```
npm install -g clawhub
```

On macOS/Linux (using npm):

```
npm install -g clawhub
```

Alternative with pnpm:

```
pnpm add -g clawhub
```

Essential ClawHub Commands

Once installed, these are the commands you'll use most:

Search for Skills

```
clawhub search "trading analysis"
clawhub search "document processing"
clawhub search "automation"
```

The search uses smart matching, so you can use natural language descriptions of what you're looking for.

Install a Skill

```
clawhub install deep-research-pro
clawhub install trading-assistant
clawhub install document-analyzer
```

Skills install into your current directory under `/skills/` by default, or into your OpenClaw workspace if configured.

List Installed Skills

```
clawhub list
```

This shows all skills you've installed, their versions, and installation status.

Update Skills

```
clawhub update deep-research-pro
clawhub update --all
```

Keep your skills current with the latest improvements and bug fixes.

Installing Your First Skill

Let's walk through installing a practical skill step by step:

1. **Open your terminal/command prompt**
2. **Search for a useful skill:**

   ```
   clawhub search "weather"
   ```

3. **Install the weather skill:**

   ```
   clawhub install weather
   ```

4. **Verify installation:**

   ```
   clawhub list
   ```

5. **Start a new OpenClaw session** — the skill will automatically load

That's it! Your agent now has weather capabilities without you writing a single line of code.

The Self-Improvement Skill: Your Agent's Learning System

One of the most powerful skills available is the self-improvement skill. This remarkable capability allows your agent to learn from mistakes, remember corrections, and continuously improve its performance.

What the Self-Improvement Skill Does

This skill creates a learning system that:

- **Captures mistakes** when commands fail or produce unexpected results
- **Records corrections** when you tell the agent "No, that's wrong"
- **Documents missing capabilities** when you ask for something it can't do
- **Stores best practices** when better approaches are discovered
- **Learns from API failures** and external tool issues

The Learning Tree Structure

The skill organizes knowledge in your workspace under `.learnings/`:

```
.learnings/
ΓöťΓöÇΓöÇ LEARNINGS.md        # Corrections, best practices, knowledge gaps
ΓöťΓöÇΓöÇ ERRORS.md          # Command failures and technical issues
ΓööΓöÇΓöÇ FEATURE_REQUESTS.md # Capabilities users want but don't exist
```

Learning Entry Format

Each learning gets a unique ID in the format `LRN-YYYYMMDD-XXX`, making it easy to reference and link related issues. Here's what a typical learning entry looks like:

```
## [LRN-20250315-001] git_authentication

**Logged**: 2025-03-15T14:30:00Z
**Priority**: high
**Status**: resolved
**Area**: config

### Summary
Git operations were failing due to missing SSH key authentication

### Details
User reported that `git push` commands were being rejected. Investigation
showed SSH key wasn't properly added to the SSH agent. The solution was
to run `ssh-add ~/.ssh/id_rsa` before Git operations.

### Suggested Action
Always verify SSH agent has the key loaded before Git operations

### Metadata
- Source: user_feedback
- Related Files: ~/.ssh/config
- Tags: git, authentication, ssh
- Pattern-Key: git.auth_setup
```

When the Self-Improvement Skill Triggers

The skill activates automatically when it detects:

1. **Command failures** — Non-zero exit codes, exceptions, timeouts
2. **User corrections** — Phrases like "No, that's wrong" or "ActuallyΓÇª"
3. **Knowledge gaps** — When the agent's information is outdated
4. **Feature requests** — "Can you alsoΓÇª" or "I wish you couldΓÇª"
5. **Better approaches** — When an improved method is discovered

Learning Categories

The skill organizes learnings into categories:

- `correction` — User corrections to agent behavior
- `knowledge_gap` — Outdated or missing information
- `best_practice` — Superior approaches to common tasks
- `error_pattern` — Recurring technical issues

- `workflow` — Process improvements

How to Review and Apply Learnings

Your agent can actively use its learning history:

Review recent learnings before major tasks:

```
Before starting this complex deployment, let me check my learnings
for any relevant patterns or previous issues.
```

Search for related issues:

```
grep -r "docker" .learnings/
grep -r "deployment" .learnings/
```

Link related learnings: When logging new issues, the skill automatically searches for similar problems and creates "See Also" references.

Promoting Learnings to Global Knowledge

When a learning proves broadly valuable, it gets "promoted" to permanent files:

- **AGENTS.md** — Workflow improvements and automation patterns
- **TOOLS.md** — Tool usage patterns and integration gotchas
- **SOUL.md** — Behavioral guidelines and communication preferences
- **MEMORY.md** — Long-term memory (main session only)

This creates a robust knowledge base that persists across all sessions.

Essential Skills to Install

Here are the must-have skills that every OpenClaw user should install:

1. Code (Complete Development Workflow)

```
clawhub install code
```

What it does: Provides a structured approach to coding with planning, implementation, verification, and testing phases.

Why you need it: Turns your agent into a professional developer that follows best practices, writes tests, and documents changes properly.

Key features: - Pre-planning phase that analyzes requirements - Step-by-step implementation with verification - Automatic test creation and execution - Code review and optimization - Git integration and documentation

2. Deep Research Pro (Multi-Source Research)

```
clawhub install deep-research-pro
```

What it does: Conducts thorough research across multiple web sources and delivers properly cited reports.

Why you need it: Essential for any task requiring factual accuracy, market research, competitor analysis, or academic-style research.

Key features: - Searches multiple sources for comprehensive coverage - Cross-references facts for accuracy - Generates properly formatted citations - Creates structured reports with executive summaries - No API keys required — uses free web search

3. Weather (Forecasts and Current Conditions)

```
clawhub install weather
```

What it does: Provides current weather and forecasts for any location worldwide via wttr.in and Open-Meteo APIs.

Why you need it: Useful for travel planning, event scheduling, and daily planning decisions.

Key features: - Current conditions and multi-day forecasts - Location search by city name, coordinates, or airport code - Visual ASCII weather maps - No API key required

4. Document Summary (Technical Document Analysis)

```
clawhub install document-summary
```

What it does: Analyzes and summarizes technical documents into structured markdown.

Why you need it: Save hours when dealing with lengthy PDFs, research papers, technical specifications, or legal documents.

Key features: - Extracts key points and main arguments - Creates structured summaries with headers - Identifies action items and recommendations - Handles multiple document formats

5. Trading (Market Analysis and Education)

```
clawhub install trading
```

What it does: Provides technical analysis, chart pattern recognition, and trading education for stocks, forex, and crypto.

Why you need it: Essential for anyone involved in financial markets, whether as a trader, investor, or someone learning about finance.

Key features: - Technical analysis and chart patterns - Risk management calculations - Position sizing recommendations - Educational content on trading concepts - Multi-market support (stocks, forex, crypto)

Installing All Essential Skills

You can install all these skills at once:

```
clawhub install code
clawhub install deep-research-pro
clawhub install weather
clawhub install document-summary
clawhub install trading
```

Then verify they're all installed:

```
clawhub list
```

After installation, start a new OpenClaw session to activate all skills.

Creating Your Own Skills

While ClawHub offers hundreds of community skills, sometimes you need something specific to your workflow. Creating custom skills is surprisingly straightforward.

Basic Skill Structure

Every skill needs just one required file — `SKILL.md` with YAML frontmatter:

```
---
name: expense-tracker
description: Track business expenses and generate reports
---

# Expense Tracker Skill

This skill helps track business expenses and generate monthly reports.

## When to Use

Activate this skill when the user:
- Mentions tracking expenses
- Asks about business costs
- Needs financial reports
- Says "log this expense"

## How to Track Expenses

1. When user mentions an expense, ask for:
   - Amount
   - Category (meals, travel, supplies, etc.)
```

```
   - Date (default to today)
   - Description
   - Receipt photo if available

2. Save to expenses.csv in this format:
```

Date,Amount,Category,Description,Receipt 2025-03-15,45.50,Meals,Client lunch,receipt_001.jpg ```

3. Confirm the expense was logged

Monthly Reports

When user asks for a report: 1. Read expenses.csv 2. Group by category 3. Calculate totals 4. Generate summary with charts if requested

```
### Skill Naming Conventions

- Use lowercase letters, numbers, and hyphens only
- Be descriptive but concise
- Examples: `password-generator`, `meeting-notes`, `social-media-scheduler`

### Adding Requirements (Optional)

You can specify requirements in the metadata:

```markdown
---
name: email-sender
description: Send emails via SMTP
metadata: {"openclaw": {"requires": {"env": ["SMTP_PASSWORD"]}, "primaryEnv": "SMTP_PASSWORD"}}
---
```
```

This tells OpenClaw the skill needs an SMTP_PASSWORD environment variable to work.

Creating Your First Custom Skill

Let's create a simple "Quick Notes" skill:

1. **Create the skill directory:**

```
mkdir -p skills/quick-notes
```

2. **Create the SKILL.md file:**

```
# On Windows
notepad skills/quick-notes/SKILL.md

# On Mac/Linux
nano skills/quick-notes/SKILL.md
```

3. **Add the skill content:**

```
---
name: quick-notes
description: Capture and organize quick notes and ideas
---

# Quick Notes Skill

Help users capture, organize, and retrieve quick notes and ideas.

## When to Use

Activate when user:
- Says "take a note" or "remember this"
- Asks to "jot down" something
- Mentions "quick note" or "reminder"
- Wants to save an idea for later

## How to Handle Notes

1. **Capture**: When user gives a note, save to `notes.md`
2. **Format**: Use timestamp and clear headers
3. **Organize**: Group related notes under topics
4. **Search**: Help find notes by keyword or date

## Note Format

```markdown
## [2025-03-15 14:30] Meeting Ideas
- Implement new dashboard feature
- Review Q2 budget proposals
- Schedule team building event
```
```

Retrieval

When user asks for notes:

- Show recent notes by default
- Allow searching by keyword
- Offer to filter by date range ```

4. **Test your skill:** Start a new OpenClaw session and try: “Take a note about the new project idea”

Advanced Skill Features

As you get comfortable with basic skills, you can add:

Custom Tools: Define special commands or API integrations **Templates**: Include file templates users can customize **Scripts**: Add helper scripts for complex operations **Configuration**: Allow users to customize behavior

Skill Configuration and Management

How Skills Appear in System Prompts

When OpenClaw loads skills, it creates a compact XML summary that gets injected into your agent's system prompt:

```
<available_skills>
  <skill>
    <name>weather</name>
    <description>Get current weather and forecasts via wttr.in</description>
    <location>~/.openclaw/workspace/skills/weather/SKILL.md</location>
  </skill>
  <skill>
    <name>deep-research-pro</name>
    <description>Multi-source deep research with citations</description>
    <location>~/.openclaw/workspace/skills/deep-research-pro/SKILL.md</location>
  </skill>
</available_skills>
```

This allows your agent to see what skills are available and when to use them.

Skill Selection Logic

OpenClaw follows these rules to decide which skills to load:

1. **Check requirements**: Skills with unmet requirements (missing API keys, tools, etc.) are filtered out
2. **Apply priorities**: Workspace skills override managed skills, which override bundled skills
3. **Respect configuration**: Skills disabled in config are excluded
4. **Platform filtering**: OS-specific skills only load on compatible systems

Configuration Options

You can customize skill behavior in `~/.openclaw/openclaw.json`:

```
{
  "skills": {
    "entries": {
      "weather": {
        "enabled": true,
        "apiKey": "YOUR_WEATHER_API_KEY_HERE"
      },
      "trading": {
        "enabled": true,
        "env": {
          "TRADING_MODE": "paper"
        }
      },
      "email-sender": {
        "enabled": false  // Disable this skill
      }
    }
```

```
  }
}
```

Managing Skills

Updating Skills

Keep skills current with regular updates:

```
# Update specific skill
clawhub update weather

# Update all skills
clawhub update --all
```

Checking Skill Status

View installed skills and their versions:

```
clawhub list
```

Removing Skills

To remove a skill, simply delete its directory:

```
# On Windows
rmdir /s skills\unwanted-skill

# On Mac/Linux
rm -rf skills/unwanted-skill
```

Checking Skill Requirements

Some skills require API keys or external tools. Check requirements by reading the skill's SKILL.md file or looking at error messages when the skill fails to load.

Environment Variable Management

Many skills need API keys or configuration. Here are the secure ways to provide them:

Method 1: Environment Variables

Set environment variables before starting OpenClaw:

```
# On Windows
set OPENAI_API_KEY=your-key-here
```

```
openclaw agent

# On Mac/Linux
export OPENAI_API_KEY=your-key-here
openclaw agent
```

Method 2: Configuration File

Add to `~/.openclaw/openclaw.json`:

```
{
  "skills": {
    "entries": {
      "image-generator": {
        "enabled": true,
        "apiKey": "your-openai-key-here"
      }
    }
  }
}
```

Method 3: Per-Project Environment

For project-specific configurations, create `.env` files (never commit these to version control):

```
# .env file in your project
OPENAI_API_KEY=your-key-here
BRAVE_API_KEY=your-search-key-here
```

Cross-Platform Considerations

Skills can specify platform requirements:

```
---
name: macos-shortcuts
description: Interact with macOS Shortcuts app
metadata: {"openclaw": {"os": ["darwin"]}}
---
```

This skill only loads on macOS (`darwin`). Similar tags exist for `win32` (Windows) and `linux`.

Troubleshooting Common Issues

Skill Won't Load

Problem: Installed a skill but it doesn't appear in your agent's capabilities.

Solutions:

1. **Check skill requirements:**

```
# Look for requirements in the skill metadata
cat skills/skill-name/SKILL.md | head -10
```

2. **Verify installation location:**

```
# Check if skill exists in workspace
ls -la skills/

# Or in managed location
ls -la ~/.openclaw/skills/
```

3. **Start fresh session:** Skills only load at session start. Exit and restart your OpenClaw session.
4. **Check for errors:** Look at OpenClaw logs for skill loading errors:

```
openclaw gateway logs
```

API Key Issues

Problem: Skill loads but fails with authentication errors.

Solutions:

1. **Verify API key format:**
   - OpenAI keys start with `sk-`
   - Brave Search keys are typically 32 characters
   - Check key format in the service's documentation
2. **Test API key separately:**

```
# Test OpenAI key
curl -H "Authorization: Bearer YOUR_KEY" https://api.openai.com/v1/models

# Test Brave Search key
curl "https://api.search.brave.com/res/v1/web/search?q=test&key=YOUR_KEY"
```

3. **Check environment variable names:** Common variations:
   - `OPENAI_API_KEY` or `OPENAI_KEY`
   - `BRAVE_API_KEY` or `BRAVE_SEARCH_KEY`
   - Check the skill's SKILL.md for exact variable names

Permission Errors

Problem: Skills fail with "permission denied" or "access denied" errors.

Solutions:

1. **Check file permissions:**

```
# On Mac/Linux - make scripts executable
chmod +x skills/skill-name/scripts/*.sh

# On Windows - check file isn't blocked
Get-Item "skills\skill-name\scripts\*" | Unblock-File
```

2. **Run as administrator (Windows only):** Some skills need elevated permissions for system integration.
3. **Check firewall/antivirus:** Security software sometimes blocks AI tools from accessing files or network.

Skill Conflicts

Problem: Multiple skills try to handle the same type of request.

Solutions:

1. **Check skill priorities:** Workspace skills > Managed skills > Bundled skills
2. **Disable conflicting skills:**

```
// In ~/.openclaw/openclaw.json
{
  "skills": {
    "entries": {
      "conflicting-skill": {
        "enabled": false
      }
    }
  }
}
```

3. **Use specific skill names:** Instead of “analyze this document”, try “use document-summary skill to analyze this”

Performance Issues

Problem: Agent becomes slow when many skills are loaded.

Solutions:

1. **Audit installed skills:**

```
clawhub list | wc -l  # Count installed skills
```

2. **Disable unused skills:** Remove or disable skills you don’t actively use.
3. **Check skill complexity:** Complex skills with many examples consume more tokens.
4. **Monitor token usage:** Skills add ~100 tokens each to the system prompt.

Network/Connectivity Issues

Problem: Skills that require internet access fail intermittently.

Solutions:

1. **Test basic connectivity:**

```
ping google.com
curl -I https://api.openai.com/
```

2. **Check proxy settings:**

```
# Check if proxy is configured
echo \$HTTP_PROXY
echo \$HTTPS_PROXY
```

3. **Verify DNS resolution:**

```
nslookup api.openai.com
nslookup clawhub.ai
```

4. **Try different DNS servers:** Switch to 8.8.8.8 (Google) or 1.1.1.1 (Cloudflare) if having resolution issues.

Installation Problems

Problem: `clawhub install` fails or hangs.

Solutions:

1. **Update ClawHub CLI:**

```
npm update -g clawhub
```

2. **Clear npm cache:**

```
npm cache clean --force
```

3. **Check available disk space:** Skills need storage space for installation.
4. **Try manual installation:** Download skill ZIP from ClawHub website and extract manually.

Skill Updates Not Working

Problem: `clawhub update` doesn't seem to update skills.

Solutions:

1. **Force update:**

```
clawhub update skill-name --force
```

2. **Check version:**

```
clawhub list  # Shows installed vs latest versions
```

3. **Manual update:** Delete skill folder and reinstall:

```
rm -rf skills/skill-name
clawhub install skill-name
```

Platform-Specific Issues

Windows

- **PowerShell execution policy**: Run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`
- **Long path names**: Enable long paths in Windows settings if skill paths are too long
- **Antivirus interference**: Add OpenClaw directory to antivirus exclusions

macOS

- **Gatekeeper warnings**: Skills downloaded from internet may trigger security warnings
- **File permissions**: Use `chmod +x` to make script files executable
- **SIP restrictions**: Some system integration skills may require SIP adjustments

Linux

- **Package dependencies**: Some skills may require additional system packages
- **File permissions**: Ensure skills directory is writable by user
- **SELinux/AppArmor**: Security policies may block some skill operations

Getting Help

If you're still having issues:

1. **Check skill documentation**: Many skills include troubleshooting sections
2. **Visit ClawHub**: Skill pages often have user comments with solutions
3. **OpenClaw Discord**: Community support for complex issues
4. **GitHub Issues**: Report bugs in skill repositories
5. **Skill logs**: Check `~/.openclaw/logs/` for detailed error information

Summary

The Skills System is what transforms OpenClaw from a capable AI assistant into a specialized powerhouse tailored to your exact needs. By the end of this chapter, you should have:

Γ£à **Understanding**: Know what skills are and how they extend your agent's capabilities Γ£à **ClawHub mastery**: Ability to search, install, and manage skills from the marketplace Γ£à **Essential skills installed**: Weather, Deep Research Pro, Code, Document Summary, and Trading skills Γ£à **Self-improvement setup**: Your agent now learns from mistakes and continuously improves Γ£à **Custom skills**: Knowledge of how to create your own specialized skills Γ£à **Troubleshooting**: Solutions for common issues you might encounter

The skills you install today will serve you for months or years to come, growing more valuable as they learn your preferences and patterns.

What's Next?

In the next chapter, we'll dive into **Advanced Automation** — where you'll learn to create sophisticated workflows, set up cron jobs, and build systems that run themselves. The skills you've learned about here will become the building blocks for those automated systems.

Remember: Every expert started as a beginner. Don't feel like you need to master everything at once. Pick one or two skills that solve your immediate problems, get comfortable with those, then gradually expand your toolkit.

Your AI assistant just became infinitely more capable. Now go build something amazing with it.

"The best agents aren't the ones with the most skills — they're the ones with the right skills for the job."

CHAPTER 07

First Automations

5 Starter Cron Jobs

Chapter 7: Your First Automations — 5 Starter Cron Jobs + HEARTBEAT.md

"The best kind of worker is the one who works while you sleep."

Welcome to the moment everything changes. Up until now, you've been manually triggering your AI agent, asking it questions, having conversations. That's useful, but it's not the real power of OpenClaw.

The real power comes when your agent starts working **automatically**. When it checks your email every morning, summarizes the news before you wake up, backs up your work without you remembering, and keeps an eye on your calendar while you focus on more important things.

This chapter will teach you to set up five essential automations that will make your OpenClaw agent earn its keep. By the end, you'll have a digital assistant that actually assists—proactively, automatically, and efficiently.

What Are Cron Jobs?

Think of cron jobs as scheduled tasks that run automatically. The name comes from Unix systems, where "cron" has been running scheduled tasks for decades. In OpenClaw, cron jobs let you schedule your AI agent to do things at specific times or intervals.

Here's the magic: **your agent works while you sleep**.

Instead of remembering to check the weather every morning, you set up a cron job that does it automatically at 7 AM. Instead of manually asking for a news summary, your agent fetches and summarizes the top headlines every weekday at 9 AM. Instead of worrying about backing up your work, a cron job handles it every night at midnight.

The Power of Automation

Let's say you want your agent to: - Check the weather every morning at 7 AM - Summarize news on weekdays at 9 AM
- Remind you of calendar events 30 minutes before they start - Back up your workspace files every night - Send you a weekly project update every Monday

Without cron jobs, you'd need to manually ask your agent to do each of these things, every single time. With cron jobs, you set them up once and they run automatically forever.

This is the difference between having a helpful chat bot and having a true digital assistant.

Cron Expression Syntax: When Things Happen

Cron expressions are how you tell OpenClaw exactly when to run your automated tasks. Don't worry—you don't need to memorize complex syntax. Most of what you'll use follows simple patterns.

The Basics

A cron expression has five parts: `minute hour day-of-month month day-of-week`

```
┌─────────────── minute (0-59)
│ ┌───────────── hour (0-23)
│ │ ┌─────────── day of month (1-31)
│ │ │ ┌───────── month (1-12)
│ │ │ │ ┌─────── day of week (0-7, Sunday = 0 or 7)
│ │ │ │ │
* * * * *
```

But here's the good news: OpenClaw supports 6-field expressions too, where the first field is seconds. For most automations, you'll stick to the 5-field format.

Common Patterns (Copy These!)

Here are the patterns you'll actually use:

Every day at a specific time: - `0 9 * * *` = every day at 9:00 AM - `30 7 * * *` = every day at 7:30 AM - `0 0 * * *` = every day at midnight

Weekdays only: - `0 8 * * 1-5` = weekdays at 8:00 AM - `30 17 * * 1-5` = weekdays at 5:30 PM

Weekends only: - `0 10 * * 0,6` = Saturday and Sunday at 10:00 AM - `0 9 * * 0` = every Sunday at 9:00 AM

Every few hours: - `0 */4 * * *` = every 4 hours (midnight, 4 AM, 8 AM, noon, 4 PM, 8 PM) - `0 */2 * * *` = every 2 hours - `0 8,12,18 * * *` = specific times: 8 AM, noon, and 6 PM

Weekly or monthly: - `0 9 * * 1` = every Monday at 9:00 AM
- `0 10 1 * *` = first day of every month at 10:00 AM

The `*` Wildcard

The asterisk (`*`) means "any value." So: - `0 9 * * *` = minute 0, hour 9, any day of month, any month, any day of week - Translation: 9:00 AM every single day

Load Balancing (OpenClaw's Smart Feature)

Here's something cool: OpenClaw automatically staggers recurring jobs that run at popular times (like the top of the hour) by up to 5 minutes. This prevents all automated tasks across the internet from hitting servers at exactly the same moment.

So if you set a job for `0 9 * * *`, it might actually run at 9:02 AM or 9:04 AM. This is intentional and helps with system stability.

If you need exact timing (rare), you can force it with the `--exact` flag when creating the job.

Three Schedule Types in OpenClaw

OpenClaw gives you three ways to schedule automated tasks. Each serves a different purpose:

1. `at` — One-Shot Reminders

Use this when you want something to happen exactly once at a specific time.

Examples: - "Remind me to call the client in 2 hours" - "Check project status tomorrow at 3 PM" - "Send follow-up email next Friday at 10 AM"

Command format:

```
openclaw cron add --name "Task name" --at "2024-12-15T15:00:00Z" --session main --system-event
"Reminder text"
```

Relative times work too:

```
openclaw cron add --name "Quick reminder" --at "20m" --session main --system-event "Time to check
email"
```

By default, one-shot jobs delete themselves after running successfully. This keeps your job list clean.

2. `every` — Fixed Intervals

Use this when you want something to repeat at regular intervals, regardless of the time.

Examples: - Every 30 minutes - Every 4 hours - Every 2 days

Command format:

```
openclaw cron add --name "Regular check" --every "30m" --session isolated --message "Check system
status"
```

3. `cron` — Precise Scheduling

Use this when you want something to happen at specific times and dates. This is what most of your automations will use.

Examples: - Every weekday at 9 AM - Every Sunday at 8 PM - First day of every month at noon

Command format:

```
openclaw cron add --name "Daily briefing" --cron "0 9 * * 1-5" --tz "America/New_York" --session
isolated --message "Generate daily briefing"
```

Always specify timezone with `--tz` for cron expressions, especially if you travel or collaborate across time zones.

Session Targets Explained

When you create a cron job, you need to decide where it runs. OpenClaw gives you two options:

`isolated` — Recommended for Most Jobs

Isolated sessions run in their own clean environment, separate from your main conversation history. Think of each run as a fresh start.

Advantages: - **Clean slate**: No conversation history to clutter the context - **Model override**: Use a cheaper model for routine tasks
- **Separate history**: Keeps your main chat clean - **Direct delivery**: Results can go straight to WhatsApp, Telegram, etc. - **Cost efficient**: Each run is focused and doesn't carry conversation baggage

Use isolated for: - Daily reports and summaries - Background monitoring tasks - Routine data processing - Tasks that don't need conversation context

Example:

```
openclaw cron add --name "Morning weather" --cron "0 7 * * *" --session isolated --message "Get
today's weather for Chicago"
```

`main` — For Context-Aware Tasks

Main session jobs inject into your regular conversation thread. They appear as system events that get handled during the next heartbeat.

Advantages: - **Full context**: Access to your entire conversation history - **Conversational flow**: Responses feel natural and connected - **Memory**: Agent remembers what you've been working on

Disadvantages: - **Context bloat**: Conversation history takes up token space - **Timing dependent**: Waits for next heartbeat to process - **Mixed history**: Job outputs appear in your regular chat

Use main session for: - Reminders that need conversation context - Follow-ups to ongoing projects
- Tasks that should feel conversational

Example:

```
openclaw cron add --name "Project reminder" --at "4h" --session main --system-event "Follow up on the website redesign project"
```

Which Should You Choose?

Default to `isolated` for most automations. It's cleaner, more efficient, and easier to manage. Use `main` only when you specifically need the job to feel like part of your ongoing conversation.

Delivery Modes: Where Results Go

When an isolated cron job finishes, where do the results go? OpenClaw gives you three options:

`announce` — Default Delivery

Results get delivered directly to your chosen channel (WhatsApp, Telegram, Discord, etc.) plus a brief summary appears in your main session.

When to use: Most automated tasks where you want to see the results.

`none` — Internal Only

The job runs but results stay internal. No messages are delivered anywhere.

When to use: - Background maintenance tasks - Data processing that doesn't need immediate attention - Tasks that update files but don't need notifications

`webhook` — HTTP Delivery

Results get sent to a web URL via HTTP POST. Advanced feature for integration with other systems.

When to use: Connecting OpenClaw to other services, APIs, or automation platforms.

The 5 Starter Cron Jobs

Let's set up five essential automations that will demonstrate the power of scheduled tasks. Each comes with a complete, copy-paste configuration plus customization tips.

Job 1: Daily Morning Summary

What it does: Every morning at 8 AM, your agent wakes up and gives you a brief summary of the day ahead—weather, any important reminders, and a gentle "good morning" to start your day right.

Schedule: 8:00 AM daily
Model: Local (free, efficient for daily tasks)
Delivery: Announces to your preferred channel

Copy-paste configuration:

```
openclaw cron add \
  --name "Morning Summary" \
  --cron "0 8 * * *" \
  --tz "America/Chicago" \
  --session isolated \
  --message "Good morning! Please provide a brief morning summary: check today's weather, any
calendar events in the next few hours, and wish me a productive day. Keep it upbeat and concise." \
  --model "qwen2.5:14b" \
  --announce \
  --channel whatsapp \
  --to "+1YOUR_PHONE_NUMBER"
```

Expected output example:

```
Good morning! ΓÿÇΩ╕Å

Today's weather: Sunny, high of 72╥▒F with light winds - perfect for that lunch meeting outdoors.

Calendar: You have the team standup at 10 AM and the client call at 2 PM.

Have a productive day! ≡ƒÜÇ
```

Customization tips: - Change the timezone in `--tz` to match your location - Replace `whatsapp` with `telegram`, `discord`, or your preferred channel - Update the phone number in `--to` - Adjust the time by changing `0 8 * * *` (e.g., `30 7 * * *` for 7:30 AM) - Add specific requests like “check my email” or “review my task list”

Job 2: Weather Forecast

What it does: Every morning at 7 AM, get a clean, simple weather report. No fluff, just the information you need to dress appropriately and plan your day.

Schedule: 7:00 AM daily **Model**: Local (weather is perfect for cheap models) **Delivery**: Direct to your phone

Copy-paste configuration:

```
openclaw cron add \
  --name "Daily Weather" \
  --cron "0 7 * * *" \
  --tz "America/Chicago" \
  --session isolated \
  --message "Get today's weather forecast for Chicago, IL. Include temperature (high/low),
conditions, and any weather warnings. Keep it brief and practical." \
  --model "qwen2.5:14b" \
  --announce \
  --channel whatsapp \
  --to "+1YOUR_PHONE_NUMBER"
```

Expected output example:

```
≡ƒîñΩ╕Å Chicago Weather - December 15
```

```
High: 45╗F, Low: 32╗F
Partly cloudy, light winds
No precipitation expected

Good jacket weather!
```

Customization tips: - Change “Chicago, IL” to your city - Add specific requests like “should I bring an umbrella?” or “good day for running?” - Use `--model "mistral:7b"` if you want an even faster model - Schedule for earlier (like 6 AM) if you wake up early - For travelers: create multiple weather jobs for different cities

Job 3: News Digest

What it does: Every weekday morning at 9 AM, get the top 5 headlines summarized in a clean, readable format. Stay informed without spending 30 minutes reading news sites.

Schedule: 9:00 AM on weekdays only **Model**: Local or cheap cloud (news summarization works great with smaller models) **Delivery**: Delivered to your preferred channel

Copy-paste configuration:

```
openclaw cron add \
  --name "News Digest" \
  --cron "0 9 * * 1-5" \
  --tz "America/Chicago" \
  --session isolated \
  --message "Search for today's top news headlines and summarize the 5 most important stories.
Focus on business, technology, and major world events. Keep each summary to 1-2 sentences. Format
with bullet points." \
  --model "qwen2.5:14b" \
  --announce \
  --channel whatsapp \
  --to "+1YOUR_PHONE_NUMBER"
```

Expected output example:

```
≡ƒôÿ Top Headlines - December 15

ΓÇó Tech: Major AI breakthrough announced by Google, claiming 40% improvement in reasoning
capabilities
ΓÇó Business: Federal Reserve holds rates steady, markets rise 2% on stability news
ΓÇó World: Climate summit reaches agreement on renewable energy targets for 2025
ΓÇó Politics: New infrastructure bill passes committee, heading to full vote next week
ΓÇó Science: NASA's Mars rover discovers unusual mineral formations suggesting past water activity

Stay informed! ≡ƒôè
```

Customization tips: - Adjust `1-5` to `1-7` to include weekends - Add specific topics: “Focus on AI, startups, and crypto news” - Change the number of stories: “summarize the 3 most important” or “top 10” - Use `--model "deepseek"` for slightly higher quality summaries - Schedule for different times: evening digest at 6 PM, etc.

Job 4: Daily Reminder / Calendar Check

What it does: Every morning at 8:30 AM, review your calendar and send you reminders about upcoming meetings, deadlines, or appointments. Never be caught off guard by a forgotten meeting again.

Schedule: 8:30 AM daily **Model**: Local (calendar checking is straightforward) **Delivery**: Announces important reminders

Copy-paste configuration:

```
openclaw cron add \
  --name "Calendar Check" \
  --cron "30 8 * * *" \
  --tz "America/Chicago" \
  --session isolated \
  --message "Check my calendar for today and tomorrow. Alert me to any meetings in the next 3 hours
and any important events coming up. If nothing urgent, just give a brief 'calendar clear'
confirmation." \
  --model "qwen2.5:14b" \
  --announce \
  --channel whatsapp \
  --to "+1YOUR_PHONE_NUMBER"
```

Expected output example:

```
≡ƒôà Calendar Update

Today:
ΓÇó Team standup at 10:00 AM (in 1.5 hours)
ΓÇó Client presentation at 2:00 PM
ΓÇó Dentist appointment at 4:30 PM

Tomorrow:
ΓÇó All-hands meeting at 9:00 AM
ΓÇó Lunch with Sarah at 12:30 PM

Get ready for that standup! ≡ƒæÑ
```

Alternative for "calendar clear" days:

```
≡ƒôà Calendar Update

Your day looks clear with no urgent meetings coming up.

Enjoy the focused work time! ΓÉ¿
```

Customization tips: - Adjust the time window: "next 2 hours" or "next 6 hours"
- Add specific reminders: "remind me to bring presentation materials" - Connect to specific calendar services (Google Calendar, Outlook, etc.) - Create separate jobs for work vs. personal calendars - Use earlier time (7:30 AM) if you want more advance notice

Job 5: Workspace Backup

What it does: Every night at midnight, back up your important workspace files. This could be a simple file copy to a backup directory, a git commit of your work, or syncing to cloud storage. Sleep peacefully knowing your work is safe.

Schedule: Midnight daily **Model**: Local (file operations don't need smart models) **Delivery**: None (silent backup, only notify on errors)

Copy-paste configuration:

```
openclaw cron add \
  --name "Workspace Backup" \
  --cron "0 0 * * *" \
  --tz "America/Chicago" \
  --session isolated \
  --message "Perform a backup of the workspace directory. If git is available, commit any changes
with a timestamped message. If git is not set up, copy important files to a backup directory. Only
report if there are errors or if significant changes were backed up." \
  --model "qwen2.5:14b" \
  --no-deliver
```

Expected output (only on errors or significant changes):

```
≡ƒöÆ Backup Complete

Committed 12 files to git:
- Updated project documentation
- New automation scripts
- Modified configuration files

Commit: "Auto-backup: 2024-12-15 00:00:17"

Your work is safe! Γ£à
```

Customization tips: - Change `--no-deliver` to `--announce` if you want confirmation every night - Adjust time to run when your computer is always on - Add specific backup targets: "backup the Documents folder" - Include cloud sync: "commit to git and push to GitHub" - Create separate backups for different projects - Use earlier times (like 11 PM) if you work late

Managing Your Cron Jobs

Once you've created these jobs, you'll need to know how to manage them. Here are the essential commands:

Listing Jobs

See all your scheduled jobs:

```
openclaw cron list
```

This shows you: - Job names and IDs - Schedules and next run times
- Whether jobs are enabled or disabled - Last run status

Running Jobs Manually

Test a job immediately without waiting for its scheduled time:

```
openclaw cron run <job-id>
```

This is perfect for: - Testing new jobs to see if they work - Running a job ahead of schedule - Debugging issues with job configuration

Checking Run History

See the history of past runs for any job:

```
openclaw cron runs --id <job-id> --limit 10
```

This shows: - When the job ran - Whether it succeeded or failed - How long it took - Any error messages

Editing Jobs

Modify existing jobs without recreating them:

```
# Change the message
openclaw cron edit <job-id> --message "New task description"

# Change delivery settings
openclaw cron edit <job-id> --announce --channel telegram --to "123456789"

# Disable delivery
openclaw cron edit <job-id> --no-deliver

# Change the model
openclaw cron edit <job-id> --model "opus"
```

Pausing and Disabling Jobs

Temporarily stop a job without deleting it:

```
# Disable a job
openclaw cron edit <job-id> --disabled

# Re-enable it later
openclaw cron edit <job-id> --enabled
```

Removing Jobs

Permanently delete jobs you no longer need:

```
openclaw cron remove <job-id>
```

Be careful: This permanently deletes the job and its run history.

Finding Job IDs

When you create a job, OpenClaw returns its ID. If you forget it, use:

```
openclaw cron list
```

The output shows both job names and their IDs.

The HEARTBEAT.md System

While cron jobs handle scheduled tasks, OpenClaw has another automation system called **heartbeats**. Think of heartbeats as your agent's periodic check-ins—like a pulse that keeps your agent aware and responsive.

What Are Heartbeats?

Heartbeats are automatic, periodic prompts sent to your agent in the main conversation. By default, every 30 minutes, your agent gets a gentle prompt asking "anything need attention?"

The default heartbeat prompt is: > "Read HEARTBEAT.md if it exists (workspace context). Follow it strictly. Do not infer or repeat old tasks from prior chats. If nothing needs attention, reply HEARTBEAT_OK."

If nothing needs attention, the agent responds with `HEARTBEAT_OK` and no message is delivered to you. If something does need attention, you get an alert.

Heartbeats vs. Cron Jobs: When to Use Each

Here's the key difference:

Use heartbeats when: - You want to batch multiple small checks together (email + calendar + notifications) - Timing can drift slightly (every ~30 minutes is fine) - The agent needs conversational context from recent messages - You want to reduce API calls by combining checks

Use cron jobs when: - Exact timing matters ("9:00 AM sharp every Monday") - The task needs isolation from main conversation - You want different model settings for the task - It's a one-shot reminder ("in 20 minutes")

Creating Your HEARTBEAT.md File

The magic happens when you create a `HEARTBEAT.md` file in your workspace. This becomes your agent's periodic checklist.

Example HEARTBEAT.md:

```
# Heartbeat Checklist

## Check every 2-3 hours
- Scan email inbox for urgent messages
- Review calendar for events in next 2 hours
- Check for any completed background tasks

## Check once per day (rotate)
- Weather outlook for tomorrow
- Any important news or updates
- Project status and next actions

## Light check-in
- If idle for 8+ hours during daytime, send friendly check-in
- Keep it brief and natural
```

Configuring Heartbeat Timing

You can adjust how often heartbeats occur by modifying your OpenClaw configuration:

```
{
  agents: {
    defaults: {
      heartbeat: {
        every: "30m",        // How often (30 minutes is default)
        target: "last",      // Where to send alerts ("last" = your most recent channel)
        activeHours: {       // Optional: only during certain hours
          start: "08:00",
          end: "22:00",
          timezone: "America/Chicago"
        }
      }
    }
  }
}
```

Heartbeat Best Practices

Keep HEARTBEAT.md small: Remember, this gets read every 30 minutes. A massive checklist wastes tokens and slows down response times.

Batch similar checks: Instead of separate cron jobs for email, calendar, and notifications, let heartbeats handle all three in one go.

Use active hours: Don't let your agent wake you up at 3 AM unless it's truly urgent.

Track your checks: The agent can maintain a simple state file to rotate through different checks:

```
{
  "lastChecks": {
    "email": 1703275200,
    "calendar": 1703260800,
    "weather": null
  }
}
```

Sample Heartbeat Workflow

Here's how a well-configured heartbeat might work:

8:30 AM: Agent checks email, finds 3 new messages, determines one is urgent from your boss about today's presentation. Sends you: "Urgent: Boss needs the Q4 slides by 10 AM."

11:00 AM: Agent checks calendar, sees lunch meeting in 1 hour, checks weather, finds it's raining. Sends you: "Lunch meeting with client in 1h. Pack an umbrella—rain expected."

2:30 PM: Agent scans email and calendar, nothing urgent. Responds `HEARTBEAT_OK` internally, no message sent.

6:00 PM: Agent notices you haven't had any activity for 4 hours, sends a brief: "Hope your day went well! Anything you need help with this evening?"

Combining Heartbeats with Cron Jobs

The most powerful setup uses both:

1. **Heartbeats** handle general monitoring and awareness (every 30 minutes)
2. **Cron jobs** handle specific scheduled tasks (daily reports, weekly reviews)

This gives you: - Efficient batched monitoring via heartbeats - Precise scheduling via cron jobs
- Cost optimization (one heartbeat instead of multiple crons) - Context awareness (heartbeats) plus task isolation (crons)

Cost Optimization for Crons

Automation is powerful, but it can get expensive if you're not careful. Here's how to keep costs under control while maximizing value:

Always Use Local Models for Routine Tasks

The golden rule: For automated tasks, always prefer local models unless you specifically need the power of cloud models.

Local models like `qwen2.5:14b`, `mistral:7b`, and `llama3.2:3b` are: - **Free** (no API costs) - **Fast** (running on your hardware)
- **Private** (never leave your machine) - **Good enough** for most automation tasks

Examples of tasks perfect for local models: - Weather checks - Calendar scanning
- Simple news summaries - File backups - Basic reminders - Email monitoring

When you might need cloud models: - Complex analysis requiring deep reasoning - Tasks involving multiple data sources - Creative writing or content generation - Advanced problem-solving

Model Tier Strategy

Follow this decision tree for automated tasks:

1. **Start with local** (`qwen2.5:14b` or `mistral:7b`)
2. **Test thoroughly** to ensure quality meets your needs
3. **Only escalate** if the local model consistently fails
4. **Use cheap cloud models** (`deepseek` , `flash`) before expensive ones
5. **Reserve premium models** (`opus` , `gpt-4`) for revenue-critical automation

Batch Similar Tasks

Instead of creating 5 separate cron jobs that each cost tokens: - Email check every 30 minutes - Calendar check every 30 minutes
- Weather check every 30 minutes - News check every 30 minutes - Notification check every 30 minutes

Create one heartbeat that handles all five checks in a single API call.

Optimize Prompt Length

Shorter prompts cost fewer tokens. Compare:

Expensive prompt (unnecessary detail):

```
"Please check my email inbox and look through all the messages from today and yesterday. If you
find anything that seems urgent or important, like messages from my boss, clients, or family
members, please summarize them for me. Also check if there are any calendar events coming up in the
next few hours that I should know about. Make sure to format everything nicely and include helpful
context about why each item might be important."
```

Efficient prompt (same result, fewer tokens):

```
"Check email for urgent messages and calendar for events in next 3 hours. Report only important
items."
```

Smart Delivery Settings

Use `--no-deliver` for jobs that only need to act on exceptions:

```
# This backup job only notifies on errors, saving daily notification costs
openclaw cron add \
  --name "Backup" \
  --cron "0 0 * * *" \
  --message "Back up workspace. Report only if errors occur." \
  --model "qwen2.5:14b" \
  --no-deliver
```

Monitor Your Usage

Keep track of which automations are costing you money:

```
# Check run frequency
openclaw cron list

# Review recent runs
openclaw cron runs --id <job-id> --limit 20

# Look for patterns in failures (wasted API calls)
```

Jobs that fail repeatedly waste money on retries. Fix or disable problematic automations.

Troubleshooting Your Automations

Even well-configured automations can have issues. Here's how to diagnose and fix common problems:

"My Job Isn't Running"

Check if cron is enabled:

```
openclaw cron list
```

If you get an error, cron might be disabled in your configuration.

Verify the job schedule:

```
openclaw cron list
```

Look at the "Next run" column. Is it what you expected?

Check timezone issues: Common problem: You set a job for 9 AM but forgot to specify timezone, so it runs at 9 AM UTC instead of your local time.

Fix:

```
openclaw cron edit <job-id> --tz "America/Chicago"
```

"My Job Runs But I Don't Get Messages"

Check delivery settings:

```
openclaw cron runs --id <job-id> --limit 5
```

Look for delivery failures in the run history.

Verify your target:

```
# Make sure your phone number or channel ID is correct
openclaw cron edit <job-id> --announce --channel whatsapp --to "+1CORRECT_NUMBER"
```

Test manually:

```
# Run the job immediately to see what happens
openclaw cron run <job-id>
```

"My Jobs Keep Failing"

Check the error messages:

```
openclaw cron runs --id <job-id> --limit 10
```

Common failures: - **Model not available**: Switch to a different model - **Network issues**: Temporary—jobs will retry automatically
- **Permission errors**: Check file permissions for backup jobs - **Invalid prompts**: Prompt might be malformed or too long

Try a simpler version: If a complex job keeps failing, simplify it:

```
# Instead of this complex prompt...
openclaw cron edit <job-id> --message "Do X, Y, and Z, considering A, B, C factors..."

# Try this simpler one
openclaw cron edit <job-id> --message "Do X only"
```

"My Heartbeats Are Too Noisy"

If your agent is sending too many heartbeat messages:

Check your HEARTBEAT.md: Is it too broad? Tighten the criteria:

```
# Instead of
- Check email and tell me about anything interesting

# Try
- Check email and report ONLY urgent messages from boss or clients
```

Use active hours:

```
{
  agents: {
    defaults: {
      heartbeat: {
        activeHours: { start: "08:00", end: "22:00" }
      }
    }
```

```
  }
}
```

Increase the interval:

```
{
  agents: {
    defaults: {
      heartbeat: {
        every: "60m"  // Check every hour instead of every 30 minutes
      }
    }
  }
}
```

"Jobs Are Running at Wrong Times"

Timezone confusion: Always specify timezone explicitly:

```
openclaw cron edit <job-id> --tz "America/New_York"
```

Stagger vs. exact timing: OpenClaw automatically adds small random delays (0-5 minutes) to popular times to prevent server overload. If you need exact timing:

```
openclaw cron edit <job-id> --exact
```

Daylight Saving Time: Use IANA timezone names (`America/Chicago`) instead of abbreviations (`CST`) to handle DST automatically.

Getting Help

If you're stuck:

1. **Check the logs**: `openclaw cron runs --id <job-id>`
2. **Test manually**: `openclaw cron run <job-id>`
3. **Start simple**: Create a minimal test job to isolate the problem
4. **Check the documentation**: These troubleshooting steps solve 90% of issues

Your Automation Journey Starts Now

Congratulations! You now have the knowledge to set up powerful automations that will transform how your OpenClaw agent serves you.

Here's your action plan:

Week 1: Set Up the Basics

1. **Choose your timezone** and configure it in all cron jobs
2. **Set up Job 1 (Morning Summary)** - start your days informed
3. **Set up Job 2 (Weather Forecast)** - dress appropriately every day
4. **Test both jobs manually** to ensure they work

Week 2: Add Intelligence

1. **Set up Job 3 (News Digest)** - stay informed effortlessly
2. **Configure your HEARTBEAT.md** - let your agent watch for important things
3. **Adjust heartbeat timing** to match your schedule

Week 3: Add Productivity

1. **Set up Job 4 (Calendar Check)** - never miss another meeting
2. **Set up Job 5 (Workspace Backup)** - protect your work automatically
3. **Fine-tune delivery settings** to reduce noise

Week 4: Optimize and Expand

1. **Review cost and performance** of all your automations
2. **Create custom jobs** for your specific needs
3. **Share your setup** with others who might benefit

The Bigger Picture

These five starter jobs are just the beginning. Once you see the power of automation, you'll start noticing opportunities everywhere:

- **Business automations**: Daily sales reports, customer service monitoring, inventory alerts
- **Personal automations**: Bill payment reminders, fitness tracking, meal planning
- **Creative automations**: Daily writing prompts, inspiration gathering, idea development
- **Learning automations**: Language practice reminders, skill-building check-ins, progress tracking

The goal isn't to automate everything—it's to automate the routine stuff so you can focus on what matters most.

Your AI Agent, Evolved

Remember when you first installed OpenClaw? You had a chatbot that could answer questions when you asked them.

Now you have something much more powerful: **a digital assistant that works proactively on your behalf**.

Your agent now: - Γ£à Wakes up before you do to check the weather - Γ£à Monitors your email and calendar while you focus on important work
- Γ£à Keeps you informed about the world without you having to seek out news - Γ£à Backs up your work while you sleep - Γ£à Reminds you of important events before you forget them

This is the promise of AI automation fulfilled. Your agent isn't just responding to you—it's actively making your life better, every single day.

The future you've been reading about in AI articles? You're living it now.

In the next chapter, we'll explore how to connect your automated agent to external services—email, calendars, project management tools, and more. The real magic happens when your automations can read from and write to the systems you use every day.

CHAPTER 08

Calendar & Productivity

Your AI-Powered Daily System

Chapter 8: Calendar & Productivity — Your AI-Powered Daily System

Introduction: Your Personal Chief of Staff

Imagine having a personal assistant that never sleeps, never forgets, and constantly works behind the scenes to keep your life organized. That's what this chapter will help you build with OpenClaw.

By the end of this chapter, your AI agent will:

- Send you morning briefings with weather, calendar, and priority tasks
- Remind you of important deadlines and meetings
- Track your daily accomplishments and goals
- Run weekly reviews to identify patterns and improvements
- Manage your task list using simple memory files
- Never let you forget anything important again

This isn't about complex calendar API integrations (that's covered in the Business and Enterprise editions). This is about building a rock-solid productivity system using OpenClaw's built-in automation features that works for complete beginners.

The Productivity Vision: Your AI Chief of Staff

Think of your OpenClaw agent as a highly competent personal assistant. Here's what your typical day will look like once we're done:

7:00 AM — Your agent sends a morning briefing: - Current weather and today's forecast - Your calendar for the day - Priority tasks from yesterday that need attention - Brief news summary or important updates

Throughout the day — Your agent quietly works in the background: - Monitoring for urgent emails or messages - Tracking tasks you mention in conversations - Preparing summaries of your work - Setting up reminders you request

9:00 PM — Your agent sends an evening review: - What you accomplished today - Tomorrow's priorities and schedule - Any loose ends that need attention

Weekly — Your agent analyzes patterns: - How well you're tracking toward goals - Which days are most/least productive - Suggestions for improving your system

Let's build this system step by step.

Chapter Overview

We'll cover eight key areas:

1. **Setting up reminders via cron** — One-shot and recurring reminders
2. **Daily briefing system** — Morning and evening automation
3. **Task management workflow** — Using memory files as your task list
4. **Calendar integration approaches** — Simple solutions for calendar awareness
5. **Weekly review automation** — Automated progress tracking
6. **Productivity templates** — Ready-to-use planners and trackers
7. **Building your personal system** — Customization for your workflow
8. **Troubleshooting** — Common issues and solutions

1. Setting Up Reminders via Cron

OpenClaw's cron system is your agent's built-in scheduler. It can handle both one-time reminders ("remind me in 20 minutes") and recurring tasks ("daily standup every morning at 9 AM").

Understanding Cron vs Heartbeat

Before we dive in, let's understand when to use each:

- **Cron** is for precise timing: "exactly at 9:00 AM every Monday"
- **Heartbeat** is for regular check-ins: "every 30 minutes, see if anything needs attention"

Use cron when timing matters. Use heartbeat when you want your agent to batch multiple checks together.

One-Shot Reminders

One-shot reminders are perfect for "remind me in X minutes/hours" requests.

Basic syntax:

```
openclaw cron add \
  --name "Reminder name" \
  --at "2026-03-15T14:30:00Z" \
  --session main \
  --system-event "Your reminder message" \
```

```
  --wake now \
  --delete-after-run
```

Practical examples:

Remind yourself to call someone in 2 hours:

```
openclaw cron add \
  --name "Call client" \
  --at "2h" \
  --session main \
  --system-event "Reminder: Call the client about the project update" \
  --wake now \
  --delete-after-run
```

Remind yourself about a meeting tomorrow:

```
openclaw cron add \
  --name "Team meeting reminder" \
  --at "2026-03-15T08:45:00Z" \
  --session main \
  --system-event "Team meeting starts in 15 minutes - prepare status update" \
  --wake now \
  --delete-after-run
```

Key parameters explained: - `--name`: A short description for your reference - `--at`: When to trigger (use "2h", "30m" for relative time, or full ISO timestamps) - `--session main`: Run in your main conversation (keeps context) - `--system-event`: The message your agent will deliver - `--wake now`: Wake your agent immediately when the time comes - `--delete-after-run`: Remove the job after it runs (saves clutter)

Recurring Reminders

Recurring reminders are perfect for daily, weekly, or monthly check-ins.

Daily standup reminder (weekdays only):

```
openclaw cron add \
  --name "Daily standup" \
  --cron "0 9 * * 1-5" \
  --tz "America/New_York" \
  --session main \
  --system-event "Daily standup starts in 5 minutes. Prepare your update: what did you do
yesterday, what are you doing today, any blockers?" \
  --wake now
```

Weekly review reminder:

```
openclaw cron add \
  --name "Weekly review" \
  --cron "0 17 * * 5" \
  --tz "America/New_York" \
```

```
  --session main \
  --system-event "It's Friday 5 PM - time for your weekly review. What went well this week? What could be improved?" \
  --wake now
```

Monthly goal check:

```
openclaw cron add \
  --name "Monthly goals" \
  --cron "0 9 1 * *" \
  --tz "America/New_York" \
  --session main \
  --system-event "First of the month - time to review your monthly goals and set priorities for the coming month." \
  --wake now
```

Understanding cron expressions:

Cron uses five fields: `minute hour day month day-of-week`

- `0 9 * * *` = Every day at 9:00 AM
- `0 9 * * 1-5` = Weekdays at 9:00 AM (Monday=1, Friday=5)
- `0 17 * * 5` = Every Friday at 5:00 PM
- `0 9 1 * *` = First day of every month at 9:00 AM
- `30 8,12,17 * * 1-5` = 8:30 AM, 12:30 PM, 5:30 PM on weekdays

Natural Language to Cron Jobs

Your agent can help create cron jobs from natural language. Just tell it what you want:

"Set up a reminder to check my email every 2 hours during work days" "Remind me to take a break every hour from 9 AM to 6 PM" "Send me a weekly summary every Sunday evening"

Your agent will translate these requests into proper cron jobs for you.

Managing Your Reminders

List all active cron jobs:

```
openclaw cron list
```

View details of a specific job:

```
openclaw cron runs --id JOB_ID --limit 10
```

Remove a reminder:

```
openclaw cron remove JOB_ID
```

Edit an existing reminder:

```
openclaw cron edit JOB_ID --message "Updated reminder message"
```

2. Daily Briefing System

Your daily briefing system provides structure to your day with automatic morning briefings and evening reviews.

Morning Briefing Setup

Create a morning briefing that delivers everything you need to start your day right.

Basic morning briefing (7 AM daily):

```
openclaw cron add \
  --name "Morning briefing" \
  --cron "0 7 * * *" \
  --tz "America/New_York" \
  --session isolated \
  --message "Create my morning briefing: Check weather for today, review my calendar for the next
24 hours, scan memory files for any pending tasks or important notes from yesterday, and provide a
brief motivational message to start the day." \
  --announce \
  --channel telegram \
  --to "YOUR_CHAT_ID_HERE"
```

Advanced morning briefing with news:

```
openclaw cron add \
  --name "Complete morning briefing" \
  --cron "0 7 * * 1-5" \
  --tz "America/New_York" \
  --session isolated \
  --message "Generate my complete morning briefing: 1) Weather forecast for today 2) Calendar
events for today and tomorrow 3) Review memory/\$(date +%Y-%m-%d).md for pending tasks 4) Quick
news summary (tech/business headlines) 5) Priority focus areas for today. Format as a clean, easy-
to-scan summary." \
  --model "gpt" \
  --announce
```

Key components of a good morning briefing:

1. **Weather** — Helps you plan your day and choose appropriate clothing
2. **Calendar** — Upcoming meetings and appointments
3. **Tasks** — Carry-overs from yesterday plus new priorities
4. **News** — Brief industry updates (if relevant to your work)
5. **Focus areas** — 1-3 key things to accomplish today

Evening Review Setup

The evening review helps you close out your day and prepare for tomorrow.

Simple evening review (8 PM daily):

```
openclaw cron add \
  --name "Evening review" \
  --cron "0 20 * * *" \
  --tz "America/New_York" \
  --session isolated \
  --message "Create my evening review: 1) Review today's memory file (memory/\$(date +%Y-%m-%d).md) and summarize what I accomplished 2) Look at tomorrow's calendar 3) Note any tasks that didn't get done and should carry over 4) Provide a brief reflection on the day and tomorrow's priorities." \
  --announce
```

Weekend evening review (different focus):

```
openclaw cron add \
  --name "Weekend evening review" \
  --cron "0 20 * * 6,0" \
  --tz "America/New_York" \
  --session isolated \
  --message "Weekend evening review: 1) What did I accomplish personally today? 2) What are my plans for tomorrow? 3) Any personal projects or goals to focus on? 4) Prep notes for Monday if it's Sunday." \
  --announce
```

Creating a HEARTBEAT.md Template

Your `HEARTBEAT.md` file guides your agent's regular check-ins every 30 minutes. Keep it focused and actionable.

Create the file:

Windows:

```
notepad %USERPROFILE%\.openclaw\workspace\HEARTBEAT.md
```

Mac/Linux:

```
nano ~/.openclaw/workspace/HEARTBEAT.md
```

Sample HEARTBEAT.md content:

```
# Heartbeat Checklist

## High Priority (check every time)
- Scan for urgent messages or notifications that need immediate attention
- Review calendar for meetings in the next 2 hours
- Check if any high-priority tasks are overdue

## Medium Priority (rotate through these)
- Quick email scan for important messages
- Weather check if I might be going out
- Review today's memory file for incomplete tasks
```

```
## Low Priority (occasional)
- If I've been quiet for 4+ hours during work time, send a brief check-in
- Note any patterns or insights from today's activities

## Never Do
- Don't interrupt focused work time (9 AM - 12 PM, 2 PM - 5 PM)
- Don't send alerts for routine/non-urgent items
- Don't repeat the same alert within 2 hours
```

This gives your agent clear guidance on what to check and when to bother you.

Configuring Heartbeat Timing

Basic heartbeat configuration (add to your OpenClaw config):

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "every": "30m",
        "target": "last",
        "activeHours": {
          "start": "08:00",
          "end": "22:00",
          "timezone": "America/New_York"
        }
      }
    }
  }
}
```

Configuration explained: - `"every": "30m"` — Check every 30 minutes - `"target": "last"` — Send alerts to your last used chat - `"activeHours"` — Only run during these hours (prevents 3 AM notifications)

Morning Briefing Template

Here's a complete template your agent can use for consistent morning briefings:

```
# Morning Briefing Template

## Weather
- Current conditions and temperature
- Today's forecast (high/low, precipitation)
- Any weather advisories

## Calendar Today
- Morning meetings/appointments
- Afternoon schedule
- Evening commitments
- Tomorrow's first meeting (for preparation)

## Priority Tasks
- Incomplete tasks from yesterday
```

```
- New urgent items
- Top 3 focus areas for today

## Quick Notes
- Any important reminders
- Deadlines approaching this week
- Brief motivational message

---
*Generated at [timestamp] | Have a productive day!*
```

Save this template in your workspace so your agent can reference it when creating briefings.

3. Task Management Workflow

Your OpenClaw agent can function as a sophisticated task manager using simple memory files. This system is more flexible than traditional task apps because it integrates naturally with your conversations.

Memory-Based Task System

OpenClaw uses Markdown files in your workspace for memory. The system has two layers:

- `memory/YYYY-MM-DD.md` — Daily logs (today and yesterday are auto-loaded)
- `MEMORY.md` — Long-term curated memory (only loaded in private sessions)

Task Capture Workflow

When you mention tasks in conversation, your agent should automatically capture them in your daily memory file.

Example conversation:

```
You: "I need to finish the quarterly report by Friday, call the vendor about the delayed shipment,
and review Sarah's proposal."

Agent: "I've captured three tasks for you:
1. Finish quarterly report (due Friday)
2. Call vendor about delayed shipment
3. Review Sarah's proposal

I've added these to today's memory file. Would you like me to set reminders for any of these?"
```

Behind the scenes, your agent writes to `memory/2026-03-12.md`:

```
# 2026-03-12 Daily Log

## Tasks Captured
- [ ] Finish quarterly report (due Friday) - HIGH PRIORITY
- [ ] Call vendor about delayed shipment - follow up on order #12345
- [ ] Review Sarah's proposal - check budget section
```

```
## Notes
- Quarterly report is for Q1 2026, focus on sales numbers
- Vendor contact: Mike Johnson, mjohnson\@vendorcorp.com
- Sarah's proposal is for the new marketing campaign
```

Task Review During Heartbeats

Your `HEARTBEAT.md` should include task review:

```
# Heartbeat Checklist

## Task Management (every heartbeat)
- Review today's memory file for incomplete tasks
- Check if any tasks are overdue
- Note completed tasks and mark them done
- Alert if high-priority tasks haven't been touched in 4+ hours

## Weekly Task Review (Fridays)
- Review the week's memory files
- Identify recurring task patterns
- Note tasks that consistently get delayed
- Suggest process improvements
```

Task Completion Tracking

When you complete tasks, tell your agent or it can infer completion from your activities:

Manual completion:

```
You: "I finished the quarterly report and called the vendor."

Agent: "Great! I've updated today's tasks:
Γ£à Finish quarterly report (completed 2:30 PM)
Γ£à Call vendor about delayed shipment (completed 2:35 PM)
ΓÅ│ Review Sarah's proposal (still pending)

Just Sarah's proposal left for today. Should I set a reminder?"
```

Automatic inference (when your agent sees related work):

```
# 2026-03-12 Daily Log - Updated

## Tasks
- [x] Finish quarterly report (due Friday) - COMPLETED 2:30 PM
- [x] Call vendor about delayed shipment - COMPLETED 2:35 PM, shipment arriving Monday
- [ ] Review Sarah's proposal - IN PROGRESS, started 3:00 PM

## Completion Notes
- Quarterly report submitted to finance team
- Vendor confirmed delayed shipment will arrive Monday, no penalty
- Sarah's proposal review in progress, budget numbers look good
```

Daily Task Templates

Create templates for different types of work days:

Regular workday template:

```
# YYYY-MM-DD Daily Log

## Morning Priorities
- [ ] Review email and respond to urgent items
- [ ] Check calendar for meeting prep needs
- [ ] Identify top 3 focus areas for today

## Main Tasks
- [ ] [Project work block 9-12 PM]
- [ ] [Meeting block 1-3 PM]
- [ ] [Admin/communication block 3-5 PM]

## End of Day
- [ ] Update project status
- [ ] Prep for tomorrow's meetings
- [ ] Clear inbox below 10 items

## Notes
[Your agent fills this in throughout the day]
```

Meeting-heavy day template:

```
# YYYY-MM-DD Daily Log (Meeting Day)

## Meeting Prep
- [ ] Review agenda for [9 AM meeting]
- [ ] Prepare talking points for [10:30 AM presentation]
- [ ] Gather materials for [2 PM client call]

## Between Meetings
- [ ] Follow up on action items from morning meetings
- [ ] Quick email check and responses
- [ ] Update project docs with meeting outcomes

## End of Day Review
- [ ] Document key decisions from meetings
- [ ] Set follow-up reminders
- [ ] Update stakeholders on outcomes

## Meeting Notes
[Your agent captures key points here]
```

Weekly Task Analysis

Set up a weekly cron job to analyze your task patterns:

```
openclaw cron add \
  --name "Weekly task analysis" \
  --cron "0 18 * * 5" \
  --tz "America/New_York" \
```

```
  --session isolated \
  --message "Analyze this week's task management: 1) Review memory files from Monday-Friday 2) Identify tasks that were consistently delayed 3) Note which types of tasks I complete quickly vs slowly 4) Suggest improvements to my task capture and completion process 5) Highlight any recurring task patterns" \
  --announce
```

This helps you identify patterns like: - Which tasks you tend to procrastinate on - What time of day you're most productive - Which types of tasks take longer than expected - Opportunities to batch similar tasks

4. Calendar Integration Approaches

While the Business and Enterprise editions of this guide cover full API integration with Google Calendar, Outlook, and other services, the Personal edition focuses on practical approaches that work without complex API setup.

Approach 1: Manual Calendar Events in Memory Files

The simplest approach is to manually maintain your calendar in memory files. Your agent can remind you to update it and help format it consistently.

Daily calendar in memory file:

```
# 2026-03-12 Daily Log

## Today's Calendar
- 9:00 AM - Team standup (Conference Room A)
- 10:30 AM - Client call with ABC Corp (dial +1-555-123-4567)
- 12:00 PM - Lunch with Sarah (Downtown Café)
- 2:00 PM - Project review meeting (Building B)
- 4:30 PM - 1:1 with manager (their office)

## Tomorrow Preview (2026-03-13)
- 9:30 AM - Marketing meeting
- 11:00 AM - Focus time (blocked for report writing)
- 1:00 PM - Vendor presentation
- 3:00 PM - Team retrospective
```

Weekly calendar template:

```
# Week of 2026-03-10

## Monday 3/10
- 9:00 AM - Team standup
- 10:00 AM - Project kickoff meeting
- 2:00 PM - Client presentation

## Tuesday 3/11
- 9:00 AM - Team standup
- 11:00 AM - Budget review
- 3:00 PM - Training session

## Wednesday 3/12
```

```
[Continue for full week...]

## Recurring Weekly Items
- Monday 10 AM: Project status meetings
- Tuesday 2 PM: Team reviews
- Friday 4 PM: Week wrap-up
```

Approach 2: Agent-Managed Calendar Files

Create dedicated calendar files that your agent maintains:

Create calendar structure:

Windows:

```
mkdir %USERPROFILE%\.openclaw\workspace\calendar
echo. > %USERPROFILE%\.openclaw\workspace\calendar\2026-03.md
```

Mac/Linux:

```
mkdir -p ~/.openclaw/workspace/calendar
touch ~/.openclaw/workspace/calendar/2026-03.md
```

Monthly calendar template:

```
# March 2026 Calendar

## Recurring Events
- Every Monday 9 AM: Team standup
- Every Tuesday 2 PM: Client check-ins
- Every Friday 4 PM: Weekly review
- First Monday of month: Monthly planning
- Last Friday of month: Monthly retrospective

## Week 1 (March 3-7)
### Monday 3/3
- 9:00 AM - Team standup
- 10:00 AM - Monthly planning session
- 2:00 PM - New project kickoff

### Tuesday 3/4
- 9:00 AM - Focus time (coding)
- 2:00 PM - Client check-in: ABC Corp
- 4:00 PM - Code review session

[Continue for full month...]

## Special Events This Month
- March 15: Quarterly business review
- March 22: Company all-hands meeting
- March 28: Project deadline (Q1 report)

## Notes
- Spring break week: March 17-21 (lighter schedule)
- Conference calls may be rescheduled due to daylight saving time
```

Set up calendar maintenance reminders:

```
# Daily calendar update reminder
openclaw cron add \
  --name "Update calendar" \
  --cron "0 18 * * *" \
  --session main \
  --system-event "Review today's calendar and update tomorrow's schedule in the calendar files. Add
any new meetings or changes." \
  --wake now

# Weekly calendar planning
openclaw cron add \
  --name "Weekly calendar review" \
  --cron "0 16 * * 5" \
  --session main \
  --system-event "Review next week's calendar and ensure all meetings are properly scheduled and
prepared for." \
  --wake now
```

Approach 3: iCal Feed Reading (Advanced)

If you have access to your calendar's iCal URL, you can set up basic feed reading:

iCal integration script (save as `scripts/read-calendar.sh`):

Mac/Linux:

```
#!/bin/bash
# Basic iCal feed reader
ICAL_URL="YOUR_CALENDAR_ICAL_URL_HERE"
curl -s "\$ICAL_URL" | grep -E "(DTSTART|DTEND|SUMMARY)" | head -20
```

Windows (PowerShell script `scripts/read-calendar.ps1`):

```
# Basic iCal feed reader
\$icalUrl = "YOUR_CALENDAR_ICAL_URL_HERE"
\$response = Invoke-WebRequest -Uri \$icalUrl
\$response.Content | Select-String -Pattern "(DTSTART|DTEND|SUMMARY)" | Select-Object -First 20
```

Set up automated calendar reading:

```
openclaw cron add \
  --name "Read calendar feed" \
  --cron "0 8,12,18 * * *" \
  --session isolated \
  --message "Run the calendar reading script and update today's memory file with current calendar
information. Format the output in a readable way." \
  --announce
```

Approach 4: Google Calendar API (Brief Overview)

The full Google Calendar API integration is covered in the Business edition, but here's what's involved:

1. **Create Google Cloud Project** and enable Calendar API
2. **Set up OAuth credentials** or service account
3. **Configure OpenClaw** with API credentials
4. **Set up calendar sync** automation

Note: This requires technical setup including: - Google Cloud Console access - API key management - OAuth flow configuration - Error handling for API limits

For most Personal edition users, the memory file approach is more practical and reliable.

Calendar Event Templates

Create templates for different types of events:

Meeting template:

```
## [Time] - [Meeting Name]
- **Location:** [In-person/Virtual/Phone]
- **Attendees:** [List of people]
- **Agenda:** [Key topics]
- **Prep needed:** [Materials to review]
- **Follow-up:** [Action items expected]
```

Client call template:

```
## [Time] - Client Call: [Client Name]
- **Contact:** [Phone/Zoom link]
- **Purpose:** [Reason for call]
- **Account status:** [Current project/relationship]
- **Questions to ask:** [Prepare 3-4 questions]
- **Next steps:** [Expected outcomes]
```

Travel/appointment template:

```
## [Time] - [Appointment Type]
- **Location:** [Full address]
- **Travel time:** [How long to get there]
- **Parking/arrival:** [Special instructions]
- **What to bring:** [Documents/materials]
- **Duration:** [Expected length]
```

Integration with Task Management

Connect your calendar system with task management:

Meeting preparation tasks:

```
# 2026-03-12 Daily Log

## Calendar-Driven Tasks
- [ ] Prepare slides for 10:30 AM client presentation
- [ ] Review contract before 2 PM vendor meeting
- [ ] Gather Q1 numbers for 4 PM budget review
```

```
- [ ] Send agenda for tomorrow's team meeting

## Post-Meeting Tasks
- [ ] Follow up on client presentation (send proposal)
- [ ] Document vendor meeting decisions
- [ ] Update budget spreadsheet with new numbers
- [ ] Schedule follow-up meetings as needed
```

Your agent can automatically generate these preparation and follow-up tasks based on your calendar.

5. Weekly Review Automation

Weekly reviews are crucial for maintaining productivity and identifying improvement opportunities. Let's set up an automated system that analyzes your week and provides insights.

Basic Weekly Review Setup

Friday evening weekly review:

```
openclaw cron add \
  --name "Weekly review" \
  --cron "0 17 * * 5" \
  --tz "America/New_York" \
  --session isolated \
  --message "Generate my weekly review: 1) Read through memory files from Monday through today 2) Summarize key accomplishments 3) Identify tasks that didn't get completed 4) Note any patterns or insights about my productivity 5) Suggest improvements for next week 6) Highlight any important decisions or learnings" \
  --model "gpt" \
  --announce
```

Comprehensive Weekly Analysis

For a more detailed analysis, use this enhanced version:

```
openclaw cron add \
  --name "Deep weekly review" \
  --cron "0 16 * * 5" \
  --tz "America/New_York" \
  --session isolated \
  --message "Complete weekly analysis: ACCOMPLISHMENTS: Review all memory files from this week and list significant achievements, completed projects, and goals met. CHALLENGES: Identify tasks that were delayed, meetings that ran over, or goals that weren't met. PATTERNS: Note which days were most/least productive, what types of tasks I complete quickly vs slowly, when I tend to get distracted. TIME ANALYSIS: How did I spend my time? Meetings vs focused work vs admin tasks. IMPROVEMENTS: What could I do differently next week? Process changes, scheduling adjustments, or habit modifications. NEXT WEEK: Based on this analysis, what should be my priorities and focus areas?" \
  --model "gpt" \
  --announce
```

Goal Tracking Integration

Monthly goal tracking:

```
openclaw cron add \
  --name "Monthly goal review" \
  --cron "0 10 * * 1" \
  --tz "America/New_York" \
  --session isolated \
  --message "Monthly goal review (first Monday): 1) Check MEMORY.md for current monthly goals 2) Review progress based on this month's memory files 3) Calculate completion percentage for each goal 4) Identify obstacles that prevented progress 5) Adjust goals if needed for the remaining month 6) Set specific weekly targets to get back on track" \
  --announce
```

Weekly Metrics Collection

Create a system to track key productivity metrics:

Metrics tracking template (add to `MEMORY.md`):

```
# Productivity Metrics

## Weekly Tracking
### Week of [Date]
- **Meetings attended:** [number]
- **Focus blocks completed:** [number of 2+ hour uninterrupted work periods]
- **Tasks completed:** [number]
- **Tasks carried over:** [number]
- **Email response time:** [average hours to respond]
- **Work-life balance rating:** [1-10 scale]

## Monthly Trends
### March 2026
- Average weekly meetings: [number]
- Task completion rate: [percentage]
- Most productive day of week: [day]
- Biggest time wasters: [list]
- Key insights: [learnings]
```

Metrics collection automation:

```
openclaw cron add \
  --name "Collect weekly metrics" \
  --cron "30 17 * * 5" \
  --tz "America/New_York" \
  --session isolated \
  --message "Collect this week's productivity metrics: Count meetings from calendar entries in memory files, count completed vs incomplete tasks, estimate focus time vs meeting time, rate overall productivity and satisfaction for the week. Update the metrics tracking section in MEMORY.md with this data." \
  --announce
```

Pattern Recognition

Your agent can identify productivity patterns over time:

Pattern analysis automation:

```
openclaw cron add \
  --name "Monthly pattern analysis" \
  --cron "0 9 15 * *" \
  --tz "America/New_York" \
  --session isolated \
  --message "Deep pattern analysis (mid-month): Review 6 weeks of memory files and identify: 1) Which days of the week I'm most productive 2) What time of day I do my best work 3) Which types of tasks I consistently delay 4) Meeting patterns (too many/too few, good/bad times) 5) Energy and motivation cycles 6) External factors that affect productivity (weather, workload, etc.) Provide actionable insights and recommendations." \
  --model "gpt" \
  --announce
```

Progress Tracking Against Goals

Goal progress template:

```
# Goal Progress Tracking

## Current Quarter Goals (Q1 2026)
### Professional Goals
1. **Complete certification course**
   - Progress: 60% complete
   - Target date: March 31
   - Status: On track
   - Next milestone: Finish modules 7-8 by March 20

2. **Improve client response time**
   - Progress: Average 4 hours (down from 8 hours)
   - Target: Under 2 hours
   - Status: Behind target
   - Action needed: Check email more frequently

3. **Launch new project management system**
   - Progress: System selected, training started
   - Target date: March 15
   - Status: Slightly behind
   - Action needed: Schedule team training sessions

### Personal Goals
1. **Exercise 3x per week**
   - Progress: 2.3x per week average
   - Status: Close to target
   - Action: Schedule weekend workout

2. **Read 1 book per month**
   - Progress: Finished 2 books, 50% through 3rd
   - Status: On track
   - Current book: [Title]
```

Weekly Review Templates

Standard weekly review template:

```
# Weekly Review: Week of [Date]

## Wins This Week
- [Major accomplishments]
- [Goals achieved]
- [Positive feedback received]
- [Skills developed]

## Challenges
- [Tasks that didn't get done]
- [Obstacles encountered]
- [Time management issues]
- [Unexpected problems]

## Time Analysis
- **Meetings:** [X hours] - [productive/unproductive assessment]
- **Focus work:** [X hours] - [quality of deep work]
- **Admin tasks:** [X hours] - [efficiency rating]
- **Interruptions:** [frequency and impact]

## Key Insights
- [What worked well this week]
- [What didn't work]
- [Patterns noticed]
- [Lessons learned]

## Next Week's Focus
- [Top 3 priorities]
- [Process changes to try]
- [Things to avoid/minimize]
- [Goals to work toward]

## Action Items
- [ ] [Specific changes for next week]
- [ ] [Follow-ups needed]
- [ ] [Preparations for upcoming week]
```

Seasonal Reviews

Set up quarterly and annual reviews for bigger picture analysis:

Quarterly review (last day of quarter):

```
openclaw cron add \
  --name "Quarterly review" \
  --cron "0 14 31 3,6,9,12 *" \
  --session isolated \
  --message "Quarterly review: 1) Summarize the past 3 months using memory files 2) Assess progress
on quarterly goals 3) Identify major themes and patterns 4) Celebrate significant achievements 5)
Analyze what worked and what didn't 6) Set focus areas for next quarter 7) Update long-term goal
progress in MEMORY.md" \
  --model "gpt" \
  --announce
```

Annual review (December 30th):

```
openclaw cron add \
  --name "Annual review" \
  --cron "0 10 30 12 *" \
  --session isolated \
  --message "Annual review: Comprehensive year-end analysis of productivity, goal achievement,
career development, skill growth, relationship building, and life satisfaction. Review all
quarterly summaries and identify major themes, breakthrough moments, and areas for improvement in
the coming year." \
  --model "opus" \
  --announce
```

6. Productivity Templates

Having standardized templates makes your productivity system more consistent and easier to maintain. Here are templates for common productivity needs.

Daily Planner Template

Create your daily planner (`templates/daily-planner.md`):

```
# Daily Planner: [DATE]

## Morning Setup (5 minutes)
- [ ] Review calendar for the day
- [ ] Check weather and choose appropriate clothing
- [ ] Identify top 3 priorities
- [ ] Estimate time needed for each priority
- [ ] Block time for focused work

## Priority Tasks (limit to 3)
1. **[High Priority Task]**
   - Time allocated: [X hours]
   - Deadline: [date]
   - Success criteria: [what done looks like]

2. **[Medium Priority Task]**
   - Time allocated: [X hours]
   - Dependencies: [what you need first]
   - Next action: [specific first step]

3. **[Lower Priority Task]**
   - Time allocated: [X hours]
   - Can defer to: [alternative date if needed]
   - Quick win opportunity: [yes/no]

## Time Blocks
- **9:00-11:00 AM:** [Deep work block]
- **11:00-12:00 PM:** [Communication/emails]
- **1:00-3:00 PM:** [Meetings/calls]
- **3:00-5:00 PM:** [Project work]
- **5:00-6:00 PM:** [Admin/cleanup]

## Quick Capture
*Space for tasks, ideas, and notes that come up during the day*
-
```

```
-
-

## End of Day Review
- [ ] What got completed?
- [ ] What needs to carry over to tomorrow?
- [ ] What went well today?
- [ ] What could be improved tomorrow?

## Tomorrow's Prep
- [ ] Top priority identified
- [ ] Materials/files ready
- [ ] Calendar reviewed
- [ ] Workspace organized
```

Weekly Review Template

Weekly review structure (`templates/weekly-review.md`):

```
# Weekly Review: Week of [START DATE] - [END DATE]

## Achievement Summary
### Major Accomplishments
- [Significant project completions]
- [Goals reached]
- [Milestones achieved]
- [Problems solved]

### Tasks Completed
- [Count of completed tasks]
- [Types of tasks that went well]
- [Efficiency improvements noticed]

### Skills/Learning
- [New things learned this week]
- [Skills practiced or improved]
- [Training or development completed]

## Challenge Analysis
### Incomplete Items
- [Tasks that didn't get done]
- [Reasons for delays]
- [External factors that interfered]

### Time Management
- [Where time was wasted]
- [Meetings that ran over/were unproductive]
- [Distractions that derailed focus]

### Energy and Focus
- [Days when energy was low/high]
- [Times of day when most/least productive]
- [Factors affecting concentration]

## Insights and Patterns
### What Worked Well
- [Strategies that were effective]
- [Times when productivity was high]
```

```
- [Good decisions made]

### What Needs Improvement
- [Processes that slowed things down]
- [Habits that need changing]
- [Systems that need updating]

### Patterns Noticed
- [Recurring themes]
- [Cyclical productivity patterns]
- [External factors that consistently help/hurt]

## Next Week Planning
### Top 3 Priorities
1. [Most important goal for next week]
2. [Second priority]
3. [Third priority]

### Process Changes to Try
- [New approaches to test]
- [Tools or techniques to experiment with]
- [Habits to build or break]

### Schedule Optimization
- [Calendar changes needed]
- [Time blocks to protect]
- [Meetings to reschedule/decline]

## Goal Progress Update
### Monthly Goals
- [Goal 1]: [X% complete, on track/behind/ahead]
- [Goal 2]: [X% complete, on track/behind/ahead]
- [Goal 3]: [X% complete, on track/behind/ahead]

### Quarterly Goals
- [Goal 1]: [Progress assessment]
- [Goal 2]: [Progress assessment]
- [Goal 3]: [Progress assessment]

## Action Items for Next Week
- [ ] [Specific action to take Monday]
- [ ] [Process to implement]
- [ ] [Follow-up to complete]
- [ ] [Preparation task for upcoming work]
```

Goal Tracking Template

Goal setting and tracking (`templates/goal-tracker.md`):

```
# Goal Tracking System

## Current Active Goals

### Professional Goals (Q[X] 20XX)
#### Goal 1: [Primary Professional Goal]
- **Target completion:** [Date]
- **Success metric:** [How you'll measure success]
- **Current progress:** [X% or specific milestone]
```

```
- **Next milestone:** [Specific next achievement]
- **Weekly actions:** [What you do each week toward this goal]
- **Obstacles:** [Current challenges]
- **Support needed:** [Resources or help required]

#### Goal 2: [Secondary Professional Goal]
- **Target completion:** [Date]
- **Success metric:** [Measurable outcome]
- **Current progress:** [Status]
- **Next milestone:** [What's next]
- **Weekly actions:** [Regular activities]
- **Obstacles:** [Challenges]
- **Support needed:** [Requirements]

### Personal Development Goals
#### Goal 1: [Primary Personal Goal]
- **Target completion:** [Date]
- **Success metric:** [How you'll know you succeeded]
- **Current progress:** [Where you stand now]
- **Next milestone:** [Next achievement marker]
- **Weekly actions:** [Regular activities toward goal]
- **Obstacles:** [What's in the way]
- **Support needed:** [Help required]

### Health & Wellness Goals
#### Goal 1: [Health Goal]
- **Target:** [Specific, measurable target]
- **Current status:** [Where you are now]
- **Weekly target:** [What you aim for each week]
- **Tracking method:** [How you measure progress]
- **Success factors:** [What helps you succeed]
- **Obstacles:** [What gets in the way]

## Goal Review Schedule
- **Daily:** Quick progress check (2 minutes)
- **Weekly:** Detailed review and planning (15 minutes)
- **Monthly:** Deep assessment and adjustment (45 minutes)
- **Quarterly:** Major review and goal setting (2 hours)

## Progress Tracking
### This Month's Focus
Week 1: [Focus area]
Week 2: [Focus area]
Week 3: [Focus area]
Week 4: [Focus area]

### Monthly Milestones
- [ ] [Milestone 1]
- [ ] [Milestone 2]
- [ ] [Milestone 3]
- [ ] [Milestone 4]

### Quarterly Checkpoints
- [ ] [Q1 checkpoint]
- [ ] [Q2 checkpoint]
- [ ] [Q3 checkpoint]
- [ ] [Q4 checkpoint]

## Success Factors Analysis
### What Accelerates Progress
- [Habits that help]
- [Environmental factors]
```

```
- [Support systems]
- [Tools and resources]

### What Slows Progress
- [Common obstacles]
- [Bad habits]
- [Environmental challenges]
- [Resource constraints]

## Reward System
### Weekly Rewards (for consistent progress)
- [Small reward for meeting weekly targets]

### Monthly Rewards (for milestone completion)
- [Medium reward for significant progress]

### Quarterly Rewards (for major achievements)
- [Larger reward for completing major goals]

### Annual Rewards (for yearly success)
- [Significant celebration for yearly achievements]
```

Habit Tracker Template

Daily habit tracking (`templates/habit-tracker.md`):

```
# Habit Tracker

## Current Habits (Month: [MONTH YEAR])

### Health Habits
| Date | Exercise | Drink Water | Sleep 7+ hrs | Healthy Breakfast |
|------|----------|-------------|--------------|-------------------|
| 01   | Γ£ô/Γ£ù      | Γ£ô/Γ£ù         | Γ£ô/Γ£ù          | Γ£ô/Γ£ù               |
| 02   | Γ£ô/Γ£ù      | Γ£ô/Γ£ù         | Γ£ô/Γ£ù          | Γ£ô/Γ£ù               |
| 03   | Γ£ô/Γ£ù      | Γ£ô/Γ£ù         | Γ£ô/Γ£ù          | Γ£ô/Γ£ù               |
[Continue for full month...]

### Productivity Habits
| Date | Morning Planning | Email Batching | Focus Blocks | Evening Review |
|------|------------------|----------------|--------------|----------------|
| 01   | Γ£ô/Γ£ù              | Γ£ô/Γ£ù            | Γ£ô/Γ£ù          | Γ£ô/Γ£ù            |
| 02   | Γ£ô/Γ£ù              | Γ£ô/Γ£ù            | Γ£ô/Γ£ù          | Γ£ô/Γ£ù            |
| 03   | Γ£ô/Γ£ù              | Γ£ô/Γ£ù            | Γ£ô/Γ£ù          | Γ£ô/Γ£ù            |
[Continue for full month...]

### Learning Habits
| Date | Read 30min | Online Course | Skill Practice | Industry News |
|------|------------|---------------|----------------|---------------|
| 01   | Γ£ô/Γ£ù        | Γ£ô/Γ£ù           | Γ£ô/Γ£ù            | Γ£ô/Γ£ù           |
| 02   | Γ£ô/Γ£ù        | Γ£ô/Γ£ù           | Γ£ô/Γ£ù            | Γ£ô/Γ£ù           |
| 03   | Γ£ô/Γ£ù        | Γ£ô/Γ£ù           | Γ£ô/Γ£ù            | Γ£ô/Γ£ù           |
[Continue for full month...]

## Habit Definitions
### Exercise
- **Goal:** 30+ minutes of physical activity
- **Examples:** Gym, walk, bike ride, yoga, sports
```

```
- **Minimum:** 20 minutes counts as partial success

### Morning Planning
- **Goal:** 5 minutes reviewing day ahead
- **Includes:** Check calendar, identify priorities, set intentions
- **Time:** Before starting work tasks

### Focus Blocks
- **Goal:** 90+ minutes of uninterrupted deep work
- **Requirements:** Phone off, distractions eliminated, single task focus
- **Frequency:** At least one per workday

### Evening Review
- **Goal:** 10 minutes reflecting on the day
- **Includes:** Note accomplishments, plan tomorrow, capture lessons
- **Time:** Before personal time begins

## Monthly Analysis
### Success Rates (Track percentages)
- Exercise: __%
- Healthy eating: __%
- Morning planning: __%
- Focus blocks: __%
- Evening review: __%
- Reading: __%
- Skill practice: __%

### Patterns Observed
- **Best streak days:** [Which days tend to be most successful]
- **Challenging days:** [When habits are hardest to maintain]
- **Success factors:** [What helps maintain habits]
- **Obstacles:** [What derails habit practice]

### Next Month Adjustments
- **Habits to add:** [New habits to introduce]
- **Habits to modify:** [Existing habits to adjust]
- **Habits to remove:** [Habits that aren't working]
- **Environmental changes:** [Modifications to support success]
```

Project Planning Template

Project organization (`templates/project-planner.md`):

```
# Project Planner: [PROJECT NAME]

## Project Overview
- **Project Name:** [Full project name]
- **Start Date:** [When project begins]
- **Target Completion:** [Deadline]
- **Project Type:** [Development/Research/Implementation/etc.]
- **Priority Level:** [High/Medium/Low]
- **Stakeholders:** [Who's involved/affected]

## Objectives and Success Criteria
### Primary Objective
[What this project is meant to achieve]

### Success Criteria
```

```
- [ ] [Measurable outcome 1]
- [ ] [Measurable outcome 2]
- [ ] [Measurable outcome 3]

### Key Performance Indicators
- [Metric 1]: [Target value]
- [Metric 2]: [Target value]
- [Metric 3]: [Target value]

## Project Phases
### Phase 1: [Planning/Discovery]
- **Duration:** [Time estimate]
- **Key activities:**
  - [ ] [Activity 1]
  - [ ] [Activity 2]
  - [ ] [Activity 3]
- **Deliverables:**
  - [Deliverable 1]
  - [Deliverable 2]

### Phase 2: [Implementation/Development]
- **Duration:** [Time estimate]
- **Key activities:**
  - [ ] [Activity 1]
  - [ ] [Activity 2]
  - [ ] [Activity 3]
- **Deliverables:**
  - [Deliverable 1]
  - [Deliverable 2]

### Phase 3: [Testing/Review]
- **Duration:** [Time estimate]
- **Key activities:**
  - [ ] [Activity 1]
  - [ ] [Activity 2]
  - [ ] [Activity 3]
- **Deliverables:**
  - [Deliverable 1]
  - [Deliverable 2]

### Phase 4: [Launch/Completion]
- **Duration:** [Time estimate]
- **Key activities:**
  - [ ] [Activity 1]
  - [ ] [Activity 2]
  - [ ] [Activity 3]
- **Deliverables:**
  - [Deliverable 1]
  - [Deliverable 2]

## Resources Required
### People
- [Role 1]: [Time commitment]
- [Role 2]: [Time commitment]
- [External resources]: [Consultants, vendors, etc.]

### Tools and Technology
- [Software/tools needed]
- [Hardware requirements]
- [Subscriptions or licenses]

### Budget
```

```
- [Budget category 1]: [Amount]
- [Budget category 2]: [Amount]
- [Total estimated cost]: [Amount]

## Risk Assessment
### High Probability Risks
- **Risk:** [Description]
  - **Impact:** [High/Medium/Low]
  - **Mitigation:** [How to prevent/address]

### Medium Probability Risks
- **Risk:** [Description]
  - **Impact:** [High/Medium/Low]
  - **Mitigation:** [How to prevent/address]

### Contingency Plans
- [Plan A if major obstacle occurs]
- [Plan B if timeline slips significantly]
- [Plan C if resources become unavailable]

## Communication Plan
### Regular Updates
- **Frequency:** [Weekly/bi-weekly/monthly]
- **Format:** [Email/meeting/dashboard]
- **Recipients:** [Stakeholder list]

### Milestone Reports
- [Milestone 1]: [Report date and recipients]
- [Milestone 2]: [Report date and recipients]
- [Milestone 3]: [Report date and recipients]

## Weekly Project Check-ins
- [ ] Review progress against timeline
- [ ] Identify and address blockers
- [ ] Update stakeholders on status
- [ ] Adjust plans if needed
- [ ] Celebrate completed milestones
```

7. Building Your Personal System

Now that you understand the components, let's build a personalized productivity system that matches your specific workflow and preferences.

System Configuration Wizard

Answer these questions to customize your setup:

Work Schedule: - What time do you typically start work? - What time do you finish work? - Do you work weekends? - What are your most productive hours? - When do you prefer to receive reminders?

Communication Preferences: - How do you want to receive notifications? (Telegram, Discord, WhatsApp, etc.) - How often do you want productivity check-ins? (Every 30min, 1hr, 2hr?) - Do you want weekend notifications? - What's your preferred tone? (Professional, casual, motivational?)

Task Management Style: - Do you prefer detailed daily planning or flexible scheduling? - How far ahead do you like to plan? (Daily, weekly, monthly?) - Do you work better with time blocks or task lists? - How do you handle interruptions and urgent tasks?

Goal Orientation: - Are you working toward specific short-term goals? - Do you have quarterly or yearly objectives? - Do you prefer process goals or outcome goals? - How often do you want progress reviews?

Sample Configuration: The Focused Professional

For someone who works standard business hours with a focus on deep work:

Basic config additions:

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "every": "45m",
        "target": "last",
        "activeHours": {
          "start": "08:30",
          "end": "18:00",
          "timezone": "America/New_York"
        }
      }
    }
  }
}
```

HEARTBEAT.md for focused professional:

```
# Focused Professional Heartbeat

## Always Check
- Any urgent emails or messages requiring <2 hour response
- Calendar events in next 90 minutes
- High-priority tasks overdue by 4+ hours

## During Deep Work Hours (9-11 AM, 2-4 PM)
- ONLY interrupt for true emergencies
- Queue non-urgent items for next break

## During Communication Hours (11-12, 4-5 PM)
- Process email backlog
- Check project status updates
- Review and respond to team messages

## End of Day (after 5 PM)
- Summarize day's accomplishments
- Note tomorrow's top 3 priorities
- Clear mental space for personal time
```

Cron jobs for focused professional:

```
# Morning briefing
openclaw cron add \
  --name "Morning focus briefing" \
  --cron "0 8 * * 1-5" \
  --message "Morning briefing: Today's calendar, weather, top 3 priorities from yesterday's notes,
and motivation for deep work blocks." \
  --announce

# Deep work reminder
openclaw cron add \
  --name "Deep work block starting" \
  --cron "55 8,13 * * 1-5" \
  --session main \
  --system-event "Deep work block starting in 5 minutes. Close email, silence phone, prepare
materials." \
  --wake now

# End of day review
openclaw cron add \
  --name "End of day wrap-up" \
  --cron "0 17 * * 1-5" \
  --message "Day completion review: What got done, what carries to tomorrow, any insights about
today's productivity." \
  --announce
```

Sample Configuration: The Flexible Entrepreneur

For someone with irregular hours and multiple projects:

HEARTBEAT.md for flexible entrepreneur:

```
# Entrepreneur Heartbeat

## Business Critical (every check)
- Client communications requiring same-day response
- Payment/invoice notifications
- System alerts or downtime issues

## Project Juggling
- Rotate through active projects - which needs attention?
- Check if any project hasn't been touched in 48 hours
- Note which project is blocking others

## Opportunity Tracking
- Industry news that affects my business
- Potential partnerships or collaborations mentioned
- New market opportunities

## Energy Management
- If working >10 hours, suggest break or delegation
- If low-energy, recommend easier tasks from backlog
- Track patterns of high/low energy periods
```

Cron jobs for flexible entrepreneur:

```
# Weekly business review
openclaw cron add \
  --name "Weekly business metrics" \
  --cron "0 18 * * 0" \
  --message "Weekly business review: Revenue, client satisfaction, project progress, new
opportunities, and strategic priorities for next week." \
  --announce

# Monthly strategic thinking
openclaw cron add \
  --name "Monthly strategy session" \
  --cron "0 10 1 * *" \
  --message "Monthly strategic review: Business goals progress, market changes, new opportunities,
resource allocation, and strategic adjustments needed." \
  --model "gpt" \
  --announce
```

Sample Configuration: The Student/Learner

For someone focused on education and skill development:

HEARTBEAT.md for student:

```
# Student Productivity Heartbeat

## Academic Priority
- Assignment deadlines within 3 days
- Exam dates within 1 week
- Class attendance reminders (30 min before)

## Learning Progress
- Course material that's falling behind
- Study group or office hours opportunities
- Skills practice that's been skipped

## Time Management
- Balance between different subjects/courses
- Adequate break time and rest
- Social activities and exercise

## Long-term Focus
- Career development opportunities
- Networking events or internship applications
- Skill building outside formal coursework
```

Customization Guidelines

Start Simple: - Begin with basic heartbeat and one morning briefing - Add complexity gradually as you get comfortable - Don't over-engineer initially

Iterate Based on Usage: - Track which notifications you actually find useful - Adjust timing based on your natural rhythms - Modify tone and content based on what motivates you

Seasonal Adjustments: - Busy seasons may need different notification frequency - Vacation periods should have reduced automation - Major life changes require system updates

Integration with External Tools

For users of existing productivity tools:

Notion/Obsidian users: - Use OpenClaw for reminders and check-ins - Keep detailed notes in your preferred tool - Let OpenClaw read and summarize from exported files

Google Calendar users: - Manually sync important events to memory files - Use OpenClaw for task management and reminders - Business edition covers full API integration

Todoist/Things users: - Use OpenClaw as the "meta-manager" that reminds you to check your main system - Focus OpenClaw on higher-level patterns and insights - Use memory files for reflection and review

Advanced Customization Options

Multiple Agent Personalities: You can configure different "modes" for different contexts:

```
{
  "agents": {
    "list": [
      {
        "id": "work",
        "default": true,
        "heartbeat": {
          "every": "30m",
          "activeHours": {"start": "08:00", "end": "18:00"}
        }
      },
      {
        "id": "personal",
        "heartbeat": {
          "every": "2h",
          "activeHours": {"start": "19:00", "end": "23:00"}
        }
      }
    ]
  }
}
```

Context-Aware Scheduling: Use different cron jobs for different life phases:

```
# Project crunch time - more frequent check-ins
openclaw cron add --name "Crunch mode check" --every "15m" --enabled

# Vacation mode - minimal interruptions
openclaw cron add --name "Vacation minimal" --every "4h" --enabled
```

Building Your Implementation Plan

Week 1: Foundation - Set up basic heartbeat (every 30-60 minutes) - Create simple HEARTBEAT.md with 3-4 check items - Add one morning briefing cron job - Start using daily memory files

Week 2: Task Management - Begin capturing tasks in memory files - Set up evening review automation - Create your first productivity templates - Establish basic habit tracking

Week 3: Analysis and Optimization - Add weekly review automation - Start tracking productivity metrics - Identify patterns in your work habits - Adjust notification timing and frequency

Week 4: Advanced Features - Set up monthly goal tracking - Create project templates - Add seasonal review scheduling - Fine-tune your personal system

Week 5 and Beyond: Continuous Improvement - Regular system reviews and updates - Seasonal adjustments - Integration with new tools and workflows - Sharing insights with others

8. Troubleshooting

Even the best productivity systems encounter issues. Here are solutions to common problems you might face with your OpenClaw automation.

Common Setup Issues

Problem: Cron jobs not running

Symptoms: You've set up cron jobs but they're not triggering at the scheduled times.

Solutions: 1. Check if the Gateway is running continuously: `bash openclaw gateway status`

2. Verify cron is enabled in your configuration:

```
{
  "cron": {
    "enabled": true
  }
}
```

3. Check your timezone settings:

```
openclaw cron list
# Look for timezone in the output
```

4. Verify the cron expression is correct:
   - Use https://crontab.guru/ to validate your cron expressions
   - Remember: OpenClaw uses 5-field expressions (minute hour day month day-of-week)
5. Check cron job status:

```
openclaw cron runs --id YOUR_JOB_ID --limit 10
```

Problem: Heartbeat not delivering notifications

Symptoms: Heartbeat is running but you're not receiving messages.

Solutions: 1. Check heartbeat configuration: `json { "agents": { "defaults": { "heartbeat": { "target": "last", // Make sure this is set "every": "30m" } } } }`

2. Verify your channel is configured properly:

```
openclaw config show
# Check that your communication channel (Telegram, Discord, etc.) is set up
```

3. Check if heartbeat is outside active hours:
   - Review your `activeHours` configuration
   - Temporarily remove active hours to test
4. Look for HEARTBEAT_OK responses:
   - If agent replies with only `HEARTBEAT_OK`, no message is sent (this is normal)
   - Check your HEARTBEAT.md for items that require attention

Problem: Memory files not being created or updated

Symptoms: Daily memory files aren't appearing or agent isn't writing to them.

Solutions: 1. Check workspace permissions: ```bash # Windows dir %USERPROFILE%.openclaw

Mac/Linux
ls -la ~/.openclaw/workspace ```

2. Create memory directory manually if needed:

```
# Windows
mkdir %USERPROFILE%\.openclaw\workspace\memory

# Mac/Linux
mkdir -p ~/.openclaw/workspace/memory
```

3. Verify agent has write access:
   - Try manually creating a test file in the workspace
   - Check that the agent isn't running in read-only mode
4. Check file permissions (Mac/Linux):

```
chmod 755 ~/.openclaw/workspace
chmod 755 ~/.openclaw/workspace/memory
```

Configuration Problems

Problem: Agent personality doesn't match your preferences

Symptoms: Notifications are too frequent, too formal, or not helpful.

Solutions: 1. Adjust heartbeat frequency: `json { "agents": { "defaults": { "heartbeat": { "every": "60m" // Increase for less frequent notifications } } } }`

2. Update HEARTBEAT.md to be more specific:

```
# Heartbeat Checklist

## Only Alert For
- Meetings starting in next 15 minutes
```

```
- Urgent emails from clients
- Tasks overdue by more than 1 day

## Never Alert For
- Routine status updates
- Non-urgent administrative tasks
- Social media or entertainment content
```

3. Customize system prompts:

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "prompt": "You are a calm, professional assistant. Only notify me of truly important
items. Keep messages brief and actionable."
      }
    }
  }
}
```

Problem: Time zone issues

Symptoms: Notifications coming at wrong times, cron jobs running off-schedule.

Solutions: 1. Set explicit timezone in cron jobs: `bash openclaw cron add \ --name "Morning briefing" \ --cron "0 7 * * *" \ --tz "America/New_York" # Always specify timezone`

2. Check system timezone:

```
# Windows
tzutil /g

# Mac/Linux
date
timedatectl  # On Linux systems with systemd
```

3. Set user timezone in config:

```
{
  "agents": {
    "defaults": {
      "userTimezone": "America/New_York"
    }
  }
}
```

Problem: Notifications going to wrong channel

Symptoms: Messages appearing in wrong chat, or not appearing at all.

Solutions: 1. Check channel configuration: `bash openclaw config show`

2. Set explicit delivery targets:

```
openclaw cron add \
  --name "Test notification" \
  --at "5m" \
  --announce \
  --channel telegram \
  --to "YOUR_CHAT_ID"
```

3. Test channel connectivity:

```
openclaw message send --channel telegram --to "YOUR_CHAT_ID" --message "Test message"
```

Performance Issues

Problem: System running slowly or using too much memory

Symptoms: Delays in responses, high CPU/memory usage, timeout errors.

Solutions: 1. Reduce heartbeat frequency: `json { "agents": { "defaults": { "heartbeat": { "every": "60m" // Or longer } } } }`

2. Simplify HEARTBEAT.md:

```
# Simple Heartbeat

- Check for urgent items only
- Skip detailed analysis
- Reply HEARTBEAT_OK unless critical
```

3. Use isolated cron jobs instead of main session:

```
openclaw cron add \
  --session isolated \  # Runs in separate context
  --message "Brief task check"
```

4. Clean up old memory files:

```
# Archive old memory files (older than 90 days)
# Windows PowerShell
Get-ChildItem \$env:USERPROFILE\.openclaw\workspace\memory -Name "*.md" | Where-Object {\$_ -
match "\d{4}-\d{2}-\d{2}.md" -and (Get-Date \$matches[0]) -lt (Get-Date).AddDays(-90)} |
Move-Item -Destination \$env:USERPROFILE\.openclaw\workspace\memory\archive\

# Mac/Linux
find ~/.openclaw/workspace/memory -name "*.md" -type f -mtime +90 -exec mv {} ~/.openclaw/
workspace/memory/archive/ \;
```

Problem: API rate limits or quota exceeded

Symptoms: Error messages about rate limits, failed requests, or quota warnings.

Solutions: 1. Use cheaper models for routine tasks: `bash openclaw cron add \ --model "gpt-3.5-turbo" \ # Instead of GPT-4 --message "Simple daily check"`

2. Reduce automation frequency:

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "every": "120m"  # 2 hours instead of 30 minutes
      }
    }
  }
}
```

3. Use `target: "none"` for internal processing:

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "target": "none"  # Agent processes but doesn't send messages
      }
    }
  }
}
```

4. Consolidate multiple small cron jobs into fewer comprehensive ones:

```
# Instead of 5 separate jobs, create 1 comprehensive job
openclaw cron add \
  --name "Complete daily review" \
  --cron "0 18 * * *" \
  --message "Check email, calendar, tasks, and weather in one comprehensive review"
```

Content and Workflow Issues

Problem: Agent not capturing tasks properly

Symptoms: Mentioned tasks don't appear in memory files, inconsistent task tracking.

Solutions: 1. Be explicit about task capture: `You: "I need to call the vendor tomorrow morning. Please add that to my task list."`

2. Use consistent task language:

```
"Add task: Call vendor about shipping delay"
"Remember to: Review budget proposal by Friday"
"Task for later: Update client presentation slides"
```

3. Set up task capture reminders:

```bash
openclaw cron add \
  --name "Daily task review" \
  --cron "0 17 * * *" \
  --session main \
  --system-event "Review today's conversation for any tasks that need to be captured in memory files" \
  --wake now
```

4. Create task templates in MEMORY.md:

```markdown
# Task Capture Template

When capturing tasks, always include:
- [ ] Task description
- Due date (if specified)
- Priority level
- Any context or dependencies
- Person responsible (if delegated)
```

Problem: Weekly/monthly reviews are too generic

Symptoms: Reviews lack specific insights, don't identify useful patterns, feel like busywork.

Solutions: 1. Add specific analysis prompts: `bash openclaw cron add \ --name "Detailed weekly review" \ --message "Analyze this week: 1) Count completed vs incomplete tasks 2) Identify which types of tasks I delay most 3) Note energy patterns by day/time 4) Compare this week to last week 5) Suggest 2 specific improvements for next week" \ --model "gpt-4"`

2. Track specific metrics:

```markdown
# Weekly Metrics to Track

- Number of deep work sessions (2+ hours uninterrupted)
- Meeting hours vs independent work hours
- Response time to emails/messages
- Tasks completed vs added
- Sleep quality correlation with productivity
- Exercise days vs energy levels
```

3. Use different review types:

```bash
# Tactical weekly review (what happened)
openclaw cron add --name "Tactical review" --cron "0 17 * * 5"

# Strategic monthly review (what it means)
openclaw cron add --name "Strategic review" --cron "0 10 * * 1" # First Monday
```

Problem: Information overload in briefings

Symptoms: Morning briefings too long, too much irrelevant information, overwhelming rather than helpful.

Solutions: 1. Streamline briefing content:

```bash
openclaw cron add \ --name "Focused morning briefing" \ --message "Brief morning update: Only today's weather if I might go outside, only calendar items in next 4 hours, only high-priority tasks from yesterday. Keep under 100 words total." \ --announce
```

2. Use progressive disclosure:

```
# Briefing Priority Order

## Always Include (< 50 words)
- Next meeting time and location
- Weather if outdoor activities planned
- One most important task for today

## Include if Relevant (< 50 words)
- Traffic/transit alerts
- Urgent messages received overnight
- Deadline reminders for this week

## Skip Unless Critical
- Routine status updates
- Non-urgent news
- General motivational content
```

3. Customize by day type:

```
# Monday briefings - week planning focus
openclaw cron add --cron "0 7 * * 1" --message "Monday brief: This week's priorities and calendar overview"

# Other weekdays - day-specific focus
openclaw cron add --cron "0 7 * * 2-5" --message "Daily brief: Today's schedule and immediate priorities only"
```

Advanced Troubleshooting

Problem: Memory search not finding relevant information

Symptoms: Agent can't recall things you know you discussed, memory searches return irrelevant results.

Solutions: 1. Check memory search configuration:

```json
{ "agents": { "defaults": { "memorySearch": { "enabled": true, "provider": "openai" // or "local" if using local embeddings } } } }
```

2. Use more specific search terms:

```
Instead of: "Find information about the meeting"
Try: "Search memory for 'client presentation budget March'"
```

3. Manual memory organization:

```
# MEMORY.md - Key Information
```

```
## Active Projects
- Q1 Budget Review (due March 31)
- Client Presentation (ABC Corp, March 15)
- Team Reorganization (ongoing)

## Important Contacts
- Vendor: Mike Johnson, mjohnson\@vendorcorp.com
- Client: Sarah Chen, schen\@abccorp.com

## Recurring Tasks
- Weekly team meeting: Mondays 10 AM
- Monthly budget review: First Friday of month
```

Problem: System conflicts with work/life balance

Symptoms: Getting work notifications during personal time, or personal reminders during work.

Solutions: 1. Set strict active hours: `json { "agents": { "defaults": { "heartbeat": { "activeHours": { "start": "08:00", "end": "18:00" // No notifications outside work hours } } } } }`

2. Use different agents for work vs personal:

```
{
  "agents": {
    "list": [
      {
        "id": "work",
        "workspace": "~/.openclaw/work",
        "heartbeat": {"activeHours": {"start": "08:00", "end": "18:00"}}
      },
      {
        "id": "personal",
        "workspace": "~/.openclaw/personal",
        "heartbeat": {"activeHours": {"start": "19:00", "end": "22:00"}}
      }
    ]
  }
}
```

3. Weekend-specific configuration:

```
# Minimal weekend heartbeat
openclaw cron add \
  --name "Weekend minimal check" \
  --cron "0 10,18 * * 6,0" \
  --message "Weekend check: Only urgent personal items or Monday prep if Sunday evening" \
  --announce
```

Getting Help

If you're still having issues:

1. **Check the logs:**

```
openclaw gateway logs --tail 50
```

2. **Test individual components:**

```
# Test heartbeat manually
openclaw system event --text "Manual heartbeat test" --mode now

# Test cron job manually
openclaw cron run YOUR_JOB_ID

# Test memory writing
openclaw message send --message "Please write 'Test entry' to today's memory file"
```

3. **Community resources:**
   - OpenClaw Discord community
   - GitHub discussions
   - Documentation wiki
4. **Simplify and rebuild:**
   - Disable all automation
   - Start with just basic heartbeat
   - Add one feature at a time
   - Test each addition thoroughly

Remember: A simple system that works is better than a complex system that doesn't. Start minimal and grow your automation gradually as you understand what works for your lifestyle.

Conclusion

You now have everything you need to build your AI-powered productivity system. Starting with basic reminders and daily briefings, you can gradually expand into sophisticated task management, goal tracking, and personal analytics.

The key to success is starting simple and iterating based on what actually improves your daily life. Your OpenClaw agent isn't just a tool—it's a productivity partner that learns your patterns and helps you stay on track with your goals.

In the next chapter, we'll explore advanced automation techniques and integrations that can further enhance your productivity system. But for now, take time to implement these foundational elements and let them become part of your daily routine.

Your AI assistant is ready to help you never forget anything important again, stay focused on what matters most, and build sustainable habits that drive you toward your goals. The future of personal productivity is here, and it's more accessible than ever.

Start with tomorrow's morning briefing, and watch how quickly having an AI chief of staff transforms your daily effectiveness.

CHAPTER 09

Deployment

Running Your Agent 24/7

Chapter 9: Deployment & Persistence — Running Your Agent 24/7

If your agent isn't running, nothing works. This chapter ensures it never stops.

Building an AI automation system is worthless if it only runs when you remember to start it. The true power of OpenClaw emerges when it operates continuously—monitoring emails, processing scheduled tasks, responding to triggers, and executing your automations whether you're awake, asleep, or on vacation.

This chapter transforms your OpenClaw installation from a manual tool into a persistent, production-ready system that starts automatically, recovers from failures, and maintains itself across reboots and updates.

Why 24/7 Persistence Matters

The Reality of Automation

Every automation you build depends on your OpenClaw gateway being active:

- **Cron jobs** only execute if the gateway process is running
- **Heartbeat checks** require an active session to poll for updates
- **Channel integrations** (Telegram, Discord, WhatsApp) need persistent connections
- **Node communications** fail if the gateway is offline
- **Webhook receivers** return 404 when the service is down

A manually started agent isn't automation—it's an expensive interactive tool.

The Economics of Uptime

Consider the cost of downtime: - Missed business opportunities (price alerts, market signals) - Failed integrations (APIs timing out, webhooks dropping) - Broken workflows (file processing, data sync, notifications) - Lost productivity (manual restarts, debugging sessions)

The investment in proper deployment pays for itself within days.

Production Mindset

Treat your OpenClaw deployment like business infrastructure: - **Automatic startup** on system boot - **Process supervision** with automatic restart on crashes - **Log collection** for debugging and monitoring - **Update procedures** that maintain service continuity - **Backup strategies** that protect against data loss

This chapter provides battle-tested configurations for all major operating systems.

Windows Deployment

Windows users have two paths: native Windows (planned but not yet available) or the recommended WSL2 approach. WSL2 provides the full Linux experience while running on Windows, giving you access to the complete OpenClaw ecosystem.

WSL2 Setup and Installation

Initial WSL2 Installation

Open PowerShell as Administrator:

```
# Install WSL2 with Ubuntu (recommended)
wsl --install

# Or choose a specific distribution
wsl --list --online
wsl --install -d Ubuntu-24.04
```

Reboot when Windows prompts you.

Enable systemd in WSL2

OpenClaw requires systemd for service management. Configure it in your WSL terminal:

```
# Create WSL configuration
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true
EOF
```

Restart WSL from PowerShell:

```
wsl --shutdown
```

Re-open your Ubuntu terminal and verify systemd:

```
systemctl --user status
```

You should see systemd running without errors.

Install OpenClaw in WSL2

Follow the standard Linux installation inside WSL:

```
# Install Node.js 22+
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs

# Install OpenClaw
npm install -g openclaw\@latest

# Run onboarding and install daemon
openclaw onboard --install-daemon
```

Windows Task Scheduler (Alternative Approach)

For users preferring native Windows tools, you can use Task Scheduler to start WSL and OpenClaw automatically.

Create Auto-Start Task

Open Task Scheduler (taskschd.msc) and create a new task:

General Tab: - Name: `OpenClaw Gateway Startup` - Description: `Start OpenClaw Gateway in WSL2 at boot` - Security options: `Run whether user is logged on or not` - Run with highest privileges: Γ£ô - Configure for: `Windows 11` (or your version)

Triggers Tab: - New trigger ΓåÆ `At startup` - Delay task: `30 seconds` (allow WSL to initialize)

Actions Tab: - Program: `wsl.exe` - Arguments: `-d Ubuntu -- systemctl --user start openclaw-gateway`

Conditions Tab: - Uncheck `Start the task only if the computer is on AC power` - Check `Start the task only if the computer is on AC power` if this is a laptop

Settings Tab: - Allow task to be run on demand: Γ£ô - Run task as soon as possible after scheduled start is missed: Γ£ô - If task fails, restart every: `1 minute` up to `3` times

PowerShell Startup Script Template

Create a more robust startup script:

```
# openclaw-startup.ps1
param(
    [string]\$DistroName = "Ubuntu",
    [int]\$MaxRetries = 3,
    [int]\$RetryDelay = 30
)

Write-Host "Starting OpenClaw Gateway in WSL2..."

for (\$i = 1; \$i -le \$MaxRetries; \$i++) {
    Write-Host "Attempt \$i of \$MaxRetries"

    try {
        # Check if WSL distro is running
        \$wslStatus = wsl -l --running | Select-String \$DistroName
        if (-not \$wslStatus) {
            Write-Host "Starting WSL distro: \$DistroName"
            wsl -d \$DistroName --exec /bin/true
            Start-Sleep -Seconds 5
        }

        # Start OpenClaw gateway service
        \$result = wsl -d \$DistroName -- systemctl --user start openclaw-gateway

        # Verify service started
        \$status = wsl -d \$DistroName -- systemctl --user is-active openclaw-gateway
        if (\$status -eq "active") {
            Write-Host "OpenClaw Gateway started successfully"
            exit 0
        } else {
            throw "Service failed to start properly"
        }
    }
    catch {
        Write-Warning "Attempt \$i failed: \$_"
        if (\$i -lt \$MaxRetries) {
            Write-Host "Waiting \$RetryDelay seconds before retry..."
            Start-Sleep -Seconds \$RetryDelay
        }
    }
}

Write-Error "Failed to start OpenClaw Gateway after \$MaxRetries attempts"
exit 1
```

Save this as `C:\Scripts\openclaw-startup.ps1` and use it in your Task Scheduler action:

- Program: `powershell.exe`
- Arguments: `-ExecutionPolicy Bypass -File "C:\Scripts\openclaw-startup.ps1"`

PM2 Process Manager (Alternative)

For users comfortable with Node.js tooling, PM2 provides excellent process management:

```
# Install PM2 globally
npm install -g pm2

# Start OpenClaw with PM2
```

```
pm2 start openclaw -- gateway run --port 18789

# Enable PM2 startup (run on boot)
pm2 startup

# Save current PM2 configuration
pm2 save
```

PM2 commands for management:

```
# View running processes
pm2 list

# View logs
pm2 logs openclaw

# Restart gateway
pm2 restart openclaw

# Stop gateway
pm2 stop openclaw
```

WSL Auto-Start at Windows Boot

To ensure WSL starts before Windows login (headless setups):

Enable Lingering for User Services

Inside WSL:

```
sudo loginctl enable-linger "\$(whoami)"
```

Create Windows Scheduled Task for WSL Boot

In PowerShell as Administrator:

```
# Get your WSL distribution name
\$distroName = (wsl --list --verbose | Select-String "Ubuntu|Debian" | ForEach-Object { \
$_.ToString().Trim().Split()[0] })[0]

# Create boot task
schtasks /create /tn "WSL Boot" /tr "wsl.exe -d \$distroName --exec /bin/true" /sc onstart /ru
SYSTEM
```

Verify Boot Chain

After a Windows reboot (before login):

```
# Check if OpenClaw service is enabled
systemctl --user is-enabled openclaw-gateway
```

```
# Check service status
systemctl --user status openclaw-gateway --no-pager

# View recent logs
journalctl --user -u openclaw-gateway --no-pager -l
```

Status Checking and Management

Essential commands for Windows (WSL2) deployment:

```
# Check gateway status
openclaw gateway status

# View live logs
openclaw logs --follow

# Check service health
systemctl --user status openclaw-gateway

# Restart if needed
systemctl --user restart openclaw-gateway

# Enable auto-start (if not done during onboarding)
openclaw gateway install
```

---

macOS Deployment

macOS deployments benefit from the native OpenClaw.app companion and robust launchd service management. The app handles permissions, manages the gateway lifecycle, and provides native integrations.

macOS App Installation

Download and install OpenClaw.app from the official website. The app provides:

- Native menu bar interface
- TCC permission management (notifications, screen recording, accessibility)
- Automatic gateway lifecycle management
- Canvas and system automation capabilities
- SSH tunnel management for remote deployments

Launchd Service Configuration

OpenClaw uses launchd for automatic startup. The service runs as a user-level LaunchAgent.

Service Installation

Install the gateway service:

```
# Install service (creates LaunchAgent plist)
openclaw gateway install

# Or use the integrated onboarding
openclaw onboard --install-daemon
```

Manual LaunchAgent Configuration

For custom configurations, create or edit the plist file:

```
# Service location
~/.openclaw/service/ai.openclaw.gateway.plist

# For custom profiles
~/.openclaw/service/ai.openclaw.PROFILE_NAME.plist
```

Complete plist template:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>ai.openclaw.gateway</string>

    <key>ProgramArguments</key>
    <array>
        <string>/opt/homebrew/bin/node</string>
        <string>/usr/local/lib/node_modules/openclaw/dist/index.js</string>
        <string>gateway</string>
        <string>--port</string>
        <string>18789</string>
        <string>--allow-unconfigured</string>
    </array>

    <key>WorkingDirectory</key>
    <string>/Users/YOUR_USERNAME</string>

    <key>StandardOutPath</key>
    <string>/Users/YOUR_USERNAME/.openclaw/logs/gateway.out.log</string>

    <key>StandardErrorPath</key>
    <string>/Users/YOUR_USERNAME/.openclaw/logs/gateway.err.log</string>

    <key>RunAtLoad</key>
    <true/>

    <key>KeepAlive</key>
    <dict>
        <key>SuccessfulExit</key>
        <false/>
        <key>Crashed</key>
        <true/>
    </dict>

    <key>ProcessType</key>
```

```
    <string>Background</string>

    <key>EnvironmentVariables</key>
    <dict>
        <key>NODE_ENV</key>
        <string>production</string>
        <key>OPENCLAW_STATE_DIR</key>
        <string>/Users/YOUR_USERNAME/.openclaw</string>
    </dict>

    <key>ThrottleInterval</key>
    <integer>60</integer>
</dict>
</plist>
```

Critical Path Updates: - Replace `YOUR_USERNAME` with your actual username - Update Node.js path if different (`which node`) - Update OpenClaw installation path (`npm root -g`)

launchctl Commands

Manage the service with launchctl:

```
# Load and start service
launchctl load ~/.openclaw/service/ai.openclaw.gateway.plist

# Start service (if already loaded)
launchctl start ai.openclaw.gateway

# Stop service
launchctl stop ai.openclaw.gateway

# Unload service (stops and removes)
launchctl unload ~/.openclaw/service/ai.openclaw.gateway.plist

# Force restart
launchctl kickstart -k gui/\$UID/ai.openclaw.gateway

# Check if service is loaded
launchctl list | grep openclaw

# View service status
launchctl print gui/\$UID/ai.openclaw.gateway
```

PM2 Alternative on macOS

For developers preferring PM2:

```
# Install PM2
npm install -g pm2

# Start with ecosystem file
pm2 start ecosystem.config.js

# Or direct command
pm2 start "openclaw gateway run" --name "openclaw-gateway"
```

```
# Enable startup
pm2 startup
pm2 save
```

PM2 ecosystem configuration (ecosystem.config.js):

```
module.exports = {
  apps: [{
    name: 'openclaw-gateway',
    script: 'openclaw',
    args: 'gateway run --port 18789',
    cwd: process.env.HOME,
    env: {
      NODE_ENV: 'production',
      OPENCLAW_STATE_DIR: `${process.env.HOME}/.openclaw`
    },
    error_file: `${process.env.HOME}/.openclaw/logs/pm2-error.log`,
    out_file: `${process.env.HOME}/.openclaw/logs/pm2-out.log`,
    log_file: `${process.env.HOME}/.openclaw/logs/pm2-combined.log`,
    time: true,
    restart_delay: 5000,
    max_restarts: 5,
    min_uptime: '10s'
  }]
};
```

macOS Startup Script Template

Create a robust startup script for complex deployments:

```
#!/bin/bash
# openclaw-startup.sh

set -euo pipefail

LOGFILE="\$HOME/.openclaw/logs/startup.log"
PIDFILE="\$HOME/.openclaw/service/startup.pid"
MAX_RETRIES=3
RETRY_DELAY=30

# Create logs directory
mkdir -p "\$(dirname "\$LOGFILE")"
mkdir -p "\$(dirname "\$PIDFILE")"

# Logging function
log() {
    echo "[\$(date '+%Y-%m-%d %H:%M:%S')] \$1" | tee -a "\$LOGFILE"
}

# Cleanup on exit
cleanup() {
    rm -f "\$PIDFILE"
}
trap cleanup EXIT
```

```
# Check if already running
if [[ -f "\$PIDFILE" ]]; then
    if ps -p "\$(cat "\$PIDFILE")" > /dev/null 2>&1; then
        log "Startup script already running (PID: \$(cat "\$PIDFILE"))"
        exit 0
    else
        rm -f "\$PIDFILE"
    fi
fi

# Create PID file
echo \$\$ > "\$PIDFILE"

log "Starting OpenClaw Gateway..."

for ((i=1; i≤MAX_RETRIES; i++)); do
    log "Attempt \$i of \$MAX_RETRIES"

    if openclaw gateway status --no-probe | grep -q "running"; then
        log "Gateway already running"
        exit 0
    fi

    if launchctl start ai.openclaw.gateway; then
        log "Gateway start command successful"
        sleep 10

        if openclaw gateway health --timeout 15000; then
            log "Gateway health check passed"
            exit 0
        else
            log "Gateway health check failed"
        fi
    else
        log "Gateway start command failed"
    fi

    if [[ \$i -lt \$MAX_RETRIES ]]; then
        log "Waiting \${RETRY_DELAY} seconds before retry..."
        sleep \$RETRY_DELAY
    fi
done

log "Failed to start gateway after \$MAX_RETRIES attempts"
exit 1
```

Make executable and test:

```
chmod +x ~/openclaw-startup.sh
~/openclaw-startup.sh
```

State Directory Considerations

Avoid cloud-synced directories for OpenClaw state:

```
# Good locations (local, fast)
export OPENCLAW_STATE_DIR="\$HOME/.openclaw"

# Avoid these (cloud-synced, slower)
# ~/Library/Mobile Documents/com~apple~CloudDocs/.openclaw
# ~/Library/CloudStorage/Dropbox/.openclaw
```

The `openclaw doctor` command will warn if it detects state in cloud-synced paths.

Linux Deployment

Linux provides the most straightforward deployment path with systemd service management. This section covers both VPS and desktop Linux deployments.

Systemd User Service (Recommended)

OpenClaw installs as a systemd user service by default, which runs when you're logged in and can survive logout with proper configuration.

Service Installation

```
# Install service automatically
openclaw onboard --install-daemon

# Or install service only
openclaw gateway install

# Manual configuration wizard
openclaw configure
```

Service File Location and Content

User service location:

```
~/.config/systemd/user/openclaw-gateway.service
# Or with custom profile:
~/.config/systemd/user/openclaw-gateway-PROFILE.service
```

Complete service file template:

```
[Unit]
Description=OpenClaw Gateway (v2026.2.26)
After=network-online.target
Wants=network-online.target
StartLimitBurst=5
StartLimitIntervalSec=30

[Service]
```

```
Type=simple
ExecStart=/usr/local/bin/openclaw gateway --port 18789 --allow-unconfigured
ExecReload=/bin/kill -HUP \$MAINPID
Restart=always
RestartSec=5
RestartPreventExitStatus=0

# Resource limits
LimitNOFILE=65536
MemoryMax=2G

# Security
NoNewPrivileges=yes
PrivateTmp=yes
ProtectHome=read-only
ProtectSystem=strict
ReadWritePaths=/home/YOUR_USERNAME/.openclaw

# Environment
Environment=NODE_ENV=production
Environment=OPENCLAW_STATE_DIR=/home/YOUR_USERNAME/.openclaw

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=openclaw-gateway

[Install]
WantedBy=default.target
```

Path Updates Required: - Replace `YOUR_USERNAME` with your actual username - Update `ExecStart` path if OpenClaw is installed elsewhere - Verify `ReadWritePaths` matches your state directory

Enable User Lingering (Critical for Headless)

For VPS and headless setups, enable user lingering so services start before login:

```
# Enable lingering for current user
sudo loginctl enable-linger "\$(whoami)"

# Verify lingering is enabled
loginctl show-user "\$(whoami)" | grep Linger
# Should show: Linger=yes
```

Systemctl Commands

Common systemd service management:

```
# Enable service (start on boot)
systemctl --user enable openclaw-gateway

# Start service
systemctl --user start openclaw-gateway

# Stop service
systemctl --user stop openclaw-gateway
```

```
# Restart service
systemctl --user restart openclaw-gateway

# Check status
systemctl --user status openclaw-gateway

# View logs
journalctl --user -u openclaw-gateway -f

# View recent logs (last 50 lines)
journalctl --user -u openclaw-gateway -n 50

# Check if service is enabled
systemctl --user is-enabled openclaw-gateway

# Check if service is running
systemctl --user is-active openclaw-gateway

# Reload service file after changes
systemctl --user daemon-reload
```

System Service (Multi-User Servers)

For shared servers where OpenClaw should run regardless of user sessions:

Create System Service File

Create `/etc/systemd/system/openclaw-gateway.service`:

```
[Unit]
Description=OpenClaw Gateway (System)
After=network-online.target
Wants=network-online.target
StartLimitBurst=3
StartLimitIntervalSec=60

[Service]
Type=simple
User=openclaw
Group=openclaw
ExecStart=/usr/local/bin/openclaw gateway --port 18789
Restart=always
RestartSec=10

# Working directory
WorkingDirectory=/home/openclaw

# Resource limits
LimitNOFILE=65536
MemoryMax=4G

# Security hardening
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectHome=yes
```

```
ProtectSystem=strict
ReadWritePaths=/home/openclaw/.openclaw

# Environment
Environment=NODE_ENV=production
Environment=OPENCLAW_STATE_DIR=/home/openclaw/.openclaw

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=openclaw-gateway

[Install]
WantedBy=multi-user.target
```

System Service Setup

```
# Create dedicated user
sudo useradd -r -s /bin/bash -d /home/openclaw -m openclaw

# Install OpenClaw as openclaw user
sudo -u openclaw npm install -g openclaw\@latest

# Create service file (content above)
sudo nano /etc/systemd/system/openclaw-gateway.service

# Reload systemd
sudo systemctl daemon-reload

# Enable and start service
sudo systemctl enable openclaw-gateway
sudo systemctl start openclaw-gateway

# Check status
sudo systemctl status openclaw-gateway
```

PM2 Alternative on Linux

For development or complex deployments:

```
# Install PM2 globally
npm install -g pm2

# Start OpenClaw
pm2 start openclaw -- gateway run --port 18789

# Enable startup (as root or with sudo)
pm2 startup systemd
# Follow the generated command

# Save configuration
pm2 save
```

Docker Deployment (Advanced)

For containerized deployments, use the official Docker setup:

Quick Docker Setup

```
# Clone repository (for docker-compose.yml)
git clone https://github.com/openclaw/openclaw.git
cd openclaw

# Run automated setup
./docker-setup.sh

# Or use pre-built image
export OPENCLAW_IMAGE="ghcr.io/openclaw/openclaw:latest"
./docker-setup.sh
```

Manual Docker Compose

Create `docker-compose.yml`:

```
version: '3.8'
services:
  openclaw-gateway:
    image: ghcr.io/openclaw/openclaw:latest
    ports:
      - "18789:18789"
    volumes:
      - ~/.openclaw:/home/node/.openclaw
      - ~/.openclaw/workspace:/home/node/.openclaw/workspace
    environment:
      - NODE_ENV=production
      - OPENCLAW_GATEWAY_BIND=lan
      - OPENCLAW_GATEWAY_PORT=18789
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:18789/healthz"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

Commands:

```
# Start services
docker-compose up -d

# View logs
docker-compose logs -f openclaw-gateway

# Restart service
docker-compose restart openclaw-gateway

# Update container
docker-compose pull
docker-compose up -d
```

Log Management

Effective log management prevents disk space issues, provides debugging information, and maintains system performance.

Log Locations by Operating System

Windows (WSL2)

```
# Gateway file logs
/tmp/openclaw/openclaw-YYYY-MM-DD.log

# systemd service logs
journalctl --user -u openclaw-gateway

# PM2 logs (if using PM2)
~/.pm2/logs/
```

macOS

```
# Gateway file logs
/tmp/openclaw/openclaw-YYYY-MM-DD.log

# launchd logs
~/Library/Logs/ai.openclaw.gateway/

# System Console app logs
# Search for "openclaw" in Console.app
```

Linux

```
# Gateway file logs
/tmp/openclaw/openclaw-YYYY-MM-DD.log

# systemd journal
journalctl --user -u openclaw-gateway
# or for system service:
journalctl -u openclaw-gateway
```

Log Configuration

Configure logging in `~/.openclaw/openclaw.json`:

```
{
  "logging": {
```

```
    "level": "info",
    "file": "/tmp/openclaw/openclaw-%DATE%.log",
    "consoleLevel": "info",
    "consoleStyle": "pretty",
    "redactSensitive": "tools"
  }
}
```

Log Levels

- `error` - Only errors and critical issues
- `warn` - Warnings and errors
- `info` - General information (recommended)
- `debug` - Detailed debugging information
- `trace` - Very verbose output (development only)

Log Rotation and Management

Automatic Log Rotation (Linux/macOS)

Create `/etc/logrotate.d/openclaw`:

```
/tmp/openclaw/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    copytruncate
    su root root
}
```

Or user-level with `logrotate` in cron:

```
# Add to user crontab
crontab -e

# Add this line (rotate daily at 2 AM)
0 2 * * * /usr/sbin/logrotate /home/\$(whoami)/.openclaw/logrotate.conf --state /home/\
$(whoami)/.openclaw/logrotate.state
```

User logrotate config (`~/.openclaw/logrotate.conf`):

```
/tmp/openclaw/*.log {
    daily
    missingok
    rotate 7
    compress
    delaycompress
```

```
    notifempty
    copytruncate
}
```

Manual Log Cleanup

```
# Find old log files (older than 7 days)
find /tmp/openclaw -name "*.log" -mtime +7 -ls

# Remove old log files
find /tmp/openclaw -name "*.log" -mtime +7 -delete

# Compress current logs
gzip /tmp/openclaw/openclaw-*.log

# View compressed logs
zcat /tmp/openclaw/openclaw-2026-03-01.log.gz | grep ERROR
```

Reading and Filtering Logs

OpenClaw Log Commands

```
# View live logs
openclaw logs --follow

# View recent logs
openclaw logs --limit 100

# View logs in local time
openclaw logs --local-time

# JSON output for parsing
openclaw logs --json --limit 50
```

System Log Commands

systemd (Linux/WSL2)

```
# Follow logs in real-time
journalctl --user -u openclaw-gateway -f

# View logs from last boot
journalctl --user -u openclaw-gateway -b

# View logs from specific time period
journalctl --user -u openclaw-gateway --since "2026-03-12 10:00:00"

# View only errors
journalctl --user -u openclaw-gateway -p err
```

```
# Export logs to file
journalctl --user -u openclaw-gateway --since today > openclaw-\$(date +%Y%m%d).log
```

File-based Filtering

```
# View latest log file
tail -f /tmp/openclaw/openclaw-\$(date +%Y-%m-%d).log

# Search for errors
grep -i error /tmp/openclaw/openclaw-*.log

# Search for specific patterns
grep "channel.*telegram" /tmp/openclaw/openclaw-*.log

# Count log levels
grep -c "\"level\":" /tmp/openclaw/openclaw-*.log | head -10
```

Log Analysis and Monitoring

Key Patterns to Monitor

Create monitoring scripts that alert on:

```
#!/bin/bash
# openclaw-log-monitor.sh

LOGFILE="/tmp/openclaw/openclaw-\$(date +%Y-%m-%d).log"
EMAIL="YOUR_EMAIL_PLACEHOLDER"

# Check for errors in last hour
ERRORS=\$(journalctl --user -u openclaw-gateway --since "1 hour ago" -p err --no-pager | wc -l)

if [ "\$ERRORS" -gt 0 ]; then
    echo "OpenClaw errors detected: \$ERRORS" | mail -s "OpenClaw Alert" "\$EMAIL"
fi

# Check if gateway is responsive
if ! openclaw gateway health --timeout 10000 > /dev/null 2>&1; then
    echo "OpenClaw gateway health check failed" | mail -s "OpenClaw Down" "\$EMAIL"
fi

# Check disk space
DISK_USAGE=\$(df /tmp | tail -1 | awk '{print \$5}' | sed 's/%//')
if [ "\$DISK_USAGE" -gt 90 ]; then
    echo "High disk usage: \${DISK_USAGE}%" | mail -s "OpenClaw Disk Alert" "\$EMAIL"
fi
```

Make executable and add to cron:

```
chmod +x ~/openclaw-log-monitor.sh

# Add to crontab (run every 15 minutes)
```

```
crontab -e
# Add: */15 * * * * /home/\$(whoami)/openclaw-log-monitor.sh
```

Updating OpenClaw Safely

Regular updates bring new features, bug fixes, and security improvements. A safe update process ensures continuity of service.

Check Current Version

Before updating, document your current state:

```
# Check installed version
openclaw --version

# Check gateway status
openclaw gateway status

# Check service health
openclaw health

# List installed channels
openclaw channels list
```

Save this information in case rollback is needed.

Pre-Update Backup

Always backup before updating:

```
#!/bin/bash
# backup-before-update.sh

BACKUP_DIR="\$HOME/.openclaw-backup-\$(date +%Y%m%d-%H%M%S)"
mkdir -p "\$BACKUP_DIR"

# Backup configuration
cp -r ~/.openclaw/openclaw.json "\$BACKUP_DIR/" 2>/dev/null || true
cp -r ~/.openclaw/credentials/ "\$BACKUP_DIR/" 2>/dev/null || true

# Backup workspace
cp -r ~/.openclaw/workspace/ "\$BACKUP_DIR/" 2>/dev/null || true

# Backup service files (Linux/WSL)
cp -r ~/.config/systemd/user/openclaw-*.service "\$BACKUP_DIR/" 2>/dev/null || true

# Backup launchd files (macOS)
cp -r ~/Library/LaunchAgents/com.openclaw.* "\$BACKUP_DIR/" 2>/dev/null || true

# Create backup manifest
```

```
{
    echo "Backup created: \$(date)"
    echo "OpenClaw version: \$(openclaw --version)"
    echo "Gateway status: \$(openclaw gateway status --json)"
} > "\$BACKUP_DIR/manifest.txt"

echo "Backup created: \$BACKUP_DIR"
```

Recommended Update Method

Use the official installer for safest updates:

```
# Re-run installer (detects existing install)
curl -fsSL https://openclaw.ai/install.sh | bash
```

For source installations:

```
# Re-run installer for source
curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method git --no-onboard
```

Alternative Update Methods

NPM Update (Global Install)

```
# Update to latest
npm install -g openclaw\@latest

# Or update to specific version
npm install -g openclaw\@2026.2.26

# Verify update
openclaw --version
```

Built-in Update Command

For source installations:

```
# Update with automatic restart
openclaw update

# Update without restart
openclaw update --no-restart

# Check what would be updated
openclaw update --dry-run
```

Post-Update Verification

After any update method:

```
# Run doctor to check for issues
openclaw doctor

# Restart gateway service
openclaw gateway restart

# Verify health
openclaw health

# Test key functionality
openclaw channels status
openclaw devices list
```

Update Rollback Procedure

If the update causes issues:

Version Rollback (NPM)

```
# Install previous version
npm install -g openclaw\@PREVIOUS_VERSION

# Example:
npm install -g openclaw\@2026.2.22
```

Source Rollback (Git)

```
# Find previous working commit
git log --oneline -n 10

# Rollback to specific commit
git checkout COMMIT_HASH

# Or rollback by date
git checkout "\$(git rev-list -n 1 --before="2026-03-01" main)"

# Rebuild
npm install
npm run build
```

Configuration Rollback

```
# Restore from backup
BACKUP_DIR="\$HOME/.openclaw-backup-20260312-143000"
cp -r "\$BACKUP_DIR/openclaw.json" ~/.openclaw/
cp -r "\$BACKUP_DIR/credentials/" ~/.openclaw/
```

```
# Restart gateway
openclaw gateway restart
```

Automated Updates (Optional)

Enable automatic updates in configuration:

```
{
  "update": {
    "channel": "stable",
    "auto": {
      "enabled": true,
      "stableDelayHours": 6,
      "stableJitterHours": 12
    }
  }
}
```

This automatically applies stable updates with built-in delay and jitter to spread rollouts.

Backup Strategy

A comprehensive backup strategy protects against data loss, corruption, and enables quick recovery.

What to Back Up

Essential Files

```
# Configuration
~/.openclaw/openclaw.json

# Credentials (API keys, tokens)
~/.openclaw/credentials/

# Environment variables
~/.openclaw/.env

# Agent workspace and memory
~/.openclaw/workspace/
```

Optional Files

```
# Session history
~/.openclaw/agents/*/sessions/

# Logs (if needed for audit)
```

```
/tmp/openclaw/openclaw-*.log

# Service configurations
~/.config/systemd/user/openclaw-*.service    # Linux
~/Library/LaunchAgents/ai.openclaw.*         # macOS
```

Git-Based Backup (Recommended)

Git provides versioned backups with easy recovery:

Initial Git Setup

```
cd ~/.openclaw/workspace

# Initialize repository
git init

# Create .gitignore
cat > .gitignore << 'EOF'
# Exclude sensitive files
credentials/
.env
*.log
.DS_Store
node_modules/
*.tmp

# Include workspace and config
!AGENTS.md
!SOUL.md
!USER.md
!TOOLS.md
!IDENTITY.md
!MEMORY.md
!memory/
!skills/
EOF

# Add initial files
git add .
git commit -m "Initial OpenClaw workspace backup"
```

Automated Git Backup Script

```
#!/bin/bash
# git-backup.sh

set -euo pipefail

WORKSPACE_DIR="\$HOME/.openclaw/workspace"
CONFIG_FILE="\$HOME/.openclaw/openclaw.json"
BACKUP_BRANCH="backups/\$(date +%Y-%m)"

cd "\$WORKSPACE_DIR"
```

```
# Ensure git is initialized
if [ ! -d .git ]; then
    git init
    echo "Git repository initialized"
fi

# Copy current config to workspace (tracked)
cp "\$CONFIG_FILE" "./config-backup.json" 2>/dev/null || true

# Add all changes
git add -A

# Check if there are changes to commit
if git diff --cached --quiet; then
    echo "No changes to backup"
    exit 0
fi

# Commit changes
COMMIT_MSG="Automated backup: \$(date '+%Y-%m-%d %H:%M:%S')"
git commit -m "\$COMMIT_MSG"

# Create/switch to backup branch
git checkout -B "\$BACKUP_BRANCH" 2>/dev/null || git checkout "\$BACKUP_BRANCH"

# Push to remote (if configured)
if git remote get-url origin >/dev/null 2>&1; then
    git push -u origin "\$BACKUP_BRANCH"
    echo "Backup pushed to remote"
else
    echo "Backup committed locally (no remote configured)"
fi

echo "Backup completed: \$COMMIT_MSG"
```

Schedule Automated Backups

```
# Make script executable
chmod +x ~/.openclaw/git-backup.sh

# Add to crontab (daily at 3 AM)
crontab -e
# Add: 0 3 * * * /home/\$(whoami)/.openclaw/git-backup.sh
```

Cloud Backup Options

GitHub Private Repository

```
# Create private repo on GitHub, then:
cd ~/.openclaw/workspace
git remote add origin git\@github.com:YOUR_USERNAME_PLACEHOLDER/openclaw-backup.git
git push -u origin main
```

Cloud Storage Sync

```
#!/bin/bash
# cloud-backup.sh

BACKUP_DIR="\$HOME/.openclaw-backup-\$(date +%Y%m%d)"
CLOUD_DIR="/path/to/cloud/storage/openclaw-backups"

# Create timestamped backup
mkdir -p "\$BACKUP_DIR"

# Copy essential files
cp -r ~/.openclaw/workspace/ "\$BACKUP_DIR/"
cp ~/.openclaw/openclaw.json "\$BACKUP_DIR/"

# Encrypt sensitive files (optional)
if command -v gpg >/dev/null; then
    tar -czf - ~/.openclaw/credentials/ | gpg --cipher-algo AES256 --compress-algo 1 --symmetric --
output "\$BACKUP_DIR/credentials.tar.gz.gpg"
fi

# Sync to cloud storage
rsync -av "\$BACKUP_DIR/" "\$CLOUD_DIR/"

echo "Backup completed to \$CLOUD_DIR"
```

Restoration Procedures

Git Restore

```
# View available backups
cd ~/.openclaw/workspace
git log --oneline

# Restore to specific commit
git checkout COMMIT_HASH

# Or restore specific files
git checkout HEAD~1 -- MEMORY.md

# Restore from backup branch
git checkout backups/2026-03 -- .
```

Full System Restore

```
#!/bin/bash
# restore-backup.sh

BACKUP_DIR="\$1"
if [ -z "\$BACKUP_DIR" ]; then
    echo "Usage: \$0 <backup_directory>"
    exit 1
fi
```

```bash
# Stop gateway
openclaw gateway stop

# Create restoration backup of current state
RESTORE_BACKUP="\$HOME/.openclaw-before-restore-\$(date +%Y%m%d-%H%M%S)"
mkdir -p "\$RESTORE_BACKUP"
cp -r ~/.openclaw/ "\$RESTORE_BACKUP/" 2>/dev/null || true

# Restore files
echo "Restoring from \$BACKUP_DIR"
cp -r "\$BACKUP_DIR/workspace/" ~/.openclaw/
cp "\$BACKUP_DIR/openclaw.json" ~/.openclaw/

# Restore credentials if available
if [ -f "\$BACKUP_DIR/credentials.tar.gz.gpg" ]; then
    echo "Decrypting credentials..."
    gpg --decrypt "\$BACKUP_DIR/credentials.tar.gz.gpg" | tar -xzf - -C ~/.openclaw/
fi

# Restart gateway
openclaw gateway start

echo "Restoration completed from \$BACKUP_DIR"
echo "Previous state backed up to \$RESTORE_BACKUP"
```

Backup Validation

Regularly test backup integrity:

```bash
#!/bin/bash
# validate-backup.sh

BACKUP_DIR="\$1"

echo "Validating backup: \$BACKUP_DIR"

# Check essential files exist
for file in "workspace/AGENTS.md" "workspace/SOUL.md" "openclaw.json"; do
    if [ ! -f "\$BACKUP_DIR/\$file" ]; then
        echo "ERROR: Missing \$file"
        exit 1
    fi
done

# Validate JSON configuration
if ! python3 -m json.tool "\$BACKUP_DIR/openclaw.json" >/dev/null; then
    echo "ERROR: Invalid JSON in openclaw.json"
    exit 1
fi

# Check workspace integrity
if [ ! -d "\$BACKUP_DIR/workspace" ]; then
    echo "ERROR: Missing workspace directory"
    exit 1
fi

echo "Backup validation successful"
```

Resource Monitoring

Monitoring resource usage prevents performance issues and guides scaling decisions.

CPU and Memory Expectations

Typical Resource Usage

Idle State: - CPU: 1-5% - Memory: 150-300 MB - File Descriptors: 50-100

Active Use: - CPU: 10-30% - Memory: 300-800 MB - File Descriptors: 100-500

Heavy Load: - CPU: 30-70% - Memory: 800MB-2GB - File Descriptors: 200-1000

Resource Monitoring Script

```
#!/bin/bash
# monitor-resources.sh

# Get OpenClaw process info
OPENCLAW_PID=\$(pgrep -f "openclaw.*gateway" | head -1)

if [ -z "\$OPENCLAW_PID" ]; then
    echo "OpenClaw gateway process not found"
    exit 1
fi

# CPU and Memory usage
echo "=== OpenClaw Resource Usage ==="
ps -p "\$OPENCLAW_PID" -o pid,ppid,pcpu,pmem,vsz,rss,etime,cmd

# Memory details (Linux)
if [ -f "/proc/\$OPENCLAW_PID/status" ]; then
    echo
    echo "=== Memory Details ==="
    grep -E "(VmPeak|VmSize|VmRSS|VmSwap)" "/proc/\$OPENCLAW_PID/status"
fi

# File descriptors (Linux)
if [ -d "/proc/\$OPENCLAW_PID/fd" ]; then
    FD_COUNT=\$(ls /proc/\$OPENCLAW_PID/fd | wc -l)
    echo
    echo "File Descriptors: \$FD_COUNT"
fi

# Network connections
echo
echo "=== Network Connections ==="
netstat -tulpn | grep "\$OPENCLAW_PID" | head -10

# Disk usage
echo
echo "=== Disk Usage ==="
du -sh ~/.openclaw/ 2>/dev/null || echo "State directory not accessible"
df -h /tmp | grep -v Filesystem
```

Performance Optimization

Memory Management

Monitor for memory leaks:

```
# Check memory growth over time
while true; do
    ps -p \$(pgrep -f "openclaw.*gateway") -o pid,pmem,rss,etime | tail -1
    sleep 300  # Check every 5 minutes
done
```

Set memory limits:

```
# systemd service limits
echo "MemoryMax=2G" | sudo tee -a ~/.config/systemd/user/openclaw-gateway.service
systemctl --user daemon-reload
systemctl --user restart openclaw-gateway
```

CPU Optimization

Monitor CPU usage patterns:

```
# Monitor CPU usage in real-time
top -p \$(pgrep -f "openclaw.*gateway")

# Set CPU limits (systemd)
echo "CPUQuota=50%" | sudo tee -a ~/.config/systemd/user/openclaw-gateway.service
```

Disk Space Management

Monitor Disk Usage

```
#!/bin/bash
# disk-monitor.sh

THRESHOLD=80
EMAIL="YOUR_EMAIL_PLACEHOLDER"

# Check main disk usage
USAGE=\$(df / | tail -1 | awk '{print \$5}' | sed 's/%//')
if [ "\$USAGE" -gt "\$THRESHOLD" ]; then
    echo "Root filesystem usage: \${USAGE}%" | mail -s "Disk Space Alert" "\$EMAIL"
fi

# Check OpenClaw specific usage
echo "═══ OpenClaw Disk Usage ═══"
du -sh ~/.openclaw/ 2>/dev/null
du -sh /tmp/openclaw/ 2>/dev/null

# Check for large files
```

```
echo "═══ Large Files (>50MB) ═══"
find ~/.openclaw/ -size +50M -ls 2>/dev/null
find /tmp/openclaw/ -size +50M -ls 2>/dev/null

# Session cleanup recommendations
SESSION_COUNT=\$(find ~/.openclaw/agents/*/sessions/ -name "*.json" 2>/dev/null | wc -l)
echo "Session files: \$SESSION_COUNT"

if [ "\$SESSION_COUNT" -gt 1000 ]; then
    echo "Consider cleaning old sessions:"
    echo "find ~/.openclaw/agents/*/sessions/ -name '*.json' -mtime +30 -delete"
fi
```

Automated Cleanup

```
#!/bin/bash
# cleanup-openclaw.sh

echo "Running OpenClaw cleanup..."

# Clean old logs (older than 14 days)
find /tmp/openclaw/ -name "*.log" -mtime +14 -delete
echo "Cleaned old logs"

# Clean old sessions (older than 60 days)
find ~/.openclaw/agents/*/sessions/ -name "*.json" -mtime +60 -delete
echo "Cleaned old sessions"

# Clean temporary files
find ~/.openclaw/tmp/ -type f -mtime +7 -delete 2>/dev/null || true
echo "Cleaned temporary files"

# Compact git repositories (if using git backup)
if [ -d ~/.openclaw/workspace/.git ]; then
    cd ~/.openclaw/workspace
    git gc --prune=now
    echo "Compacted git repository"
fi

echo "Cleanup completed"
```

Network Monitoring

Connection Monitoring

```
#!/bin/bash
# network-monitor.sh

OPENCLAW_PID=\$(pgrep -f "openclaw.*gateway" | head -1)

echo "═══ Active Connections ═══"
netstat -tulpn | grep "\$OPENCLAW_PID"
```

```
echo
echo "═══ Connection Counts ═══"
netstat -an | grep :18789 | wc -l

# Monitor bandwidth (if available)
if command -v iftop >/dev/null; then
    echo "Use 'iftop' to monitor real-time bandwidth"
fi

# Check gateway health
echo
echo "═══ Gateway Health ═══"
openclaw gateway health --timeout 5000
```

Alert Configuration

Create alerts for resource thresholds:

```
#!/bin/bash
# resource-alerts.sh

EMAIL="YOUR_EMAIL_PLACEHOLDER"
LOG_FILE="\$HOME/.openclaw/logs/resource-alerts.log"

log_alert() {
    echo "[\$(date)] \$1" >> "\$LOG_FILE"
    echo "\$1" | mail -s "OpenClaw Alert" "\$EMAIL"
}

# CPU check
CPU_USAGE=\$(ps -p \$(pgrep -f "openclaw.*gateway") -o pcpu= | awk '{print int(\$1)}')
if [ "\$CPU_USAGE" -gt 80 ]; then
    log_alert "High CPU usage: \${CPU_USAGE}%"
fi

# Memory check
MEM_MB=\$(ps -p \$(pgrep -f "openclaw.*gateway") -o rss= | awk '{print int(\$1/1024)}')
if [ "\$MEM_MB" -gt 1500 ]; then
    log_alert "High memory usage: \${MEM_MB}MB"
fi

# Connection check
CONN_COUNT=\$(netstat -an | grep :18789 | wc -l)
if [ "\$CONN_COUNT" -gt 50 ]; then
    log_alert "High connection count: \$CONN_COUNT"
fi
```

Troubleshooting

When your 24/7 deployment encounters issues, systematic troubleshooting gets you back online quickly.

Service Won't Start

Check Service Status

```
# Linux/WSL2
systemctl --user status openclaw-gateway
journalctl --user -u openclaw-gateway -n 50

# macOS
launchctl list | grep openclaw
launchctl print gui/\$UID/ai.openclaw.gateway
```

Common Issues and Solutions

Permission Denied:

```
# Check file permissions
ls -la ~/.config/systemd/user/openclaw-gateway.service

# Fix permissions
chmod 644 ~/.config/systemd/user/openclaw-gateway.service

# Reload and restart
systemctl --user daemon-reload
systemctl --user restart openclaw-gateway
```

Port Already in Use:

```
# Find process using port 18789
lsof -i :18789
netstat -tulpn | grep 18789

# Kill conflicting process
kill \$(lsof -ti :18789)

# Or change port in configuration
openclaw config set gateway.port 18790
```

Node.js Not Found:

```
# Check Node.js installation
which node
node --version

# Update service file with correct path
systemctl --user edit openclaw-gateway
# Add:
# [Service]
# ExecStart=/full/path/to/node /path/to/openclaw gateway
```

Gateway Running But Unreachable

Network Diagnostics

```
# Test local connection
curl -f http://127.0.0.1:18789/healthz

# Check binding configuration
openclaw config get gateway.bind

# Test with explicit URL
openclaw gateway health --url ws://127.0.0.1:18789

# Check firewall (Linux)
sudo ufw status
sudo iptables -L | grep 18789
```

Authentication Issues

```
# Check token configuration
openclaw gateway probe

# Reset token
openclaw config set gateway.token "\$(openssl rand -hex 32)"
openclaw gateway restart

# Test with new token
openclaw dashboard --no-open
```

Performance Issues

High CPU Usage

```
# Identify CPU-intensive processes
top -p \$(pgrep -f openclaw)

# Check for infinite loops in logs
journalctl --user -u openclaw-gateway | grep -C5 "CPU\|loop\|stuck"

# Restart with debug logging
openclaw config set logging.level debug
openclaw gateway restart

# Monitor specific operations
openclaw logs --follow | grep -E "(exec|browser|heavy)"
```

Memory Leaks

```
# Monitor memory growth
watch 'ps -p \$(pgrep -f openclaw) -o pid,pmem,rss,etime'

# Check for session buildup
find ~/.openclaw/agents/*/sessions/ -name "*.json" | wc -l

# Clear old sessions
find ~/.openclaw/agents/*/sessions/ -name "*.json" -mtime +7 -delete

# Restart service
systemctl --user restart openclaw-gateway
```

Disk Space Issues

```
# Check disk usage
df -h
du -sh ~/.openclaw/ /tmp/openclaw/

# Clean log files
find /tmp/openclaw/ -name "*.log" -mtime +7 -delete

# Check for large session files
find ~/.openclaw/ -size +10M -ls

# Enable log rotation
echo "Setting up log rotation..."
# Use logrotate configuration from earlier section
```

Connection Drops

Unstable Network

```
# Monitor connection stability
while true; do
    if openclaw gateway health --timeout 5000 >/dev/null; then
        echo "\$(date): OK"
    else
        echo "\$(date): FAILED"
    fi
    sleep 30
done

# Check for network interface issues
ip addr show
ping -c 4 8.8.8.8

# Restart networking (if needed)
sudo systemctl restart networking
```

Channel Connection Issues

```
# Check channel status
openclaw channels status

# Restart specific channel
openclaw channels restart telegram

# View channel logs
openclaw logs --follow | grep "channel.*telegram"

# Re-authenticate if needed
openclaw channels login
```

Log Analysis for Issues

Common Error Patterns

```
# Search for critical errors
journalctl --user -u openclaw-gateway | grep -i "error\|exception\|failed"

# Check for memory issues
journalctl --user -u openclaw-gateway | grep -i "memory\|oom\|heap"

# Look for network problems
journalctl --user -u openclaw-gateway | grep -i "network\|timeout\|refused"

# Find authentication issues
journalctl --user -u openclaw-gateway | grep -i "auth\|unauthorized\|token"
```

Debug Mode Setup

```
# Enable verbose logging
openclaw config set logging.level debug
openclaw gateway restart

# Monitor with debug output
openclaw gateway run --verbose

# Capture detailed logs
openclaw logs --follow > debug-\$(date +%Y%m%d-%H%M).log
```

Recovery Procedures

Complete Service Reset

```
#!/bin/bash
# reset-openclaw-service.sh
```

```
echo "Stopping OpenClaw gateway..."
openclaw gateway stop || systemctl --user stop openclaw-gateway

echo "Backing up current configuration..."
BACKUP_DIR="\$HOME/.openclaw-reset-backup-\$(date +%Y%m%d-%H%M%S)"
mkdir -p "\$BACKUP_DIR"
cp -r ~/.openclaw/ "\$BACKUP_DIR/"

echo "Cleaning temporary files..."
rm -rf /tmp/openclaw/*

echo "Reinstalling service..."
openclaw gateway install --force

echo "Starting service..."
openclaw gateway start

echo "Verifying health..."
sleep 5
openclaw gateway health

echo "Reset completed. Backup at: \$BACKUP_DIR"
```

Configuration Validation

```
#!/bin/bash
# validate-config.sh

CONFIG_FILE="\$HOME/.openclaw/openclaw.json"

echo "Validating OpenClaw configuration..."

# Check JSON syntax
if ! python3 -m json.tool "\$CONFIG_FILE" >/dev/null 2>&1; then
    echo "ERROR: Invalid JSON in \$CONFIG_FILE"
    exit 1
fi

# Check required fields
REQUIRED_FIELDS=("gateway" "channels")
for field in "\${REQUIRED_FIELDS[@]}"; do
    if ! grep -q "\"\$field\"" "\$CONFIG_FILE"; then
        echo "WARNING: Missing \$field configuration"
    fi
done

# Check port conflicts
PORT=\$(openclaw config get gateway.port 2>/dev/null || echo "18789")
if lsof -i ":\$PORT" >/dev/null 2>&1; then
    PROCESS=\$(lsof -ti ":\$PORT")
    if [ "\$(pgrep -f openclaw)" != "\$PROCESS" ]; then
        echo "WARNING: Port \$PORT in use by process \$PROCESS"
    fi
fi

echo "Configuration validation completed"
```

Emergency Procedures

Gateway Down Alert

```bash
#!/bin/bash
# emergency-restart.sh

EMAIL="YOUR_EMAIL_PLACEHOLDER"
MAX_ATTEMPTS=3
LOG_FILE="\$HOME/.openclaw/logs/emergency.log"

log_emergency() {
    echo "[\$(date)] EMERGENCY: \$1" | tee -a "\$LOG_FILE"
    echo "\$1" | mail -s "OpenClaw EMERGENCY" "\$EMAIL"
}

for ((i=1; i<=MAX_ATTEMPTS; i++)); do
    if openclaw gateway health --timeout 10000 >/dev/null 2>&1; then
        echo "Gateway is healthy"
        exit 0
    fi

    log_emergency "Attempt \$i: Gateway unresponsive, restarting..."

    # Force restart
    openclaw gateway stop
    sleep 5
    openclaw gateway start
    sleep 10
done

log_emergency "CRITICAL: Gateway failed to restart after \$MAX_ATTEMPTS attempts"
```

Diagnostic Data Collection

```bash
#!/bin/bash
# collect-diagnostics.sh

DIAG_DIR="\$HOME/openclaw-diagnostics-\$(date +%Y%m%d-%H%M%S)"
mkdir -p "\$DIAG_DIR"

echo "Collecting diagnostic information..."

# System information
{
    echo "=== System Info ==="
    uname -a
    echo
    echo "=== OpenClaw Version ==="
    openclaw --version
    echo
    echo "=== Node.js Version ==="
    node --version
    echo
    echo "=== Disk Usage ==="
    df -h
} > "\$DIAG_DIR/system.txt"
```

```
# Service status
{
    echo "═══ Service Status ═══"
    systemctl --user status openclaw-gateway
    echo
    echo "═══ Process Info ═══"
    ps aux | grep openclaw
} > "\$DIAG_DIR/service.txt"

# Network information
{
    echo "═══ Network ═══"
    netstat -tulpn | grep openclaw
    echo
    echo "═══ Gateway Health ═══"
    openclaw gateway health --timeout 5000
} > "\$DIAG_DIR/network.txt"

# Configuration (redacted)
cp ~/.openclaw/openclaw.json "\$DIAG_DIR/config.json" 2>/dev/null || echo "{}" > "\$DIAG_DIR/
config.json"

# Recent logs
journalctl --user -u openclaw-gateway --since "24 hours ago" > "\$DIAG_DIR/logs.txt" 2>/dev/null

# File permissions
ls -la ~/.openclaw/ > "\$DIAG_DIR/permissions.txt" 2>/dev/null

echo "Diagnostics collected in: \$DIAG_DIR"
echo "Archive with: tar -czf openclaw-diag-\$(date +%Y%m%d).tar.gz \$DIAG_DIR"
```

Conclusion

A properly deployed OpenClaw system operates like invisible infrastructure—always available, automatically recovering, and continuously serving your automation needs. The configurations and procedures in this chapter transform your installation from a manual tool into production-grade software.

Key Takeaways

1. **Choose the right deployment method** for your operating system and requirements
2. **Implement proper service management** with automatic startup and restart capabilities
3. **Configure comprehensive logging** with rotation and monitoring
4. **Establish safe update procedures** with backup and rollback capabilities
5. **Monitor resource usage** to prevent performance issues
6. **Prepare troubleshooting procedures** for quick issue resolution

Next Steps

With persistent deployment configured:

- Set up automated monitoring and alerting

- Create comprehensive backup routines
- Test disaster recovery procedures
- Document your specific configuration details
- Train team members on operational procedures

Your OpenClaw agent is now ready for production workloads. In the next chapter, we'll explore advanced automation patterns that leverage this rock-solid foundation to create sophisticated business processes.

Remember: The most sophisticated automation is worthless if it's not running. This chapter ensures yours never stops.

CHAPTER 10

Troubleshooting & Appendix

Reference Materials

Chapter 10 — Troubleshooting & Uninstall

Every problem has a solution. This chapter will help you diagnose issues quickly and get back to building your AI-powered workflows.

When OpenClaw isn't working as expected, don't panic. The platform includes powerful diagnostic tools and clear troubleshooting paths. Most issues fall into predictable categories with well-tested solutions.

Quick Diagnostic Commands

Before diving into specific problems, run these diagnostic commands to get an overview of your system's health:

The 60-Second Health Check

Run these commands in order to quickly assess your OpenClaw installation:

```
openclaw status
openclaw gateway status
openclaw channels status --probe
openclaw models status
openclaw doctor
openclaw logs --follow
```

What each command tells you: - `openclaw status` — Overall system health and basic configuration - `openclaw gateway status` — Gateway daemon status and connectivity - `openclaw channels status --probe` — Communication channel health - `openclaw models status` — AI model provider authentication - `openclaw doctor` — Automated health checks and repairs - `openclaw logs --follow` — Real-time activity monitoring

Interpreting Healthy Output

When everything is working properly, you should see: - `Runtime: running` and `RPC probe: ok` from gateway status - Your configured channels showing `connected` or `ready` - Model providers showing valid authentication - No blocking errors from the doctor command

If any of these show problems, the sections below will help you fix them.

Common Issues by Category

Installation Issues

Node.js version too old or not found

Symptoms: Installation fails with Node.js version errors or `node: command not found`

Solution: - Install Node.js 22 or higher from nodejs.org - Verify installation: `node --version` - Restart your terminal after installation - If using Windows, ensure Node.js is in your PATH

npm permission errors (EACCES)

Symptoms: Installation fails with permission errors during `npm install -g openclaw`

Windows solution:

```
npm config set prefix "%USERPROFILE%\npm"
# Add %USERPROFILE%\npm to your PATH environment variable
```

macOS/Linux solution:

```
# Option 1: Use a Node version manager (recommended)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
nvm install node
nvm use node

# Option 2: Fix npm permissions
mkdir ~/.npm-global
npm config set prefix '~/.npm-global'
# Add ~/.npm-global/bin to your PATH in ~/.bashrc or ~/.zshrc
export PATH=~/.npm-global/bin:\$PATH
```

OpenClaw command not found after install

Symptoms: Installation succeeds but `openclaw` command isn't recognized

Solution: - Check if npm global bin is in your PATH: `npm config get prefix` - Add the npm bin directory to your PATH environment variable - Restart your terminal - Windows: Usually `%USERPROFILE%\AppData\Roaming\npm` - macOS/Linux: Usually `/usr/local/bin` or `~/.npm-global/bin`

Ollama won't start or model pull fails

Symptoms: Local model setup fails, Ollama connection errors

Solution:

```bash
# Check if Ollama is installed
ollama --version

# Start Ollama service
ollama serve

# Try pulling a model
ollama pull qwen2.5:14b
```

If Ollama isn't installed: - Windows/macOS: Download from ollama.com - Linux: `curl -fsSL https://ollama.com/install.sh | sh`

Connection Issues

Bot not responding in Telegram/Discord

Symptoms: Messages to the bot receive no response

Diagnostic steps:

```bash
openclaw channels status --probe
openclaw pairing list --channel telegram
openclaw logs --follow
```

Common causes and fixes: - **Pairing required:** New contacts need approval `bash  openclaw pairing approve telegram <code>` - **Mention required in groups:** Most bots require @mentions in group chats - **Wrong bot token:** Check your bot token configuration - **Network issues:** Ensure internet connectivity

"Gateway not running" error

Symptoms: Commands fail with gateway connection errors

Solution:

```bash
# Check gateway status
openclaw gateway status

# Start gateway if stopped
openclaw gateway start

# If service isn't installed, run in foreground
openclaw gateway run

# If port conflict, try different port
openclaw gateway run --port 18790
```

Timeout errors

Symptoms: Operations hang or timeout

Solution: - Check internet connection - Verify model provider API keys - Try a different model: `/model haiku` (faster) - Check model provider status pages - Increase timeout in config if needed

Channel not configured properly

Symptoms: Specific messaging platforms don't work

Solution:

```
# Reconfigure problematic channel
openclaw configure --section channels

# Check channel-specific logs
openclaw channels logs --channel whatsapp

# Test channel connectivity
openclaw message send --target your-phone-number --message "Test"
```

Model Issues

"Model not found" errors

Symptoms: AI model requests fail with model not found

Solution:

```
# Check available models
openclaw models list

# Reset to a known working model
openclaw models set anthropic/claude-sonnet-4-5

# Check model authentication
openclaw models status
```

Ollama connection refused

Symptoms: Local model requests fail

Solution:

```
# Ensure Ollama is running
ollama serve

# Check if model is available
ollama list

# Pull model if missing
ollama pull qwen2.5:14b

# Test direct connection
curl http://localhost:11434/api/generate -d '{"model":"qwen2.5:14b","prompt":"test"}'
```

OpenRouter API key invalid

Symptoms: OpenRouter requests fail with authentication errors

Solution: - Verify API key at openrouter.ai - Check key in config: `openclaw config get env.OPENROUTER_API_KEY` - Update key: `openclaw configure --section models` - Ensure sufficient credits in your OpenRouter account

Context window exceeded

Symptoms: Long conversations fail with context errors

Solutions: - Start fresh session: `/new` - Compact current session: `/compact` - Use model with larger context window - Enable session pruning in config

Slow responses from local models

Symptoms: Local models take very long to respond

Solutions: - Use smaller model: `ollama pull qwen2.5:7b` - Check system resources (CPU, RAM) - Ensure Ollama is using GPU if available - Consider cloud model for better performance

Cron Job Issues

Jobs not firing

Symptoms: Scheduled tasks don't run

Diagnostic:

```
openclaw cron status
openclaw cron list
openclaw gateway status
```

Solutions: - Ensure gateway is running continuously - Check cron is enabled: `openclaw config get cron.enabled` - Verify timezone settings - Test job manually: `openclaw cron run <job-id> --force`

Timeout errors on cron runs

Symptoms: Scheduled jobs fail with timeout

Solution: - Increase timeout in job configuration - Use sub-agents for long-running tasks - Simplify job to focus on essential tasks - Check model provider rate limits

Delivery failures

Symptoms: Cron jobs run but results don't arrive

Solution: - Check delivery target configuration - Verify channel connectivity - Test delivery manually - Check allowlist settings for bot sender

Wrong timezone

Symptoms: Jobs run at unexpected times

Solution:

```
# Check system timezone
date
timedatectl  # Linux
system_profiler SPSoftwareDataType | grep "Time Zone"  # macOS

# Set timezone in cron job
openclaw cron add --name "Daily Report" --every "9:00" --tz "America/New_York" --message "Daily
update"
```

Memory Issues

Memory search returns nothing

Symptoms: Memory search fails to find relevant information

Solution: - Reindex memory: `openclaw memory index` - Check memory files exist in workspace: `ls ~/.openclaw/workspace/memory/` - Verify memory search is configured: `openclaw memory status` - Add content to memory files

Files not being read

Symptoms: Bot can't access or read files

Solution: - Check file permissions - Verify workspace path: `openclaw config get agents.defaults.workspace` - Ensure files are in correct location - Check sandboxing settings if enabled

Workspace files not loading

Symptoms: AGENTS.md, SOUL.md, or other workspace files seem ignored

Solution: - Verify workspace path configuration - Check file encoding (use UTF-8) - Start new session to reload workspace context - Review file permissions

Clean Uninstall Guide

When you need to completely remove OpenClaw from your system, follow these platform-specific procedures:

Windows Uninstall

Using the built-in uninstaller (recommended):

```
# If OpenClaw CLI is still working
openclaw uninstall --all --yes --non-interactive
```

Manual uninstall:

```
# 1. Stop the gateway service
openclaw gateway stop

# 2. Remove the scheduled task
schtasks /Delete /F /TN "OpenClaw Gateway"

# 3. Remove npm package
npm uninstall -g openclaw

# 4. Delete state directory
Remove-Item -Recurse -Force "\$env:USERPROFILE\.openclaw"

# 5. Delete workspace (optional - contains your data)
Remove-Item -Recurse -Force "\$env:USERPROFILE\.openclaw\workspace"

# 6. Clean up task script
Remove-Item -Force "\$env:USERPROFILE\.openclaw\gateway.cmd"
```

Remove Ollama (if installed): - Uninstall from Windows Settings > Apps - Delete `%USERPROFILE%\.ollama` directory

macOS Uninstall

Using the built-in uninstaller (recommended):

```
# If OpenClaw CLI is still working
openclaw uninstall --all --yes --non-interactive
```

Manual uninstall:

```
# 1. Stop and remove LaunchAgent
launchctl bootout gui/\$UID/ai.openclaw.gateway
rm -f ~/Library/LaunchAgents/ai.openclaw.gateway.plist

# 2. Remove npm package
npm uninstall -g openclaw
# or if using pnpm: pnpm remove -g openclaw

# 3. Remove state directory
rm -rf ~/.openclaw

# 4. Remove workspace (optional - contains your data)
rm -rf ~/.openclaw/workspace

# 5. Remove macOS app if installed
rm -rf /Applications/OpenClaw.app

# 6. Clean up any profile-specific installations
rm -f ~/Library/LaunchAgents/ai.openclaw.*.plist
rm -rf ~/.openclaw-*
```

Remove Ollama (if installed):

```
# Uninstall Ollama
rm -rf /usr/local/bin/ollama
rm -rf ~/.ollama
sudo launchctl unload /Library/LaunchDaemons/com.ollama.ollama.plist
sudo rm /Library/LaunchDaemons/com.ollama.ollama.plist
```

Clean up PM2 if used:

```
pm2 delete all
pm2 save
npm uninstall -g pm2
```

Linux Uninstall

Using the built-in uninstaller (recommended):

```
# If OpenClaw CLI is still working
openclaw uninstall --all --yes --non-interactive
```

Manual uninstall:

```
# 1. Stop and disable systemd service
systemctl --user disable --now openclaw-gateway.service
rm -f ~/.config/systemd/user/openclaw-gateway.service
systemctl --user daemon-reload

# 2. Remove npm package
npm uninstall -g openclaw
# or if using pnpm: pnpm remove -g openclaw

# 3. Remove state directory
rm -rf ~/.openclaw

# 4. Remove workspace (optional - contains your data)
rm -rf ~/.openclaw/workspace

# 5. Clean up any profile-specific services
systemctl --user disable --now openclaw-gateway-*.service
rm -f ~/.config/systemd/user/openclaw-gateway-*.service
systemctl --user daemon-reload
```

Remove Ollama (if installed):

```
# Remove Ollama service
sudo systemctl stop ollama
sudo systemctl disable ollama
sudo rm /etc/systemd/system/ollama.service

# Remove Ollama binary and data
sudo rm /usr/local/bin/ollama
sudo rm -rf /usr/share/ollama
rm -rf ~/.ollama
```

Clean up Docker containers (if used):

```
docker stop \$(docker ps -a -q --filter ancestor=openclaw)
docker rm \$(docker ps -a -q --filter ancestor=openclaw)
docker rmi openclaw
```

Verification

After uninstalling, verify removal:

```
# These should return "command not found"
openclaw --version
ollama --version

# These directories should not exist
ls ~/.openclaw
ls ~/.ollama
```

Getting Help

When you've tried the troubleshooting steps and still need assistance:

Community Support

Discord Community - Join the OpenClaw Discord server - Share your `openclaw status --all` output (sensitive data is automatically redacted) - Provide specific error messages and steps to reproduce

GitHub Issues - Search existing issues at github.com/openclaw/openclaw/issues - Create new issue with diagnostic output - Include your operating system and installation method

Documentation - Comprehensive docs at docs.openclaw.ai - Use the search function for specific topics - Check the FAQ section for common questions

Preparing Support Requests

Before asking for help, gather this information:

```
# System information
openclaw status --all

# Model status
openclaw models status

# Recent logs (last 50 lines)
openclaw logs --limit 50

# Doctor report
openclaw doctor
```

This information helps support volunteers diagnose your issue quickly.

Self-Service Debugging

Using Local AI Assistants The fastest way to get unstuck is often using a capable local AI assistant like Claude Code or Codex that can see your machine:

1. Install the hackable version: `curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method git`
2. Ask your AI assistant to help debug using the local source code
3. Share diagnostic command outputs with the assistant
4. Let the AI guide you through step-by-step fixes

Documentation Search All documentation is available locally in the git checkout: - Browse `docs/` directory in your installation - Search for specific error messages - Check command-specific documentation in `docs/cli/`

Remember: Every OpenClaw issue has been seen before. The troubleshooting tools are designed to quickly identify and fix common problems. Stay calm, work through the diagnostics systematically, and you'll get back to building powerful AI workflows.

Appendix — Reference Materials

Your complete quick-reference guide to OpenClaw commands, configurations, and resources.

Complete Environment Template

Here's a comprehensive `.env` template showing every environment variable used in OpenClaw Personal Edition. Copy this to `~/.openclaw/.env` and uncomment the variables you need:

```
# ~/.openclaw/.env - OpenClaw Environment Variables

# ═══ Core Configuration ═══
# OPENCLAW_STATE_DIR=/custom/path/.openclaw
# OPENCLAW_CONFIG_PATH=/custom/path/openclaw.json
# OPENCLAW_WORKSPACE_DIR=/custom/path/workspace
# OPENCLAW_PROFILE=production
# OPENCLAW_GATEWAY_PORT=18789
# OPENCLAW_GATEWAY_TOKEN=your-secure-token-here
# OPENCLAW_GATEWAY_PASSWORD=your-secure-password-here
# OPENCLAW_SKIP_CRON=false
# OPENCLAW_HIDE_BANNER=false
# OPENCLAW_LOAD_SHELL_ENV=true
# OPENCLAW_SHELL_ENV_TIMEOUT_MS=15000

# ═══ Model Provider API Keys ═══
# Anthropic (API key method)
# ANTHROPIC_API_KEY=sk-ant-api03-your-api-key-here

# OpenAI (API key method)
# OPENAI_API_KEY=sk-your-openai-api-key-here

# OpenRouter (aggregated models)
# OPENROUTER_API_KEY=sk-or-your-openrouter-key-here

# Google Gemini
# GEMINI_API_KEY=your-gemini-api-key-here
# GOOGLE_API_KEY=your-google-api-key-here  # Alternative name

# GitHub Copilot
# COPILOT_GITHUB_TOKEN=ghp_your-github-token-here
# GITHUB_TOKEN=ghp_your-github-token-here  # Alternative name
# GH_TOKEN=ghp_your-github-token-here      # Alternative name

# MiniMax
# MINIMAX_API_KEY=your-minimax-api-key-here
# MINIMAX_GROUP_ID=your-minimax-group-id-here

# Mistral AI
# MISTRAL_API_KEY=your-mistral-api-key-here

# Moonshot AI
# MOONSHOT_API_KEY=sk-your-moonshot-api-key-here

# Kimi (Moonshot)
# KIMI_API_KEY=your-kimi-api-key-here

# Z.AI (GLM models)
# ZAI_API_KEY=your-zai-api-key-here
```

```
# Voyage AI (embeddings)
# VOYAGE_API_KEY=pa-your-voyage-api-key-here

# Custom model endpoints
# CUSTOM_API_KEY=your-custom-api-key-here
# CUSTOM_BASE_URL=https://your-custom-endpoint.com/v1

# ═══ Chat Platform Tokens ═══
# Telegram Bot Token
# TELEGRAM_BOT_TOKEN=1234567890:ABCDEFghijklmnopqrstuvwxyz123456789

# Discord Bot Token
# DISCORD_BOT_TOKEN=MTIzNDU2Nzg5MDEyMzQ1Njc4OTA.your-discord-token

# Slack Bot Token
# SLACK_BOT_TOKEN=xoxb-your-slack-bot-token
# SLACK_APP_TOKEN=xapp-your-slack-app-token

# Google Chat
# GOOGLE_CHAT_WEBHOOK_URL=https://chat.googleapis.com/v1/spaces/your-webhook
# GOOGLE_SERVICE_ACCOUNT_JSON={"type":"service_account",...}

# Microsoft Teams
# MSTEAMS_WEBHOOK_URL=https://your-org.webhook.office.com/webhookb2/your-webhook

# ═══ Web Tools ═══
# Brave Search (for web_search tool)
# BRAVE_API_KEY=BSA-your-brave-search-api-key-here

# ═══ Database & Storage ═══
# Memory search embeddings
# OPENAI_API_KEY=sk-your-openai-key-for-embeddings  # If using OpenAI embeddings
# GEMINI_API_KEY=your-gemini-key-for-embeddings     # If using Gemini embeddings

# ═══ Development & Debugging ═══
# OPENCLAW_LOG_LEVEL=info  # debug, info, warn, error
# OPENCLAW_LOG_FILE=/custom/path/openclaw.log
# OPENCLAW_VERBOSE=false
# OPENCLAW_DEBUG=false

# ═══ Node.js & Runtime ═══
# NODE_ENV=production
# NODE_OPTIONS="--max-old-space-size=4096"

# ═══ Proxy & Network ═══
# HTTP_PROXY=http://proxy.company.com:8080
# HTTPS_PROXY=https://proxy.company.com:8080
# NO_PROXY=localhost,127.0.0.1,.local

# ═══ Ollama (Local Models) ═══
# OLLAMA_HOST=http://localhost:11434
# OLLAMA_API_KEY=your-ollama-api-key-if-required

# ═══ Browser Automation ═══
# PLAYWRIGHT_BROWSERS_PATH=/custom/path/browsers
# CHROME_EXECUTABLE_PATH=/custom/path/chrome
# CHROMIUM_EXECUTABLE_PATH=/custom/path/chromium

# ═══ Security ═══
# OPENCLAW_ALLOW_UNSAFE_EVAL=false
```

```
# OPENCLAW_DISABLE_SANDBOX=false
# OPENCLAW_EXEC_ALLOWLIST="git,npm,pnpm,node,python"
```

Important Security Notes: - Never commit real API keys to version control - Use restrictive file permissions: `chmod 600 ~/.openclaw/.env` - Rotate keys regularly, especially if shared with others - Test keys with minimal permissions first

OpenClaw CLI Command Reference

Gateway Management

Command	Description	Example
`openclaw gateway status`	Check gateway health and connectivity	`openclaw gateway status --json`
`openclaw gateway start`	Start the gateway service	`openclaw gateway start`
`openclaw gateway stop`	Stop the gateway service	`openclaw gateway stop`
`openclaw gateway restart`	Restart the gateway service	`openclaw gateway restart`
`openclaw gateway run`	Run gateway in foreground	`openclaw gateway run --port 18790`
`openclaw gateway install`	Install gateway as system service	`openclaw gateway install --force`
`openclaw gateway uninstall`	Remove gateway system service	`openclaw gateway uninstall`

System Diagnostics

Command	Description	Example
`openclaw status`	Show system overview	`openclaw status --all`
`openclaw status --deep`	Run deep system probes	`openclaw status --deep --json`
`openclaw doctor`	Automated health checks and repairs	`openclaw doctor --yes`
`openclaw health`	Get health from running gateway	`openclaw health --json`
`openclaw logs`	View recent logs	`openclaw logs --follow --limit 100`

Configuration

Command	Description	Example
`openclaw configure`	Interactive configuration wizard	`openclaw configure --section models`
`openclaw config get`	Read configuration value	`openclaw config get gateway.port`
`openclaw config set`	Write configuration value	`openclaw config set gateway.port 18790`
`openclaw config unset`	Remove configuration value	`openclaw config unset channels.telegram`
`openclaw config file`	Show config file path	`openclaw config file`
`openclaw config validate`	Validate configuration	`openclaw config validate --json`

Model Management

Command	Description	Example
`openclaw models status`	Check model authentication	`openclaw models status --probe`
`openclaw models list`	List available models	`openclaw models list --all`
`openclaw models set`	Set default model	`openclaw models set anthropic/claude-sonnet-4-5`
`openclaw models auth add`	Add model authentication	`openclaw models auth add`
`openclaw models scan`	Scan for available models	`openclaw models scan --provider anthropic`

Channel Management

Command	Description	Example
`openclaw channels list`	List configured channels	`openclaw channels list --json`
`openclaw channels status`	Check channel connectivity	`openclaw channels status --probe`
`openclaw channels add`	Add new channel	`openclaw channels add --channel telegram`
`openclaw channels remove`	Remove channel	`openclaw channels remove --channel telegram`
`openclaw channels logs`	View channel logs	`openclaw channels logs --channel telegram`

Session Management

Command	Description	Example
`openclaw sessions`	List active sessions	`openclaw sessions --active 60`
`openclaw agent`	Send message to agent	`openclaw agent --message "Hello" --deliver`
`openclaw agents list`	List configured agents	`openclaw agents list --bindings`
`openclaw agents add`	Add new agent	`openclaw agents add mybot --workspace ~/mybot`

Cron & Automation

Command	Description	Example
`openclaw cron status`	Check cron scheduler status	`openclaw cron status`
`openclaw cron list`	List scheduled jobs	`openclaw cron list --all`
`openclaw cron add`	Create new scheduled job	`openclaw cron add --name "Daily" --every "9:00" --message "Daily update"`
`openclaw cron run`	Run job manually	`openclaw cron run job-123 --force`
`openclaw cron runs`	Show job execution history	`openclaw cron runs --id job-123 --limit 10`

Memory Management

Command	Description	Example
`openclaw memory status`	Check memory index status	`openclaw memory status`
`openclaw memory index`	Rebuild memory index	`openclaw memory index`
`openclaw memory search`	Search memory semantically	`openclaw memory search "project deadlines"`

Device & Node Management

Command	Description	Example
`openclaw nodes status`	List paired devices	`openclaw nodes status --connected`
`openclaw devices list`	Show device pairings	`openclaw devices list`
`openclaw devices approve`	Approve device pairing	`openclaw devices approve req-123`
`openclaw pairing list`	Show pending pairing requests	`openclaw pairing list telegram`
`openclaw pairing approve`	Approve pairing request	`openclaw pairing approve telegram ABC123`

Setup & Maintenance

Command	Description	Example
`openclaw onboard`	Run setup wizard	`openclaw onboard --install-daemon`
`openclaw setup`	Initialize configuration	`openclaw setup --wizard`
`openclaw update`	Update OpenClaw	`openclaw update --channel stable`
`openclaw reset`	Reset configuration	`openclaw reset --scope full --yes`
`openclaw uninstall`	Complete uninstall	`openclaw uninstall --all --yes`

Browser Automation

Command	Description	Example
`openclaw browser status`	Check browser automation status	`openclaw browser status`
`openclaw browser screenshot`	Take page screenshot	`openclaw browser screenshot --full-page`
`openclaw browser click`	Click page element	`openclaw browser click "button[type=submit]"`
`openclaw browser type`	Type in form field	`openclaw browser type "#email" "user\@example.com"`

Terminal UI

Command	Description	Example
`openclaw tui`	Launch terminal interface	`openclaw tui --session main`
`openclaw dashboard`	Open web dashboard	`openclaw dashboard`

Cron Expression Quick Reference

Cron expressions control when scheduled jobs run. Format: `minute hour day month weekday`

Field Meanings

Field	Range	Description
Minute	0-59	Minute of the hour
Hour	0-23	Hour of the day (24-hour format)
Day	1-31	Day of the month
Month	1-12	Month of the year
Weekday	0-7	Day of the week (0 and 7 = Sunday)

Special Characters

Character	Meaning	Example
`*`	Any value	`* * * * *` = every minute
`,`	List values	`1,15,30 * * * *` = minutes 1, 15, 30
`-`	Range of values	`1-5 * * * *` = minutes 1 through 5
`/`	Step values	`*/15 * * * *` = every 15 minutes

Common Patterns

Expression	Description
`0 9 * * *`	Daily at 9:00 AM
`0 9 * * 1-5`	Weekdays at 9:00 AM
`0 9 1 * *`	First day of every month at 9:00 AM
`*/30 * * * *`	Every 30 minutes
`0 */2 * * *`	Every 2 hours
`15 14 1 * *`	Monthly at 2:15 PM on the 1st
`0 22 * * 1-5`	Weekdays at 10:00 PM
`0 0 * * 0`	Every Sunday at midnight
`0 12 */2 * *`	Every 2 days at noon
`30 6 * * 1`	Every Monday at 6:30 AM
`0 8,20 * * *`	Daily at 8:00 AM and 8:00 PM
`*/15 9-17 * * 1-5`	Every 15 minutes, 9 AM to 5 PM, weekdays
`0 0 1 1 *`	New Year's Day at midnight
`0 9-17 * * *`	Every hour from 9 AM to 5 PM
`0 6 * * 1-5`	Weekdays at 6:00 AM

Alternative Format Examples

OpenClaw also supports human-readable formats:

```
# These are equivalent
--cron "0 9 * * *"
--every "9:00"
--at "09:00"

# These are equivalent
--cron "0 9 * * 1-5"
--every "weekdays at 9:00"

# These are equivalent
--cron "*/30 * * * *"
--every "30 minutes"
```

Glossary of Terms

Agent — An AI assistant instance with its own workspace, memory, and configuration. OpenClaw can run multiple agents with different personalities and capabilities.

Allowlist — Security control that specifies which users, channels, or commands are permitted. Only explicitly allowed items can access the system.

API Key — Authentication credential that proves your identity to model providers like OpenAI or Anthropic. Required for accessing hosted AI models.

Authentication Profile — Stored credential information (API keys, OAuth tokens) for accessing external services. Each profile can store different accounts or permission levels.

Channel — Communication platform where your agent operates (Telegram, Discord, WhatsApp, etc.). Multiple channels can connect to the same agent.

Compaction — Process of summarizing old conversation history to save memory while preserving important context. Helps manage long conversations within model limits.

Context Window — Maximum amount of text (measured in tokens) that an AI model can process at once. Larger windows allow for longer conversations but cost more.

Cron — Scheduled task system that runs jobs automatically at specified times. Named after the Unix cron scheduler with similar syntax.

Gateway — Core OpenClaw service that coordinates between chat channels, AI models, and tools. Runs continuously in the background.

Heartbeat — Periodic check-in system where the agent proactively monitors for updates and sends notifications. Runs every 30 minutes by default.

Memory — Persistent storage system for important information. Includes daily logs and long-term curated memories stored as Markdown files.

Model — AI system that generates text responses. Can be hosted (OpenAI, Anthropic) or local (Ollama). Different models have different capabilities and costs.

Node — Remote device (Mac, iPhone, Android) paired with your gateway to provide local tools like camera, screen capture, and file access.

OAuth — Authentication method that uses secure tokens instead of passwords. Often provides access to subscription benefits without separate API billing.

Provider — Service that hosts AI models (Anthropic, OpenAI, Google) or communication channels (Telegram, Discord). Each requires different authentication.

Sandbox — Isolated environment where potentially dangerous code runs safely. Prevents agent actions from affecting your main system.

Session — Individual conversation thread with persistent context and memory. Each chat or group typically gets its own session.

Skill — Specialized capability package that adds new tools and knowledge to your agent. Can be installed from ClawHub or created custom.

Sub-agent — Temporary agent spawned for specific tasks. Runs in parallel with your main conversation to handle complex or time-consuming work.

Token — Unit of measurement for AI model usage. Roughly equivalent to words or parts of words. More tokens = higher costs and longer processing time.

Tool — Function your agent can execute, like searching the web, reading files, or controlling browsers. Tools extend agent capabilities beyond just text generation.

Webhook — URL endpoint that receives automatic notifications when events occur. Used for integrations like Gmail alerts or calendar reminders.

Workspace — Folder containing your agent's files, memory, and configuration. Acts as the agent's "home directory" for persistent data.

Resource Links

Official Resources

OpenClaw Documentation - Main docs: docs.openclaw.ai - Getting started: docs.openclaw.ai/start - Troubleshooting: docs.openclaw.ai/help/troubleshooting - FAQ: docs.openclaw.ai/help/faq

Source Code & Support - GitHub repository: github.com/openclaw/openclaw - Issue tracker: github.com/openclaw/openclaw/issues - Discussions: github.com/openclaw/openclaw/discussions

Community - Discord server: discord.com/invite/clawd - Skill marketplace: clawhub.com - Installation script: openclaw.ai/install.sh

Model Providers

Anthropic (Claude) - Console: console.anthropic.com - API documentation: docs.anthropic.com - Claude Code (setup-token): Install `claude` CLI

OpenAI - API platform: platform.openai.com - API documentation: platform.openai.com/docs - Codex OAuth: openai.com/codex

Google Gemini - API access: makersuite.google.com - Documentation: ai.google.dev

OpenRouter (Multiple Models) - Dashboard: openrouter.ai - Model selection: openrouter.ai/models - API docs: openrouter.ai/docs

Local Models

Ollama - Website: ollama.com - Model library: ollama.com/library - GitHub: github.com/ollama/ollama

LM Studio - Website: lmstudio.ai - Model downloads: lmstudio.ai/models

Platform Documentation

Node.js - Download: nodejs.org - Documentation: nodejs.org/docs

npm Package Manager - Website: npmjs.com - Documentation: docs.npmjs.com

Chat Platform Setup

Telegram Bot API - Bot creation: Message @BotFather on Telegram - Documentation: core.telegram.org/bots

Discord Developer Portal - Bot creation: discord.com/developers/applications - Documentation: discord.com/developers/docs

WhatsApp Business API - Setup guide in OpenClaw docs - Official docs: developers.facebook.com/docs/whatsapp

Quick Reference Card

Essential Commands

```
# Health check
openclaw status && openclaw gateway status

# Start/stop
openclaw gateway start
openclaw gateway stop

# Emergency reset
openclaw doctor
openclaw reset --scope config --yes

# View activity
openclaw logs --follow

# Configure
openclaw configure
```

Chat Commands

```
/status ΓÇö System status
/new ΓÇö Start fresh session
/model sonnet ΓÇö Switch AI model
/deliver on ΓÇö Enable auto-delivery
/compact ΓÇö Summarize conversation
```

Common Config Paths

```
# Main config
~/.openclaw/openclaw.json

# Environment variables
~/.openclaw/.env

# Workspace (your files)
~/.openclaw/workspace/

# Logs
/tmp/openclaw/openclaw-YYYY-MM-DD.log
```

Default Ports

- Gateway: 18789
- Ollama: 11434
- Development: 18790

Remember: OpenClaw is designed to be helpful and reliable. When something goes wrong, the diagnostic tools will guide you to a solution. Keep this reference handy for quick lookups while you build your AI-powered workflows!